

Software system engineering

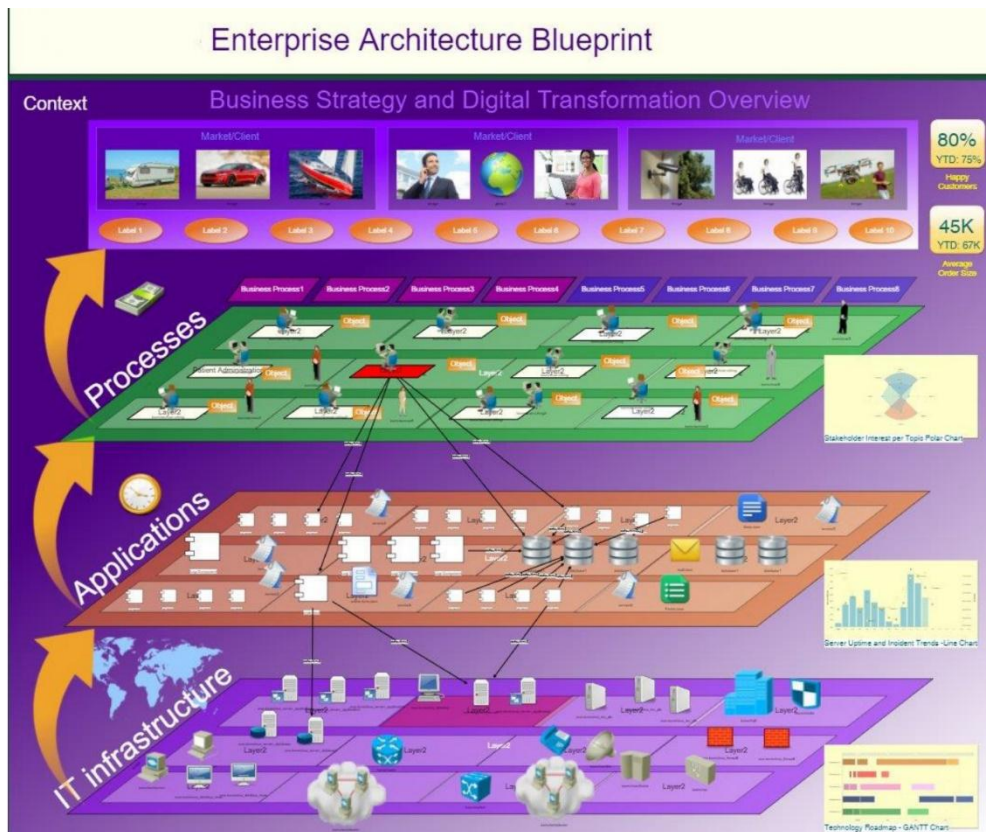
Francesco Taverna

Indice

1.	Introduzione	2
1.1.	Orchestrazione e coreografia	14
1.2.	Esercizio.....	28
2.	Prodotti Software (Alfeo)	29
3.	UML.....	40
3.1.	Workflow dei requisiti	45
3.2.	Workflow di Analisi	52
4.	Testing Workflow.....	74
5.	Esempio Modeling da linguaggio naturale	89
6.	Software factory	95
7.	Python (Alfeo)	100
7.1.	Code quality	111
7.2.	Pacchetti Python	113
7.3.	Module testing and communication	128
7.3.1.	Comunicazione con i moduli	133
8.	Consigli	138
9.	Dettagli generali sul progetto	139
10.	Modellazione task	140
11.	Project Versioning (Alfeo)	142
12.	Dalle performance fino alla productivity evaluation	152

1. Introduzione

- Sviluppare progetto in classe, ci seguirà.
- L'implementazione è la più piccola parte
- Menziona la classe di riferimento quando scrivi un feedback nel sito
- Prime 2 settimane di preparazione per iniziare il progetto



È presente una gerarchia nel mondo delle applicazioni. In una compagnia ci sono specifiche regole in ogni attività, i processi sono catene di attività, ci sarà una catena di interazioni, tra persone e vari software. I prodotti software sono sviluppati anche in periodi diversi, non tutti insieme. Quando un ospedale compra un nuovo software questo deve essere connesso con gli altri. Abbiamo una grande varietà di applicazioni, non c'è più un singolo database, ci sono vari vincoli sui dati, non puoi mischiare tutti i dati mettendoli in un solo database. Noi ci focalizzeremo sulle applicazioni, useremo Agile, aggiungeremo ogni volta nuovi componenti al nostro software. Il nostro software avrà un limite di tempo per svilupparlo.

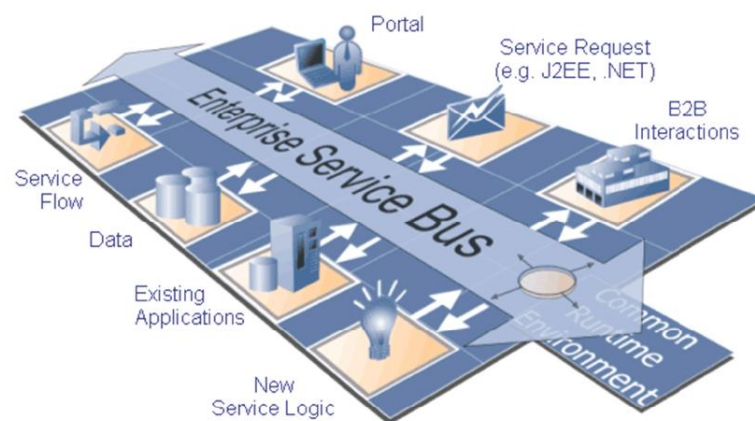
- ✓ The world is becoming a huge, system interconnected by the Internet



- ✓ Each element (object, service, human,...) plays different roles

I vari dispositivi hanno bisogno di coordinazione.

- ✓ Enterprises look like enormous processors. Example: enterprise service bus



- ✓ We need **new languages** to model the flow of work (workflow) in this service-oriented environment.
- ✓ Object-Oriented programming is not sufficient to model the orchestration or collaboration business logic

I differenti tool possono utilizzare un Enterprise Service Bus per connetterli nel contesto di un service oriented computing.

“DevOps”

- A methodology combining **software DEvelopment** and **IT OPerationS**
- It is supported by a **Toolchain**: a set of tools for the development and delivery process:
 - *Coding*: code development and review, source code management tools, code merging.
 - *Building*: integration tools, build status.
 - *Testing*: testing tools providing timely feedback on business risks.
 - *Packaging*: artifact repository, application pre-deployment staging.
 - *Releasing*: change management, release approvals, release automation.
 - *Configuring* – infrastructure configuration and management.
 - *Monitoring* – applications performance monitoring, end-user experience.

Le IT Operations sono quelle operazioni per far funzionare il software. Abbiamo bisogno di una Toolchain per gestire il project management. Vedremo l'intero life cycle del nostro prodotto.

Basic ontology for process orientation approach

- Companies compete on the basis of business processes: simple/complex, flexible, robust, clear or ambiguous, partially automated, manual, ad-hoc,...
- **Business Process**: a collection of interrelated **activities**, initiated in response to a triggering **event**, achieving a specific discrete **result** for the **customer** and other stakeholders of the process.
- An **activity** is work that might be named as a single unit but can involve multiple actors, each separately making a contribution.
- A **task** is an actionable item that needs to be performed to complete an activity. An activity may consist of more tasks. A task consists of **steps**.
- A **result** is an individually identifiable and countable entity
- A **customer** is the recipient of beneficiary of the result (person, organization, broad marketplace, ...)
- An **event** is an action, a date or time reached, a condition (rule).



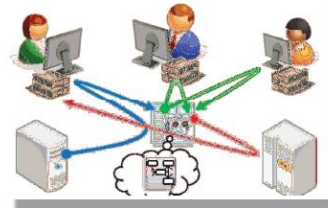
Il trigger dell'evento è l'inizio del processo: un paziente che inizia una richiesta è un trigger di un evento per esempio. Non c'è un processo senza risultato, esso è l'obiettivo finale, il progetto è focalizzato sull'ottenere il risultato.

Una attività è composta da vari task, un task è composto da molti step. Quando hai uno use case hai una sequenza di step di un utente che interagisce con l'ambiente (fai questo, fai quell'altro) come fosse un manuale utente. Le attività messe insieme compongono il processo.

- *Imperative approach*: dare comandi al computer attraverso un programma Python o Java, per esempio ordinare una lista di nomi, tramite una sequenza di istruzioni. Svolto dal software engineer.
- *Declarative approach*: le persone ti dicono come fare il prodotto, prendi la logica e la converti in un workflow. Le persone dicono cosa vogliono e tu implementi la catena di attività descritta da loro, sei molto più vincolato. Questo è sviluppato da un management engineer che effettua le attività formali.

Basic ontology for process orientation approach

- Business Process (BP) Analysis includes a broad meaning encompassing modeling, simulation, diagnosis, verification, performance of BP.
- A **workflow** is a (partial) automation of a business process. In a workflow ("flow of work") resources, information, activities, follow a coordinated control flow established by a **BP modeling language**.
- **Business Process Modeling** is the creative act of producing a workflow from a business process.
- A BP modeling language is very different from a programming language:



Automation paradigm	Description language	Execution language	Core runtime	Complexity
Service oriented computing	UML/BPMN	XML	Workflow Management System	+++
Object oriented programming	Java/C#/C++	Byte code	Virtual Machine	++
Procedural programming	C	Assembler	CPU	+

Cimino – Introduction to workflow modeling – 6

- Il *Procedural programming* prevede un comportamento deterministico dunque è semplice.
- *Object oriented* invece prevede concetti simili alle attività umane, abbiamo piattaforme differenti, per cui risulta più complicato. Si hanno differenti linguaggi, ogni cosa è compilata in byte code, differente per ogni linguaggio, c'è un livello di astrazione dipendentemente dall'HW. Abbiamo un motore virtualizzato che esegue il byte code.
- Il service oriented usa UML e BPMN. Il runtime è un motore per eseguire workflow.

Business Process Model and Notation (BPMN 2.0)

- BPMN is...

- ✓ *human-readable*: a standard visual notation for modeling business processes;
- ✓ *accessible*: easy to understand for various roles: who analyzes and defines processes, who leads the technological implementation, who is responsible for management and control;
- ✓ *machine-readable*: a notation serializable to XML for process execution (e.g. WS-BPEL 2, SOA environments).

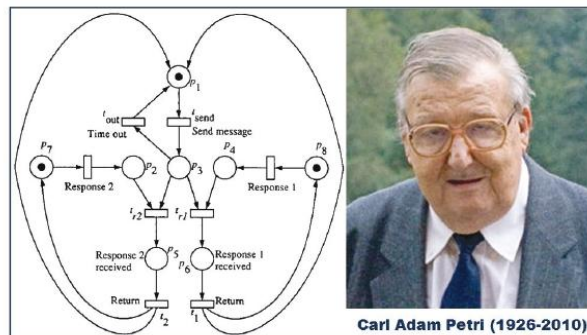
- BPMN is not...

- ✓ a language for representing data flows and object flows, although this can be done at a certain abstraction level;
- ✓ a notation to represent structures, functional decompositions, data models, organization strategies, business rules.

Quando hai un workflow devi avere differenti esperti per questo. Serve una notazione eseguibile.

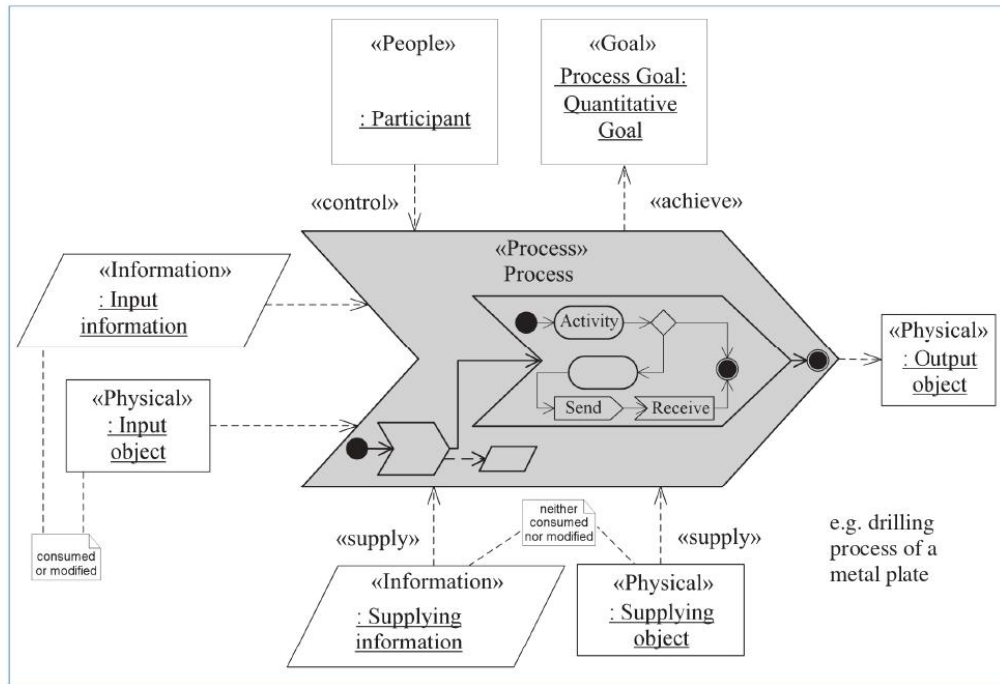
- “Relatives” of BPMN (languages for workflow-based analysis)

- ✓ *Petri Nets* (1962): formal language to model distributed systems, usable by computer scientist and designers of specialized software. It consists of a visual representation and a corresponding mathematical notation (graphs), allowing advanced analyses such as validation, verification (e.g.. *soundness* to identify deadlock and livelock)



- ✓ *UML Activity Diagram* (OMG, 1997): language for visual modeling for the object-oriented paradigm, usable by software engineers. The extended UML of Eriksson e Penker (2000) is suitable for business process modeling, and usable also by business level (non-technical) roles.

Generic example of a process diagram, with **UML extension of Eriksson-Penker**



Cosa dobbiamo fare quando sviluppiamo un software? Vediamo l'esempio in figura. Immaginiamoci un piatto di metallo in una fabbrica che necessita di essere lavorato per ottenere un buco nel mezzo.

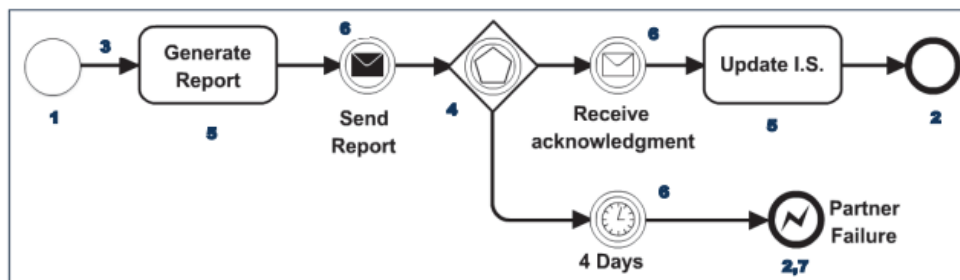
- *«Information»* può essere la grandezza del buco da fare sul piatto
- *«Physical»* è l'oggetto fisico da fornire, un piatto di metallo.
- *Supplying information* può essere un manuale.

Le entità in input sono consumate, quando faccio il buco il piatto è **consumato**, quando "attacchi" le informazioni al piatto tutto quello è consumato.

- L'output è un oggetto fisico (piatto col il buco). Tutto ciò è molto diverso di un basico di un modello input-output.
- Dentro il workflow puoi vedere alcuni eventi, rappresentati dai cerchi neri. Quella iniziale è il trigger event, poi si hanno intermediate event ed infine il end event.
- Activity è un task nel BPMN.
- Il rombo è un gateway, è una sorta di "if", simile alle eccezioni nel software. Puoi creare una gerarchia di attività.

Business Process Model and Notation (BPMN 2.0)

- *BPMN* (OMG, 2005) is specialized in representing the behavior of processes concerning the **control flow**, with the concept of a *token* traversing the process structure.
- A *Start Event*¹ generates a token that will be consumed by an *End Event*². The path of tokens is managed by a network of *Sequence Flow*³, *Gateway*⁴, *Activity*⁵ and *Intermediate Event*⁶, within the process.
- *Race pattern*: there is a race between two intermediate events⁶ after the event-based gateway⁴, i.e., “receive acknowledgment” and “four days elapsed”. When the message is not received before the 4 days cutoff date, the execution is diverted from normal execution flow in order to raise a ‘Partner Failure’ error (throw semantic)⁷.

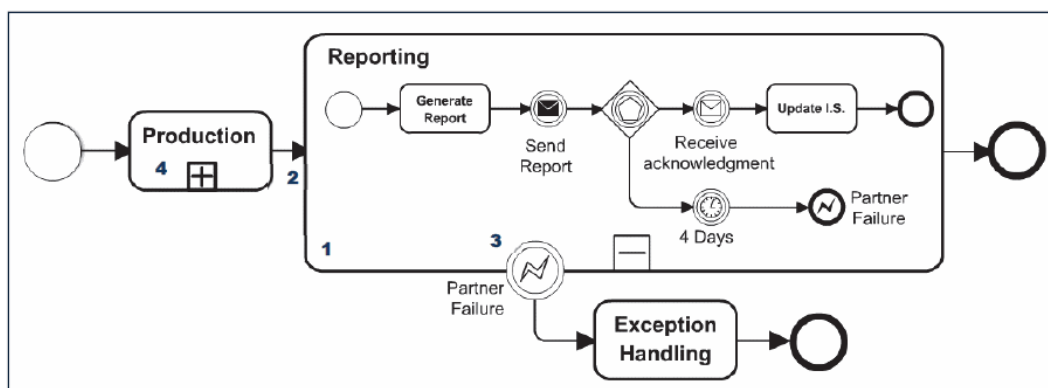


Quando esegui il software il puntatore (un token rosso) si muove. Hai un pezzo HW che si muove, stessa cosa nei server, hai vari software che si muovono. Nel token non ci sono dati ma solo una sequenza di attività.

- Pallino con bordo sottile è l'evento iniziale
- Pallino con bordo spesso è l'evento finale
- Doppio bordo con icona dentro è l'evento intermedio.
- Rombo è un gateway, permette di scegliere quale ramo.

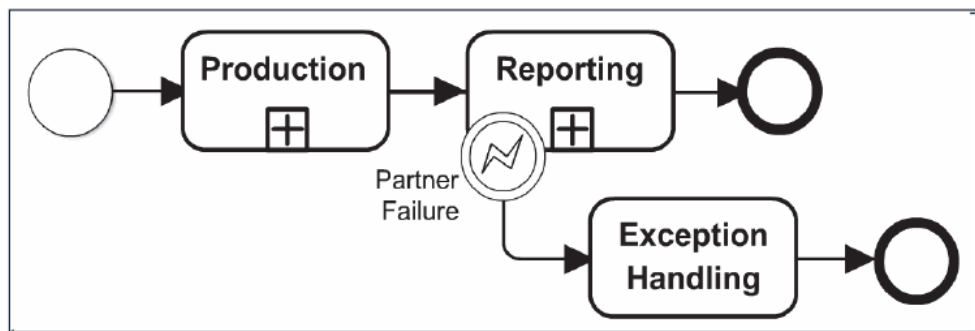
Il percorso è stabilito dall'evento (messaggio per esempio). L'evento finale distrugge il token. *Partner failure* è un evento di errore, dunque hai un'eccezione.

- *Interruption pattern*: the previous model is reused and embedded as the ‘Reporting’ sub-process (expanded notation)¹. An intermediate event (‘catch’ semantic) has been added to the boundary of that sub-process. The model contains also a collapsed sub-process (details are not visible), denoted by a “plus” sign⁴.
- The ‘Partner failure’ event (‘catch’ semantic) gets activated when the execution points reaches the ‘Reporting’ sub-process², and gets un-activated when the ‘Reporting’ sub-process completes successfully (i.e., an end event is reached).



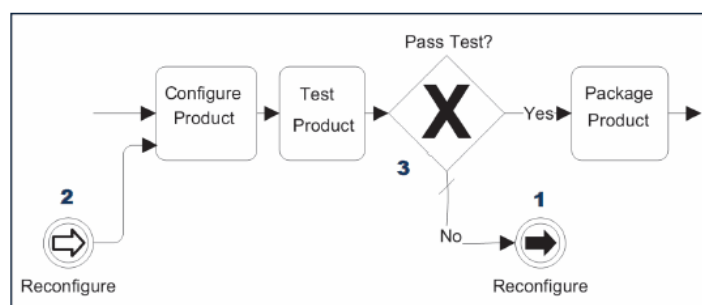
Abbiamo un modello più complesso: dentro il modello è lo stesso di quello visto in precedenza. Dopo quattro giorni, hai la *handling exception activity*.

- Note how the partner failure event ('send' semantic) raised by the reporting sub-process is caught by the 'Partner failure' event³ ('receive' semantic) and how the execution flow get diverted from that point.
- Note that if a sub-process of a diagram is expanded in the diagram, the elements inside the sub-process cannot be connected to elements outside the sub-process.
- BPMN allows the *structured modeling* of processes, i.e., views at different levels of abstraction: from the level "0" (the least amount of detail) the processes are decomposed into sub-processes, up to activities (that are atomic, the most amount of detail). For instance, in the IBM methodology the analysis stops at the level "3".

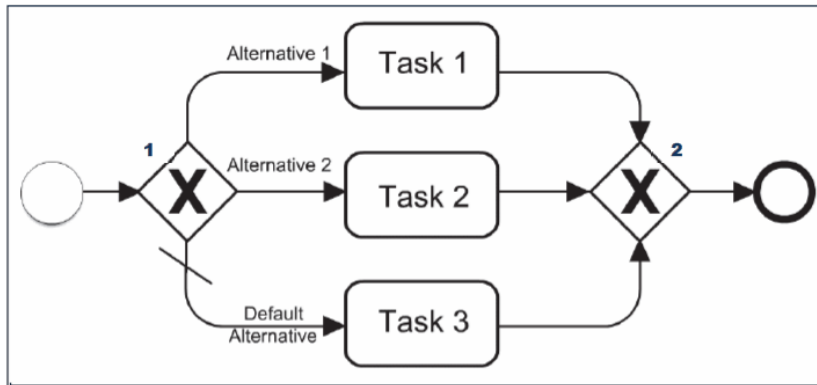


Ora abbiamo nascosto l'implementazione. Possiamo sviluppare differenti livelli di astrazione, in genere si hanno 3 livelli: 0,1 e 2, dipende dalla complessità.

- At the level 0 process is a high-level *not executable* process. Level 0 is modeled for the purpose of documenting process behavior at a modeler-defined level of detail. Thus, information needed for execution, such as formal condition expressions, are typically not included. In contrast, an *executable* process is modeled for the purpose of being executed (e.g. a WS-BPEL process).
- BPMN allows process *segmentation*, at a given level, to create different modular segments. For instance, in the IBM methodology it is recommended that the maximum number of process activities per page should be *six*.
- The *off-page connector*, generally used for printing, is an object showing where a Sequence Flow leaves one page and then restarts on the next page. A *Link Intermediate Event* with throw¹ and catch² semantic can be used as an Off-Page Connector. In figure, the flow of the exclusive gateway³ labeled with "No" leads to come back, and then to a cycle.

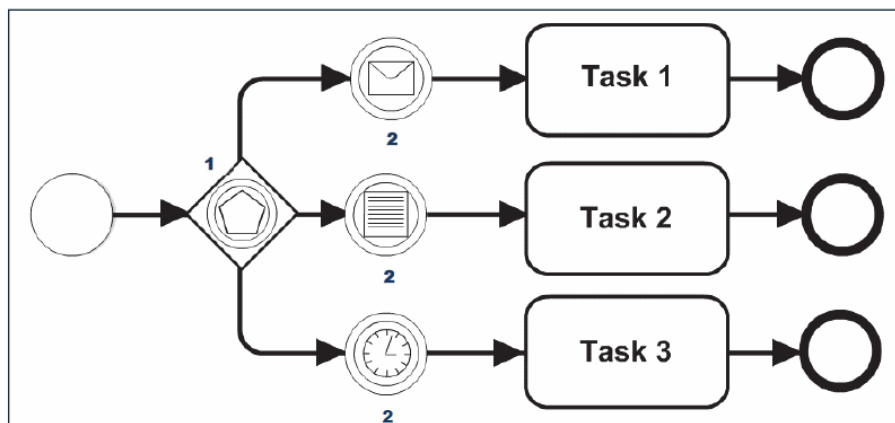


- Gateways are decisions points used to constrain the execution flow, fork an execution point into several or merge several into one. A gateway is represented by a diamond and the kind of a gateway is specified by a marker.
- An *exclusive gateway* can be used as a decision point¹ where several outgoing sequence flows are possible. Such flows are all constrained by a condition allowing only one of them to be used by a token. Such a condition will be evaluated based on the process data. The gateway can be also used as a way to merge² several sequence flows into one. The incoming token moves straight through the gateway and goes on.



È chiamato “scheme alternative”, evita che il token si intrappoli senza possibilità di movimento (deadlock). In questo esempio il diamante con la X è detto **exclusive gateway**, corrisponde allo switch-case nel c++, solo una delle alternative può essere utilizzata da un token. Nel seguente caso, in base al tipo di evento generato, il messaggio viene inoltrato al task incaricato di gestire l’evento.

- The *Exclusive Event Based gateway*¹ is similar to the Exclusive Data Based gateway. The only difference is that, instead of evaluating a set of alternatives to determine only one outgoing flow, the event based gateway will start a race between the different events² the process might receive, the first one to be received wins the race and that determines which outgoing sequence flow should be used.

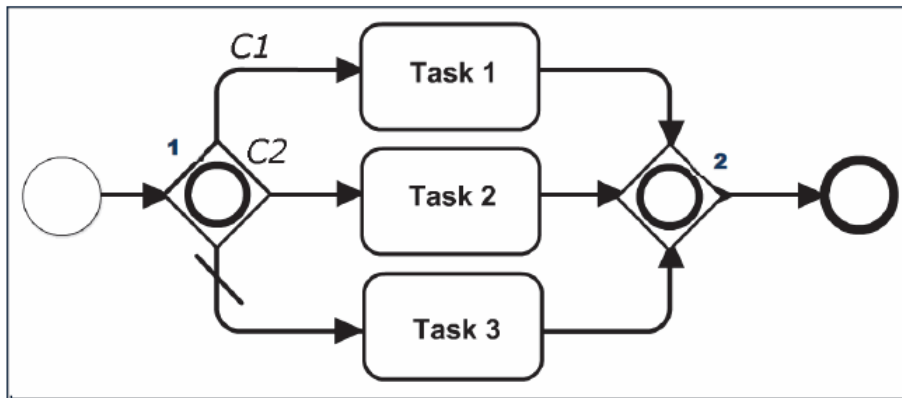


 <http://www.iet.unipi.it/m.cimino/scom/res/mov05>

- Exercise: identify the event before task 2 via the BPMN poster.

Gateways

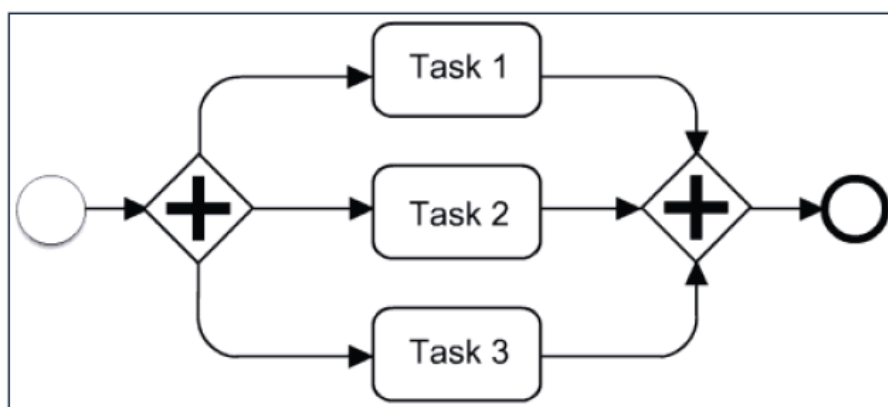
- An *Inclusive Gateway*¹ can be used as a decision point where several outgoing sequence flows are possible, they are all constrained by conditions, each outgoing sequence flow with a condition evaluated as being true will be followed. Effectively it might spawn several execution points.
- Used as a merge² the Inclusive Gateway will synchronize all the execution points produced upstream but at most one for each incoming Sequence Flow



Il diamante con il cerchio è detto **inclusive gateway**, in questo caso sia C1 che C2 possono essere vere allo stesso tempo, ogni flusso sequenziale con una condizione valutata come true verrà seguita. Se C1 e C2 non sono vere, viene seguito il percorso che porta al task 3.

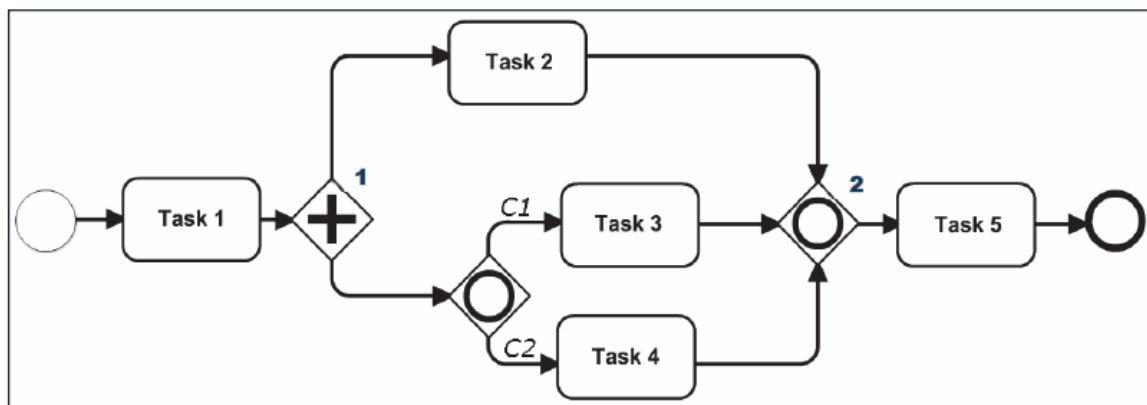
Il diamante con il + è detto **parallel gateway**, in questo caso abbiamo 3 token in parallelo che si sincronizzano tra loro.

- A gateway's exit can also be marked with a small diagonal slash as *default* sequence flow. It will be selected automatically, (only) if no condition of the other sequence flows is true. This ensures the actual selection of at least one sequence flow.
- A *Parallel Gateway*¹ provides a mechanism to fork and synchronize flows. There are no conditions associated to this gateway.



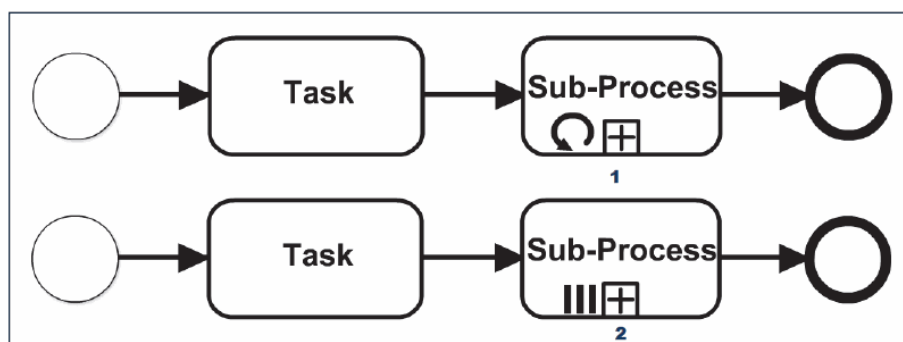
In questo esempio sotto combiniamo diversi gateway. Se sostituiamo il secondo inclusive gateway con un parallel gateway, il comportamento è diverso perchè se ad esempio C1 è falso, abbiamo un token che proviene dal task 2 e uno che proviene dal task 4, però il parallel gateway attende che tutti i token arrivino, quindi si genera deadlock perché attende l'arrivo del risultato del task 3.

- In the following model, notice how the first inclusive gateway produces two tokens because both conditions 'C1' and 'C2' are evaluated as being true. The second inclusive gateway² will not only synchronize the token produced by the upstream inclusive gateway, but also the one coming from the upstream parallel gateway
- Exercise: what happens if the second inclusive gateway is replaced by a parallel gateway? Consider a scenario in which C1 (or C2) is false.

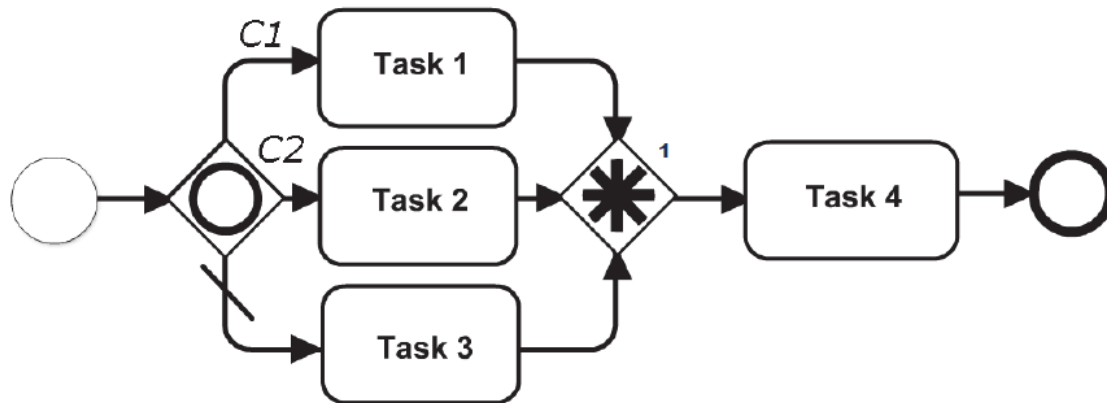


Se abbiamo un comportamento simile ad un loop in un processo possiamo rappresentarlo come nel primo modo in figura sotto, mentre se abbiamo dei task in parallelo, li rappresentiamo nel secondo modo in figura sotto.

- A *Loop activity* is an activity repeated sequentially a number of times. It is characterized by a 'Sequential Loop' marker¹ on the Task / Sub-Process. As a result, the sub-process will be instantiated several times sequentially. The number of instances to execute might be: (i) defined at design time, (ii) affected at runtime from some process data, (iii) computed at runtime.
- A *multi-instance activity* is an activity repeated in any order, sequentially or in parallel, whose number is defined in advance. It is characterized by a "Parallel Loop" marker² on the Sub-Process.

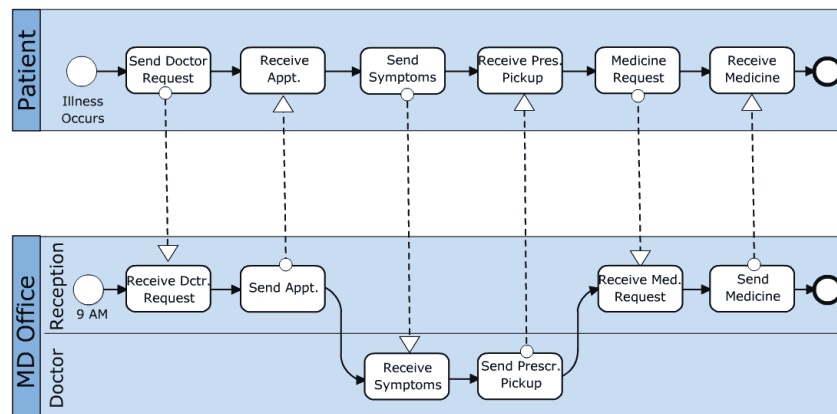


- The *Complex Gateway* addresses complex cases which would require the combination of several other gateways. To avoid this, the behavior of the complex gateway can be scripted using an expression language.



Il diamante con l'asterisco è detto **complex gateway**, gestisce casi più complessi che richiedono la combinazione di diversi tipi di gateway.

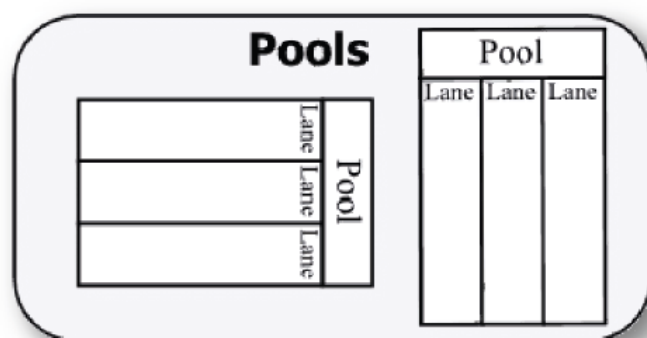
In questo esempio abbiamo i pazienti al primo piano e gli uffici dei medici al piano terra:



Vogliamo sviluppare una software factory, dove ciclicamente si sviluppano software.

- Modeling with (+) or without (–) gateways?
 - + it is not possible to model entirely without gateways
 - + gateways are more expressive and, sometimes, necessary
 - if gateways are omitted as far as possible, more compact models can be created
 - gateways with several inputs and several outputs may cause misunderstanding
- A *Pool* represents a participant in a business process. It can be a specific entity (e.g. department) or a role (e.g. assistant manager, doctor, student, vendor). A *Pool* is represented by a box.
- A *Lane* is a sub-partition within a Pool. For instance, having a pool Department, you may have Department Head and General Clerk as lanes. Same as pools, you can use lanes to represent specific entities or roles who are involved in the process.

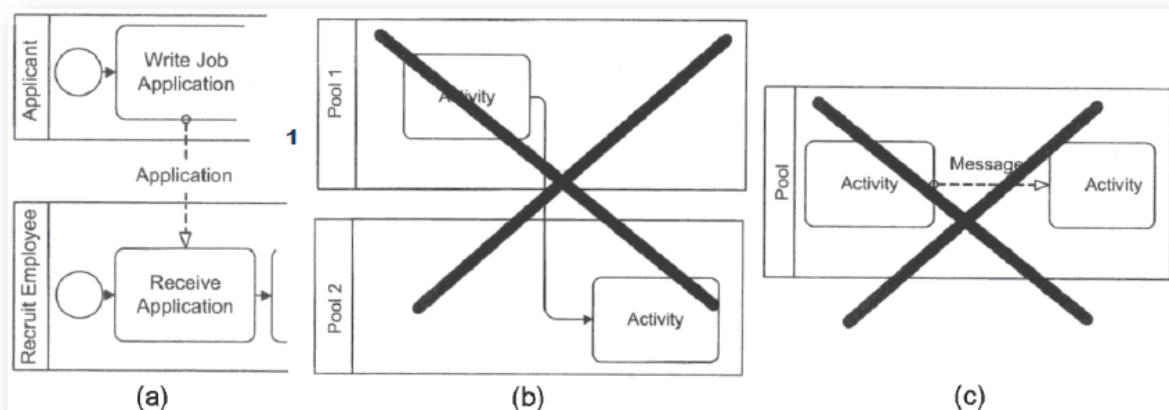
References:
 OMG, *BPMN 2.0 Specification*,
 pp. 29-41.



- Il Pool è un box che chiude la logica,
- Il lane è in orizzontale o verticale.

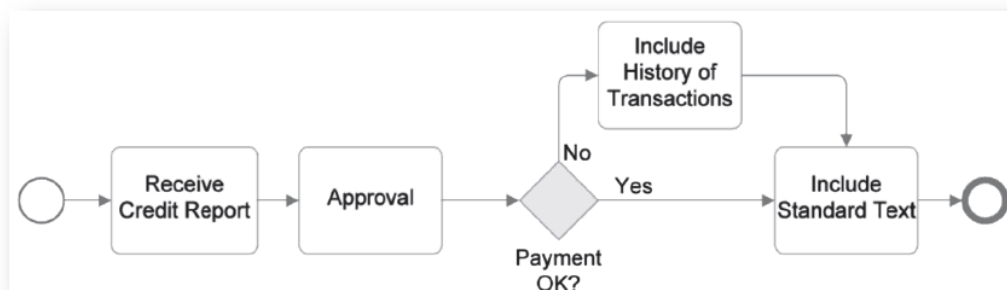
1.1. Orchestrazione e coreografia

- A *private* business process is internal to a pool, i.e., does not interact with external pools.
- A *public* process interacts with other pools via message flows¹, represented as white dashed arrow (a).
- Control flow between pools (b) and message flow within a pool (c) are **forbidden**



Consideriamo processi privati e pubblici: quando una attività/evento interagisce con pool esterni è pubblico altrimenti privato se lavoriamo internamente. *Write Job Application* è un message flow. Ogni pool non può controllare altri pool. È vietato connettere due attività nello stesso pool con un messaggio, solo con differenti pool.

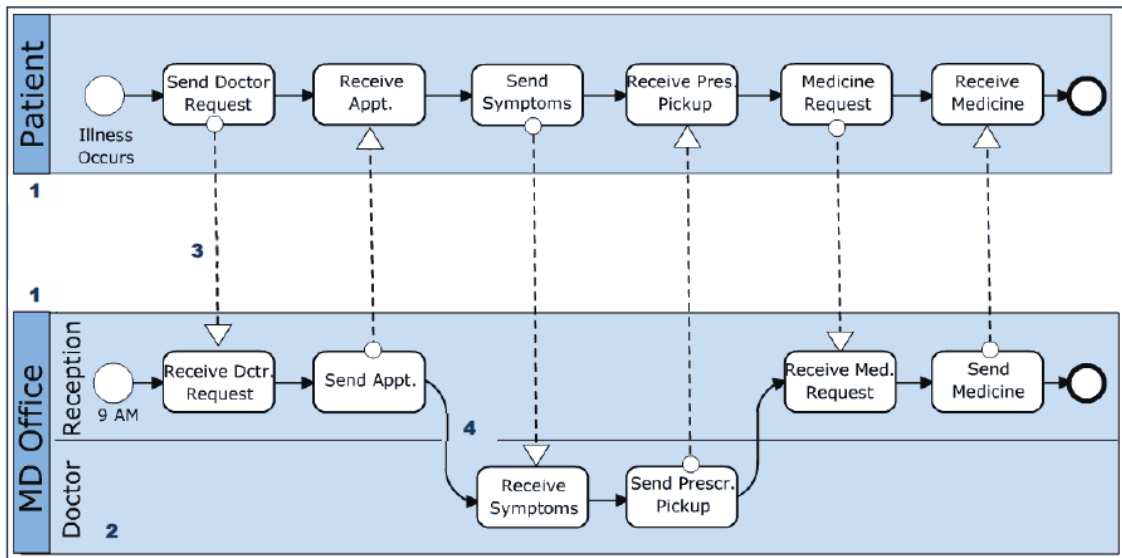
- Considering the centralization/distribution of the control flow, there are two types of BPMN processes: *orchestration* and *collaboration/choreography*.
- *Orchestration*: a process with *central control* of activities (like the control of an orchestra director), modeled as a private process, and containing optionally black-box pools presenting external entities.



- *Collaboration* (“business protocol”): the ordered set of interactions between two or more participants as shown by pools, with their public processes and message flow between them. A collaboration is characterized by *distributed control*. It is also called *choreography*: each individual participant reacts to events generated by the other participants (as in a choreographed dance where dancers react to behaviors of their peers). It can contain optionally some orchestration or just black-box pools.

Orchestrazione: tutto è determinato da una predefinita sequenza, non ho collaborazione nè scambio di messaggi. Prende ispirazione da una orchestra, quindi diciamo ad ogni attività o sistema quando iniziare. Per ogni attività ci saranno sistemi per gestirla.

- Collaboration between a patient and a medical office. The processes within pools are abstracted to hide complexity or private information to partners. Note: pool (1), lane (2), message flow (3) and control flow (4). Here, choreography is made by the message flows, whereas orchestration is made by sequence flow, events, tasks.

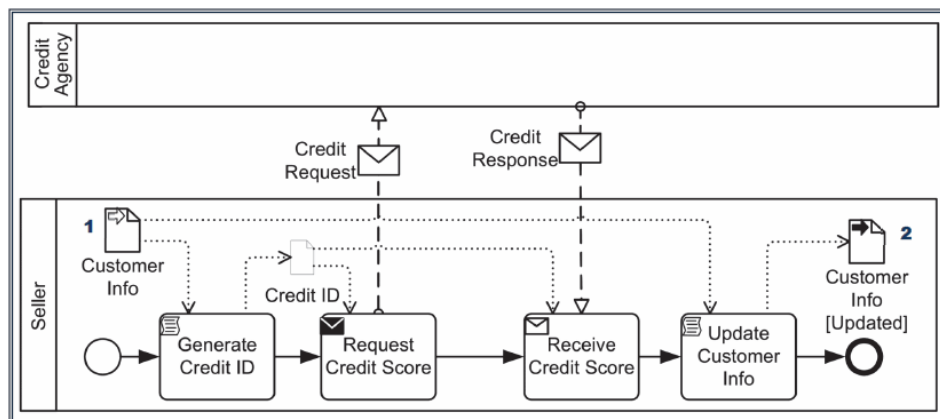


Nella immagine di sopra si ha una collaborazione perché c'è uno scambio di messaggi, coordinazione fra di essi.

Coreografia: ci sono coordinazioni senza controllo centralizzato, abbiamo sistemi collaborativi, creeremo una factory organizzata.

Data objects

- BPMN allows the explicit modeling of data transfer, via *data objects*, *messages* and *data store*. Data objects only exist within a process, whereas data store represent persistent data. UML class diagrams or technical terms diagrams could be used to refine data objects in process models → process view is integrated with data view.
- A directed data association is drawn as a dotted line to model which activity outputs and takes as input a certain data object. Data input¹ and data output² can represent a form of dependency between activities.



Abbiamo una credit agency che aiuta un venditore: se voglio comprare una macchina, mi possono essere dati soldi da una agenzia, si genera un *credit ID* e un data object non persistente chiamato "customer info" (verrà eliminato alla fine del processo).

La busta bianca indica un messo in arrivo.

La freccia nera significa mandare informazioni. Abbiamo un oggetto che arriva e viene generato il credit ID, usato come riferimento per identificare il customer in altri step. Alla fine, hai l'oggetto customer info per completare i processi.

Dall'immagine si può vedere il flusso di messaggi, il dataflow. Ne abbiamo di due tipi:

- Internal: dati mossi tra le attività
- External: hai un messaggio scambiato con http per esempio.

Workflow Modeling

10 of 73

How to name a process

(Readings: pp. 40-46, Sharp, *Workflow modeling*, 2009)

A01) The process name must be in the form *verb-noun*

e.g. *Assign Inspector*.

A02) It might be in the form *verb-qualifier-noun* or *verb-noun-noun*

e.g. *Assign Backup Inspector*, *Assign inspector to route*

A03) Processes are almost always defined in the **singular**

e.g. ~~*Handle Orders*~~ → *Fill Order*, i.e. an (a specific) order.

A04) The verb-noun name must indicate the **result** of the process.

A05) Guideline: if you flip the terms around into 'noun is verbed' form, the phrase should indicate the result of the process

e.g. *Assign Inspector* → *Inspector is Assigned*

Abbiamo modelli grafici per generare la software factory ma abbiamo bisogno di linee guida. Il *namig* è importante nel modello, ovvero i nomi delle variabili sono importanti per questo motivo vanno messi quelli giusti. I nomi in genere sono forniti dal customer/operatore della factory. In generale si usa il vocabolario di un certo dominio.

Un processo ha una vista dinamica, le sue attività hanno una parola per descriverle, chiamata un *qualifier* per caratterizzarle meglio.

C'è una regola formale in cui puoi flippare le parole, dopo la fine di un processo, ma il risultato deve essere chiaro.

A process delivers a specific, essential result

The process result in 'noun is verbed' form must meet 3 criteria:

A08) **The result is discrete and identifiable:** you can differentiate individual instances of the result, and it makes sense to talk about 'one of them'

e.g. *Inspector is Assigned* → it makes sense in a business context to talk about *when was Joe assigned to the midtown route*

A09) **The result is countable:** you can count how many of that result you have produced in an hour, a day, a week, etc.

e.g. *How many inspector assignments were completed this week*

A10) **The result is essential:** it is fundamentally necessary to the operation of the enterprise, not just a consequence of the current implementation

e.g. *Fax Inspector Assignment* → the essence of the process has not been reached by the technology of notification → *Notify Inspector of Assignment* is ok ('what, not who or how')

Il risultato è discreto, posso vedere la differenza tra le istanze dei risultati.

A11) A process should be defined in terms of the essence of what it does, i.e., the result it delivers (what), not the technologies used to support it (how) or the organizations and roles that carry it out (who)

A12) Distinguish between result and objective:

- a *result* is the *output* of a single execution of a process
e.g. *'employee is hired from the process Hire Employee'*

- an *objective* is some desired state or performance target, generally over many instances

e.g. *'Hired employees will go on to have an average tenure of greater than five years'*

e.g. *'Fewer than 10% of newly hired employees will leave the company within the first year'*

A13) An *action* verb indicates a single activity that happens at a particular point in time, for which it is easy to visualize a result.

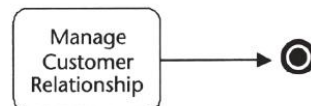
e.g. *count, evaluate, print, attach, return, prioritize, sort, provide Allocate Service Representative, Calculate Stock Index, Retrieve Sample, Issue Refund, Translate Document*

A14) Avoid mushy verbs: a *mushy* verb tends to indicate an activity or multiple activities that happen *over time*. While they might indicate some overall objective, they do not help us visualize a single, specific result.

A15) *Mushy verbs* are also called ‘360 verbs’, as they sound good but often say little about what was actually accomplished.

A16) A list of mushy verbs you should avoid using, except when naming process areas or other broad groups of activities:

- | | |
|----------------------|-------------------|
| - <i>Maintain;</i> | - <i>Manage;</i> |
| - <i>Administer;</i> | - <i>Handle;</i> |
| - <i>Process;</i> | - <i>Do;</i> |
| - <i>Improve;</i> | - <i>Support;</i> |
| - <i>Facilitate;</i> | - <i>Drive;</i> |
| - <i>Track;</i> | - <i>Review;</i> |
| - <i>Analyze;</i> | - <i>Monitor;</i> |
| - <i>Coordinate.</i> | |



Process:
Manage
Customer Relationship
(*mushy verb – noun*)

Result:
Customer Relationship
is Managed
(*noun is verbed*)

- *not discrete*
- *not countable*
- *not an essential result (an objective)*

Process naming—no mushy verbs.

Dobbiamo evitare i cosiddetti “mushy verbs”, parole a 360 gradi, non focalizzate ad uno specifico istante, nelle quali non puoi visualizzare uno specifico istante di tempo in cui il processo è gestito. Il testo va trasformato in una sequenza di attività, attraverso gli “action verbs” che andranno a creare il workflow. Dobbiamo sviluppare la nostra pipeline in una *operational view*.

A17) When describing a process it is necessary to be able to identify the *triggering or initiating event*.

Event categories:

A18) *Action event*: it happens when a person or an organization decides to do something, for any reason. You cannot predict exactly when

e.g. *a customer deciding to place an order, a manager deciding the company needs a new employee, a regulator deciding to issue a new guideline*

A19) *Temporal event*: it happens when some predetermined date or time is reached at which some activity must begin. You always know exactly when it will next happens.

e.g. *time to run the payroll, time to close the books, time to take inventory*

A20) *Condition or rule event*: it happens when a monitoring activity detects some exception condition. You cannot predict it in advance

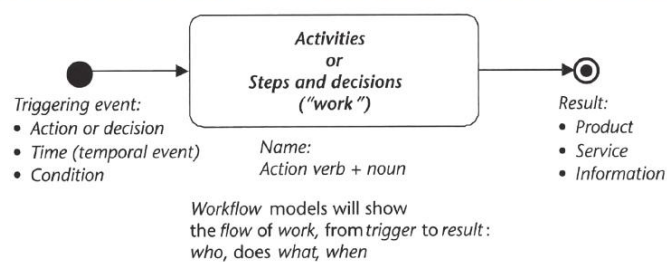
e.g. *a smoke alarm being set off, a stock price hitting a predetermined limit.*

Abbiamo varie forme per rappresentare le entità:

1. Action event: qualcuno fa qualcosa, per esempio può esserci una soglia in qualche variabile, come per esempio un sensore di temperatura.

2. Temporal event: quando hai qualche timer, alla fine del mese c'è qualche processo che inizia ad esempio.
3. Condition event: puoi monitorare la condizione, puoi vedere gli eventi e quando diventano veri.

The figure represents the *trigger-activities-result* conceptual framework.



It is in contrast with input-process-output (which sounds mechanical)

La factory è fatta da pezzi di lavoro organizzati.

Summary

A22) the process comprises an identified body of work that can be characterized as a set of related activities or a defined sequence of steps and decisions.

A23) the process name is (essentially) in action verb-noun format, although it might have a qualifier or another noun.

A24) The name is in the singular.

A25) The name, if put in "noun is verbed" format, will indicate the intended result (output) of the process.

A26) The result must be discrete and countable. If a mushy verb is used, these criteria will not be met.

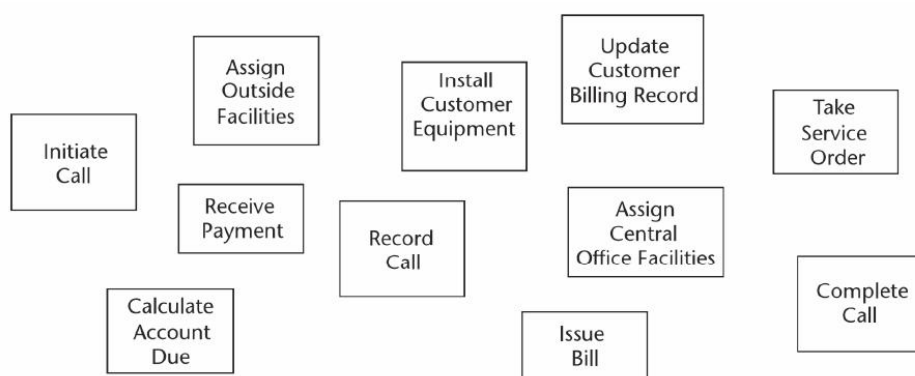
A27) The result is what the customer of the process wants.

A28) The process is initiated by a triggering event that could be action, time, or condition.

A29) Rule-of-thumb: **NO RESULT → NO PROCESS**

Ci focalizzeremo sui risultati.

Example: a Brainstorm Diagram to discover real processes in a Telephone Company



Partial results from activity identification.

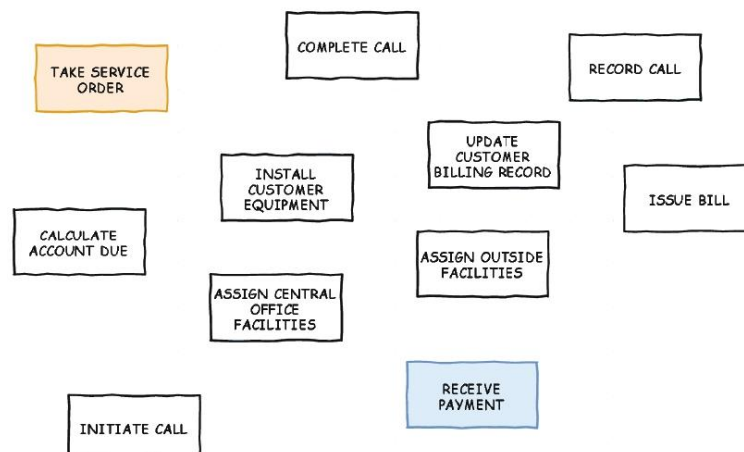
Compagnia di telefonia: prima bisogna connettere le attività fatte dalla compagnia, dai sistemi e dai client. All’inizio, per esempio, puoi prendere gli ordini di qualcuno che vuole aderire alla compagnia e sarà quindi presente qualcuno che prenderà questo ordine (attività). Ogni mese hai una parcella da pagare, devi aggiornare il numero di telefonate che cambiano la parcella. Quindi devi registrare le durate delle chiamate e altre informazioni. Devi ricevere i pagamenti che la compagnia vedrà. Ci saranno delle attività di setup che richiedono dell’equipaggiamento.

Abbiamo tante attività da organizzare, dopo avere messo le attività “sul muro” (ovvero fatta la mappa concettuale delle attività su una lavagna), ci muoveremo e inizieremo a interagire.

<https://www.draw.io/>

File > Save as > download > html

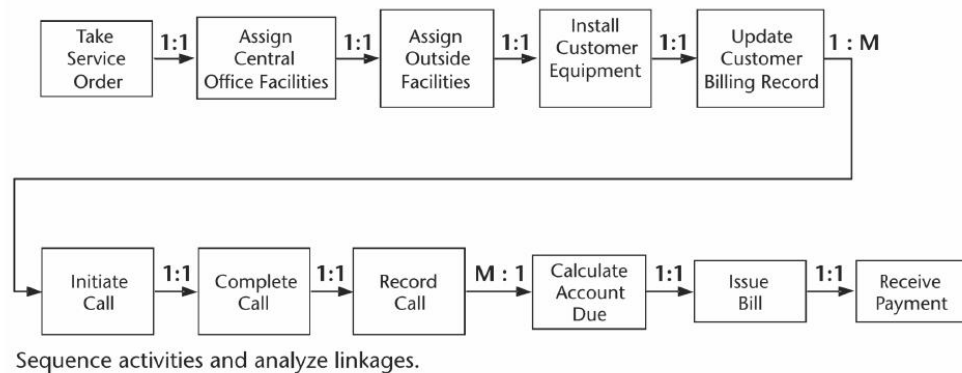
File > Open from > device > html



Va indicata inizialmente la prima attività (quella gialla, che è prendere gli ordini di servizio), poi l'ultima attività (quella azzurra che è ricevere il pagamento), la quale viene fatta dopo tutte le altre. Bisogna focalizzarsi sul caso in cui tutto funziona. C'è un altro ordine di esecuzione se qualcosa va male.

Workflow Modeling

25 of 73

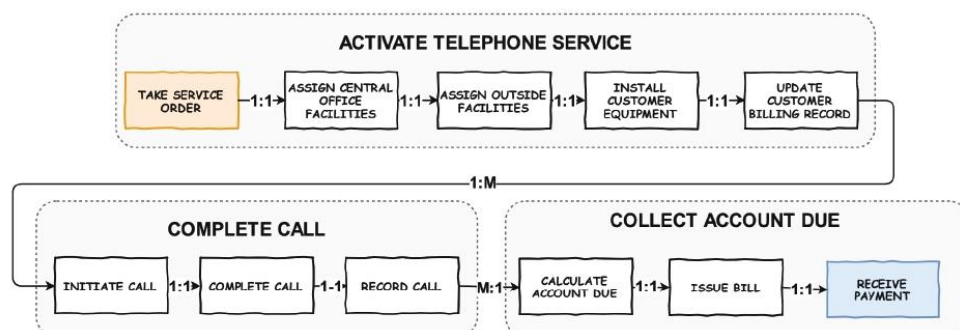


Adesso connettiamo le attività in una sorta di flusso. Abbiamo introdotto l'ordinamento nominale, dove ho una unica sequenza nominale delle attività. Adesso assegniamo le cardinalità: a differenza dei database nei quali abbiamo classici statiche e quindi si crea una relazione di cardinalità statica, qui abbiamo **dinamicità**. Es: 1:1 --> nel mondo reale significa che 1 take service order porterà ad un assign central.

Tutta la catena è chiamata *life cycle*, lo stesso che vedremo per il nostro software.

Workflow Modeling

26 of 73



Identify business processes

Abbiamo differenti processi, assegneremo un nome per ogni processo.

Un *milestone* è un obiettivo intermedio o un punto di controllo significativo in un progetto, che segna il completamento di una fase importante o l'avanzamento a una nuova fase.

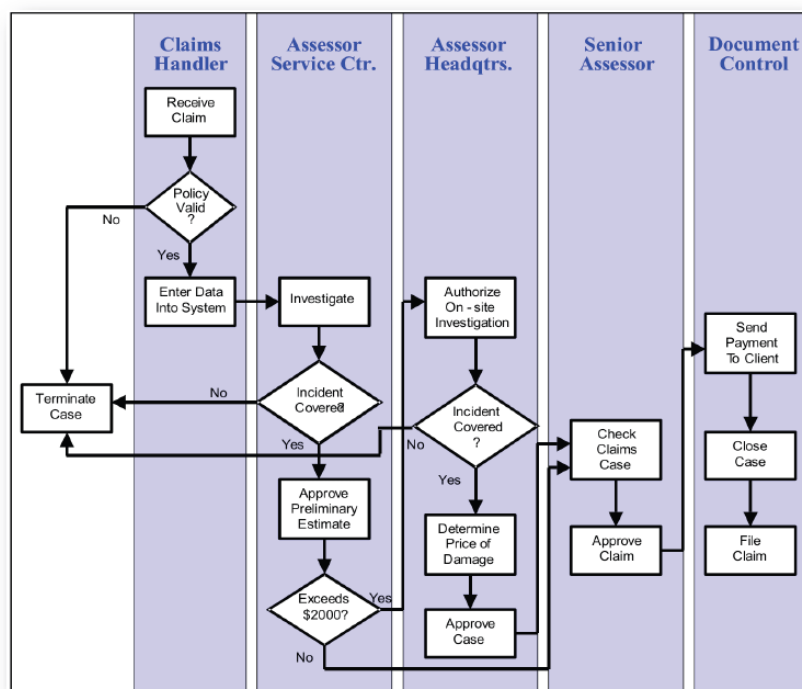
Nel contesto dello sviluppo software, un *milestone* rappresenta una pietra miliare che indica il raggiungimento di uno stato critico, come ad esempio:

- Completamento del design,
- Fine della fase di implementazione,
- Rilascio di una versione alpha o beta,
- Rilascio ufficiale del software.

Modeling from informal diagrams: a claims process at an insurance company

- A personal claims department in an insurance company handles claims made by their clients. The figure on the next page is a non-BPMN process map depicting the personal claims process in terms of swimlanes.
- The first lane corresponds to work done by a claims handler (CH) located at the client's local service center. Upon arrival of a claim, the assessor determines if the client has a valid policy. If no (5% of all cases), then the case is terminated; otherwise (95% of all cases), the assessor enters the appropriate information in the system.
- In the second lane, an assessor located at the service center (ASC) receives data from the claims handler. The assessor first determines if the claim is covered by the client's policy. If not (5% of cases), the case is terminated; otherwise (95% of cases), the assessor approves the preliminary estimate of the damage. If the damage exceeds \$2,000 (35% of cases), the claim is sent to an assessor at headquarters for approval; otherwise (65% of cases), it is sent directly to a Senior Assessor.

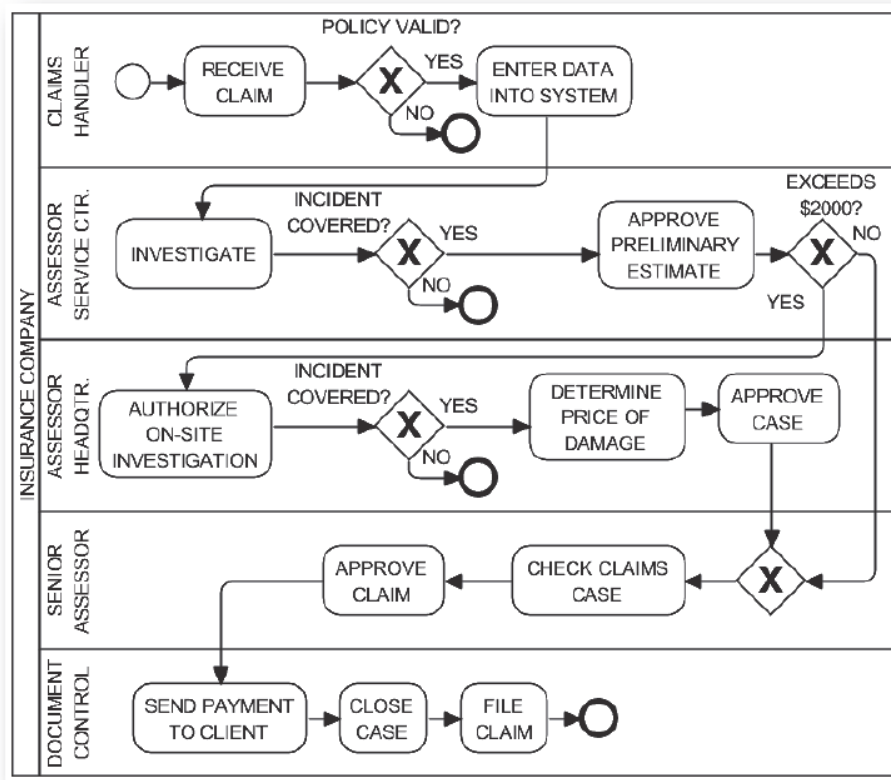
Grafico informale della compagnia:



- Lane 3 corresponds to the assessor at headquarters (AHQ). The assessor first authorizes the on-site investigation of the accident. If the investigation determines that the incident is not covered by the client's policy (2% of cases), then the case is terminated; otherwise (98% of cases), a final price is determined and the case is approved.
- In lane 4, the senior assessor (SA) receives the claim, checks it, completes it, and provides the final approval. Once the claim is approved, it is sent to documentation control.
- Documentation control (DC), in lane 5, is in charge of processing the payment to the client, closing the case, and, finally, filing the claim.

✓ Create a BPMN model

- ✓ Given 100 starting tokens, determine the number of ending tokens for each scenario (path), considering the aforementioned branching proportion (percentage of cases) for each gateway.



Abbiamo una compagnia di assicurazione con una descrizione testuale, dove è descritta una gerarchia nella compagnia e sono presenti anche differenti aspetti riguardo ad un prodotto. Abbiamo un *informal flow* prodotto dalla compagnia che dovremo convertire in un **diagramma operativo formale**. Perché va fatta questa traduzione? Perché alcuni elementi nell'informale non sono chiari. Il testo spiega il grafico informale, oltre a fornire percentuali di arrivo dei token.

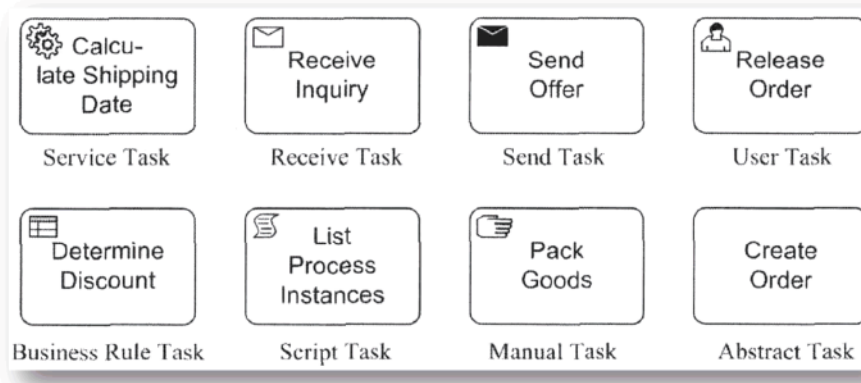
Quello formale è il flusso BPMN.

Dobbiamo seguire le frecce: abbiamo una struttura orizzontale nella quale ci sono internal lanes, simile all' UML activity diagram. Ogni membro avrà un software system per ogni lane.

Il flow parte da *receive claim*, puoi arrivare al NO, dove il token viene distrutto e in questo caso si ha un percorso corto. Se invece hai SI passi al secondo lane, il flusso viene mosso al prossimo ruolo, alla prossima applicazione. Il token si muove in avanti, il percorso può anche essere lungo. È importante pensare al modello dell'implementazione e all'implementazione stessa. Vogliamo organizzare un sistema di software e per fare ciò ci focalizziamo sul flusso di lavoro.

Types of tasks and icons

- ❑ a *service task* is an automated function processed by an external application;
- ❑ a *receive/send task* receives/sends a message;
- ❑ a *user task* expects input by a user via a UI;
- ❑ in a *business rule task* some business rules are applied (e.g. via a business rule management system launched by the process engine) to produce a result;
- ❑ a *script task* contains statements processed directly by the process engine;
- ❑ a *manual task* is carried out without IT support;
- ❑ in an *abstract task* no type is defined.



Readings:
- Section 7.4,
T. Allweyer,
BPMN 2.0,
2010

Muoviamoci su un'altra vista del workflow: i vari tipi di task e icone.

L'ingegnere informatico aggiunge automazione. Ci sono differenti tipi di task nel BPMN.

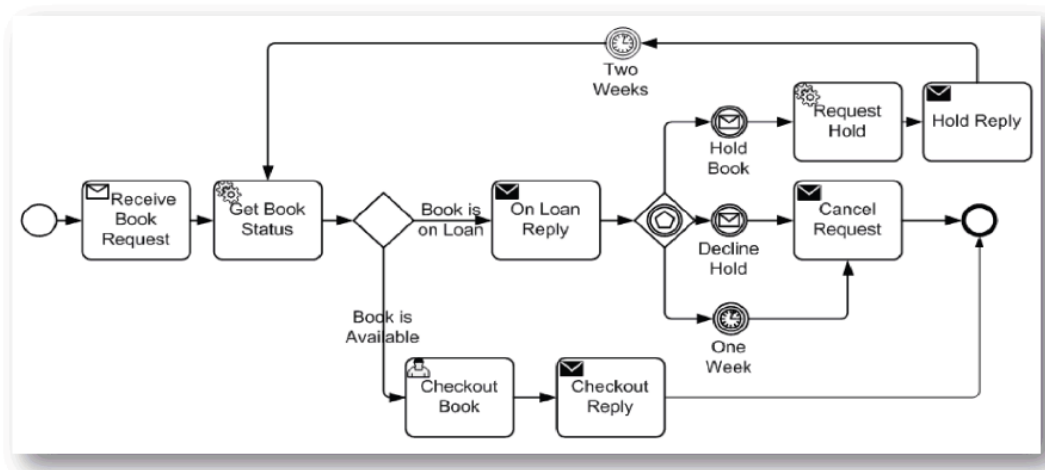
- Message task: abbiamo parlato di task con messaggi, mandi e ricevi, fai pre e post processing.
- User task: c'è un umano coinvolto (mentre con json o restful api è tutto automatizzato), hai lo user task associato con uno use case, quando vedi il grafico qual è la tua decisione? Ci sarà un set di step non solo iterazioni ma anche ragionamenti (per questo servono umani), bisogna standardizzare la compagnia, il "sì" o il "no" deve essere standardizzato, per esempio quando per te il testing va bene o no. Mi serve un ingegnere o un tecnico che faccia queste valutazioni, la scelta tra i due dipende dal task e da questo dipenderà il costo del software, scegliendo professionisti più o meno esperti. Quando hai del software invece che affidarsi a tecnici ed esperti, puoi sostituirli con sistemi AI, per esempio, focalizzandosi sulla storia dei dati. In definitiva, questo tipo di task prevede uno user che usa un sistema IT, che richiede decisione umana. Ciò che non ha bisogno di human decision può essere automatizzato, per esempio copiare e incollare. Bisogna ridurre l'uso di umani in attività inutili.
- Manual task: l'umano sta facendo il task senza supporto IT

- High maintenance task → necessitano di business rule, come ad esempio il firewall che va modificato, non puoi usare uno scripting, ma usi un linguaggio di alto livello, per non esperti che possono specificare le regole in modo semplice.
- Script task: google maps fornisce un servizio per avere la mappa, non sviluppi il servizio da te.

L'automazione è ottima ma non va utilizzata troppo perché poi il sistema risulta rigido e non può gestire così tante eccezioni.

Library

- Describe in semi-formal natural language the following orchestration, concerning the book lending process managed by a library.



Il primo task è un message task, nella libreria hai differenti software che aiutano i lettori, abbiamo un message ricevuto dal lettore, dopo il sistema cerca il book status, è un service task perché usa un servizio esterno nel quale chiedono un libro attraverso una catena di richieste. Otteniamo lo stato del libro, se il libro è disponibile viene fatta una certa operazione: quale operazione dipenderà dal diamante che è un esclusivo gateway (uguale a quello con la X anche se non ha l'icona).

All'inizio si parte da una annotazione testuale, molto utile all'inizio perché è più facile da modificare, poi quando è finita puoi andare a quella grafica. Quella grafica è più utile per discutere con le persone, altrimenti le persone perderebbero i precedenti step, mentre con questa modalità possono vedere l'overall.

Imperative vs declarative: un algoritmo è imperative programming, tu dici alla macchina come fare ogni step, con istruzioni specifiche, converti i requisiti del client in una sequenza di comandi per la macchina, tu stai comandando la macchina, l'implementazione è nascosta. Il workflow è basato sul declarative approach, gli umani sono dentro al workflow, non puoi cambiare i differenti algoritmi perché sono scelti dall'azienda.

Le persone possono incidere sulle performance se le vengono mandate troppe richieste. L'automatizzazione deve essere fatta con una analisi delle performance sull'intera catena, non tutto può essere automatizzato.

BP Modeling: semi-formal specification and scenarios

Semi-formal notation

- 1 The System receives a book request (receive task)
- 2 The System gets the book status (service task)
- 3.1 If the book is available:
 - 3.1.1 The librarian checks the book out (user task)
 - 3.1.2 The System sends a checkout-reply message (send task) → 4
- 3.2 If the book is on loan:
 - 3.2.1 The System sends an on-loan-reply message (send task)
 - 3.2.2 The system waits for an event:
 - 3.2.2.1 When a decline-hold message is received by the system, or
When a week lapsed without answer:
 - 3.2.2.1.1 The System cancels the request and sends a related message → 4
 - 3.2.2.2 When a hold-book message is received by the System:
 - 3.2.2.2.1 The System holds the request (service task)
 - 3.2.2.2.2 The System sends a hold-reply message (send task)
 - 3.2.2.2.3 The System waits for two weeks
 - 3.2.2.2.4 → 2
- 4 End

3. è il workflow principale

Ogni branch ha .1, ovvero un altro livello di numerazione. Quando usi Java ecc hai una programmazione strutturale, con blocchi. Gli umani lavorano in parallelo, è difficile capire se hai un deadlock nel workflow.

Il rombo (gateway) è il *routing* del flow, muove il flusso verso A o B, non c'è processing logic perché è calcolato prima.

Nel checkout book c'è un umano che usa un information system, mix tra physical world e uman assisted IT, è uno uman task.

L'altro rombo (doppio cerchio e icona) è comandato da eventi, bisogna aspettare per un evento esterno.

1.2. Esercizio

Questions

1. When a hold-book message is sent, no other emails are due by the Customer to have the book in hand.
☐ true ☐ false
2. When the book is on-loan, more than 5 total messages (sent or received) are due to have the book in hand, provided that it is made available by two weeks.
☐ true ☐ false
3. When the book is still on-loan after two weeks since the hold-book message was sent, it is possible to rethink the initial choice by sending a decline-hold message.
☐ true ☐ false
4. While the book is on-loan, the Customer can control the system in causing a get-book-status every about three weeks instead of two weeks.
☐ true ☐ false

Answers

1. **False:** the hold-book message must be repeated every two or three weeks to keep the book request. If not, the request will be canceled.
2. **False:** the number of messages is exactly 5. ↓ receive book request, ↑ on-loan reply, ↓ hold book, ↑ hold reply, ↑ checkout reply.
3. **True:** after two week, the System will send another on-loan-reply message, and the Customer can answer with a decline-hold message.
4. **True:** After the on-loan-reply message, the Customer can wait a little less than one week to send the hold-book message. This way, the loop for a get-book-status lasts a little less than three weeks: two weeks + a little less than one week.

2. Prodotti Software (Alfeo)

SOFTWARE PRODUCTS

Software products are generic software systems that provide functionality that is useful to a range of customers.

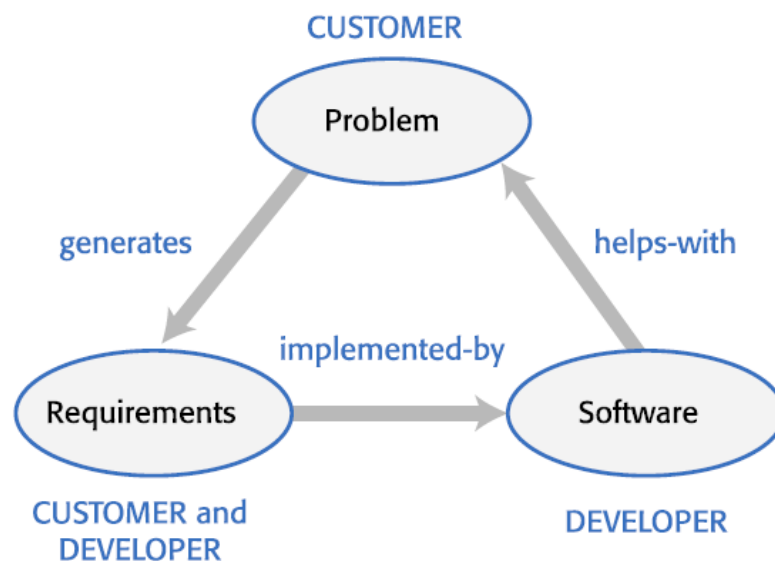
Many different types of products are available from large-scale business systems (e.g. MS Excel) through personal products (e.g. Evernote) to simple mobile phone apps and games (e.g. Sudoku).

Software product engineering methods and techniques have evolved from software engineering techniques that support the development of one-off, custom software systems.

Custom software systems are still important for large businesses, government and public bodies. They are developed in dedicated software projects.

Sono software generici per risolvere problemi, tutto lo sviluppo software al giorno d'oggi è cambiato: tutto riguarda le metodologie, l'ambiente esterno è completamente cambiato rispetto al passato, le metodologie sono cambiate. Ad oggi, lo sviluppo è molto più veloce, le applicazioni sono aggiornate più volte al mese, il prodotto software vive più lungo, cambia spesso, migliora nel tempo, dobbiamo capire la parte di gestione di tutti questi processi.

PROJECT-BASED SOFTWARE ENGINEERING



Hai il cliente che chiede un prodotto con delle specifiche, dei requisiti che determinano come il software deve essere implementato, come deve funzionare, tutto ciò permette di sviluppare ogni pezzo del software che deve comunicare con gli altri.

Qui non sei da solo, parli col customer molte volte, puoi ottenere altri requisiti e puoi avere feedback.

PROJECT-BASED SOFTWARE ENGINEERING

The starting point for the software development is a set of 'software requirements' that are owned by an external client and which set out what they want a software system to do to support their business processes.

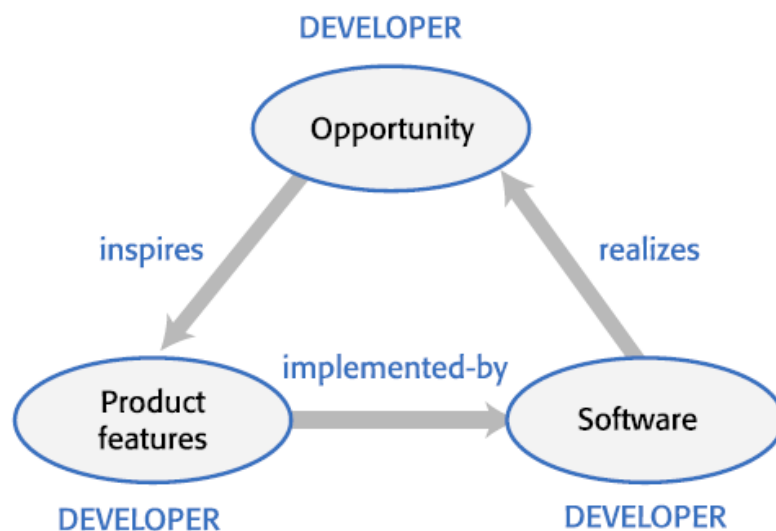
The software is developed by a software company (the contractor) who design and implement a system that delivers functionality to meet the requirements.

The customer may change the requirements at any time in response to business changes (they usually do). The contractor must change the software to reflect these requirements changes.

Custom software usually has a long-lifetime (10 years or more) and it must be supported over that lifetime.

Vogliamo un software monolitico, che non cambia nel tempo, ma magari prevede solo aggiornamenti.

PRODUCT SOFTWARE ENGINEERING



Questa nuova idea mette al centro il prodotto: è dovuto ad un bisogno del mercato, ad una opportunità che porta allo sviluppo. Parti da te stesso, osservi il mondo e decidi di sviluppare un software per soddisfare dei bisogni. Se sai i bisogni sai anche le feature per soddisfarli. Infine, l'ultima parte è implementare il software. Non hai alcun tipo di feedback, devi fare una grande ricerca per capire i bisogni.

PRODUCT SOFTWARE ENGINEERING

The starting point for product development is a business opportunity that is identified by individuals or a company. They develop a software product to take advantage of this opportunity and sell this to customers.

The company who identified the opportunity design and implement a set of software features that realize the opportunity and that will be useful to customers.

The software development company are responsible for deciding on the development timescale, what features to include and when the product should change.

Rapid delivery of software products is essential to capture the market for that type of product.

La visione del prodotto proviene da noi stessi, dalla nostra esperienza, da come funziona il mercato. Tu sei l'esperto.

SOFTWARE PRODUCT LINES AND PLATFORMS

Software product line

A set of software products that share a common core. Each member of the product line includes customer-specific adaptations and additions. Software product lines may be used to implement a custom system for a customer with specific needs that can't be met by a generic product.

Platform

A software (or software+hardware) product that includes functionality so that new applications can be built on it. An example of a platform that you probably use is Facebook. It provides an extensive set of product functionality but also provides support for creating 'Facebook apps'. These add new features that may be used by a business or a Facebook interest group.

Se non sei esperto puoi fare una intervista al cliente, tramite questa però non direttamente quali feature devi sviluppare. Non solo vendi software ma vendi anche l'infrastruttura, come ad esempio Amazon WBS.

SOFTWARE EXECUTION MODELS

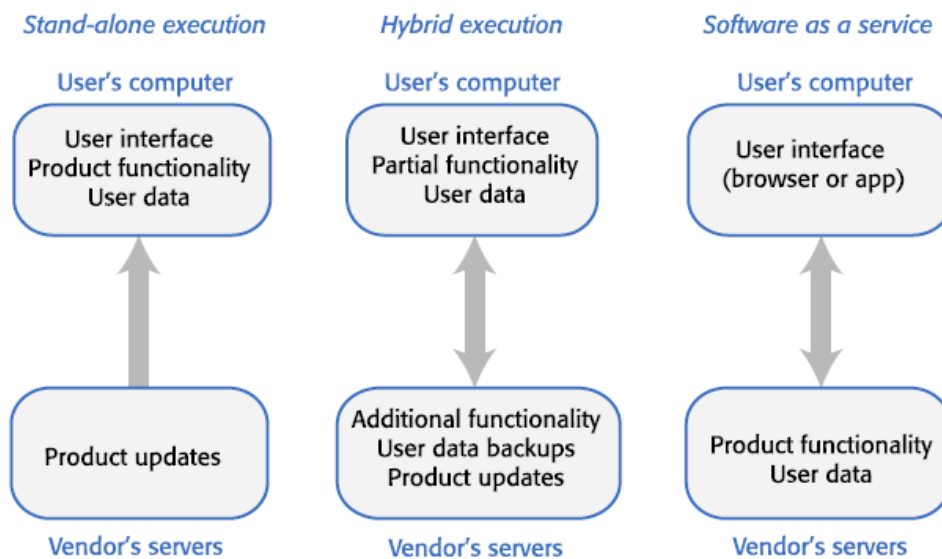
Stand-alone The software executes entirely on the customer's computers.

Hybrid Part of the software's functionality is implemented on the customer's computer but some features are implemented on the product developer's servers.

Software service All of the product's features are implemented on the developer's servers and the customer accesses these through a browser or a mobile app.

Queste 3 categorie specificano il tipo di esecuzione. *Google Colab* è un esempio di software service: il codice è eseguito su un server Google, ma esso lo fornisci te.

SOFTWARE EXECUTION MODELS



Nella Stand-alone execution ad esempio devi solo connetterti una volta al mese per ricevere gli update.

SaaS: hai i dati del client sul server, le macchine dei client usano solo il browser, le funzionalità effettive sono sul server del venditore, questo permette di lavorare facilmente durante la manutenzione.

COMPARABLE SOFTWARE DEVELOPMENT

The key feature of product development is that there is no external customer that generates requirements and pays for the software. This is also true for other types of software development:

- **Student projects** Individuals or student groups develop software as part of their course. Given an assignment, they decide what features to include in the software.
- **Research software** Researchers develop software to help them answer questions that are relevant to their research.
- **Internal tool development** Software developers may develop tools to support their work - in essence, these are internal products that are not intended for customer release.

Devi capire quali sono le migliori modalità per soddisfare i bisogni del mercato. Per esempio, puoi sviluppare un software per aiutare lo sviluppo di un altro software: ciò però lo capisci da solo non lo chiede il mercato, lavori su qualcosa che è ricorsivo.

THE PRODUCT VISION

The starting point for software product development is a 'product vision'.

Product visions are simple statements that define the essence of the product to be developed.

The product vision should answer three fundamental questions:

- What is the product to be developed?
- Who are the target customers and users?
- Why should customers buy this product?

Tutto parte dai requisiti, essi nascono per esempio da assunzioni. Ma i requisiti sono sostituiti dal **product vision** che nasce dalle 3 domande sopra.

MOORE'S VISION TEMPLATE

FOR (target customer)

WHO (statement of the need or opportunity)

The (PRODUCT NAME) is a (product category)

THAT (key benefit, compelling reason to buy)

UNLIKE (primary competitive alternative)

OUR PRODUCT (statement of primary differentiation)

Nella immagine sopra si possono vedere le domande fondamentali per avere una completa visione del prodotto. Per costruire una visione bisogna sapere: chi è interessato a comprare il prodotto, a chi lo vendi, perché sarà mio cliente, i bisogni da soddisfare che lo portano a comprarlo, la categoria del prodotto.

UNLIKE: grazie a questo si possono capire i punti chiavi di vendita, ovvero cosa il tuo software fa che altri non fanno. Cosa svolge in modo ottimale? L'obiettivo è essere competitivi nel mercato.

VISION TEMPLATE EXAMPLE

“FOR a mid-sized company's marketing and sales departments WHO need basic CRM functionality, THE CRM-Innovator is a Web-based service THAT provides sales tracking, lead generation, and sales representative support features that improve customer relationships at critical touch points. UNLIKE other services or package software products, OUR product provides very capable services at a moderate cost.”

Questo è un esempio di visione, lo sviluppo partirà da questo. Va ricordato che la competizione è tanta, l'obiettivo è capire come superare i competitors.

INFORMATION SOURCES FOR DEVELOPING A PRODUCT VISION

Domain experience

The product developers may work in a particular area (say marketing and sales) and understand the software support that they need. They may be frustrated by the deficiencies in the software they use and see opportunities for an improved system.

Product experience

Users of existing software (such as word processing software) may see simpler and better ways of providing comparable functionality and propose a new system that implements this. New products can take advantage of recent technological developments such as speech interfaces.

Customer experience

The software developers may have extensive discussions with prospective customers of the product to understand the problems that they face, constraints, such as interoperability, that limit their flexibility to buy new software, and the critical attributes of the software that they need.

Prototyping and playing around

Developers may have an idea for software but need to develop a better understanding of that idea and what might be involved in developing it into a product. They may develop a prototype system as an experiment and 'play around' with ideas and variations using that prototype system as a platform.

Per costruire una visione devi avere un contesto. L'esperienza è importante.

A VISION STATEMENT FOR A E-LEARN SYSTEM

FOR teachers and educators **WHO** need a way to help students use web-based learning resources and applications, **THE** iLearn system is an open learning environment **THAT** allows the set of resources used by classes and students to be easily configured for these students and classes by teachers themselves. **UNLIKE** Virtual Learning Environments, such as Moodle, the focus of iLearn is the learning process rather than the administration and management of materials, assessments and coursework. **OUR** product enables teachers to create subject and age-specific environments for their students using any web-based resources, such as videos, simulations and written materials that are appropriate.

Schools and universities are the target customers for the iLearn system as it will significantly improve the learning experience of students at relatively low cost. It will collect and process learner analytics that will reduce the costs of progress tracking and reporting.

Altro esempio di vision statement.

SOFTWARE PRODUCT MANAGEMENT

Software product management is a business activity that focuses on the software products developed and sold by the business.

Product managers (PMs) take overall responsibility for the product and are involved in planning, development and product marketing.

Product managers are the interface between the organization, its customers and the software development team. They are involved at all stages of a product's lifetime from initial conception through to withdrawal of the product from the market.

Product managers must look outward to customers and potential customers rather than focus on the software being developed.

Il più importante ruolo è il **software management**: devi avere un piccolo CEO, per ogni specifico prodotto.

PRODUCT MANAGEMENT CONCERNS



Il PM si interfaccia con differenti clienti per capire cosa vuole il cliente e qual è il centro del business cioè come si possono guadagnare soldi.

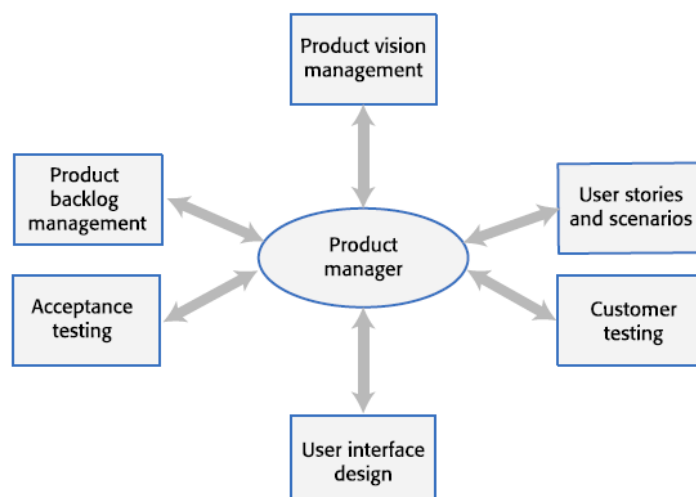
PRODUCT MANAGEMENT CONCERNS

Business needs PMs have to ensure that the software being developed meets the business goals of the software development company.

Technology constraints PMs must make developers aware of technology issues that are important to customers.

Customer experience PMs should be in regular contact with customers and potential customers to understand what they are looking for in a product, the types of users and their backgrounds and the ways that the product may be used.

TECHNICAL INTERACTIONS OF PRODUCT MANAGERS



Bisogna costruire la visione, ma essa può cambiare quando cerchi nel mercato, potresti trovare un competitore e ciò può accadere in questo livello.

Acceptance testing: non è solo la correttezza del software, la maggior parte dei test è per verificare che tutto funzioni come il client vuole, ma non solo, devi essere anche sicuro che sia facile da usare, per attirare clienti a comprarlo.

I due rettangoli a destra sostituiscono il fatto che “non ci sono requisiti”, sei tu che cerchi di dedurre quelle informazioni. Le *User stories and scenarios* sono cosa **potrebbe** pensare il client che il nostro software fa, in questo caso “includi” il cliente.

TECHNICAL INTERACTIONS OF PRODUCT MANAGERS

Product vision management

- The product manager may be responsible for helping with the development of the product vision. They should always be responsible for managing the vision, which involves assessing and evaluating proposed changes against the product vision. They should ensure that there is no ‘vision drift’

Product roadmap development

- A product roadmap is a plan for the development, release and marketing of the software. The PM should lead roadmap development and should be the ultimate authority in deciding if changes to the roadmap should be made.

User story and scenario development

- User stories and scenarios are used to refine a product vision and identify product features. Based on his or her knowledge of customers, the PM should lead the development of stories and scenarios.

Product backlog creation and management

- The product backlog is a prioritized ‘to-do’ list of what has to be developed. PMs should be involved in creating and refining the backlog and deciding on the priority of product features to be developed.

Acceptance testing

- Acceptance testing is the process of verifying that a software release meets the goals set out in the product roadmap and that the product is efficient and reliable. The PM should be involved in developing tests of the product features that reflect how customers use the product.

Customer testing

- Customer testing involves taking a release of a product to customers and getting feedback on the product’s features, usability and business. PMs are involved in selecting customers to be involved in the customer testing process and working with them during that process.

User interface design

- Product managers should understand user limitations and act as surrogate users in their interactions with the development team. They should evaluate user interface features as they are developed to check that these features are not unnecessarily complex or force users to work in an unnatural way.

PRODUCT PROTOTYPING

Product prototyping is the process of developing an early version of a product to test your ideas and to convince yourself and company funders that your product has real market potential.

- You may be able to write an inspiring product vision, but your potential users can only really relate to your product when they see a working version of your software. They can point out what they like and don't like about it and make suggestions for new features.
- A prototype may be also used to help identify fundamental software components or services and to test technology.

Building a prototype should be the first thing that you do when developing a software product. Your aim should be to have a working version of your software that can be used to demonstrate its key features.

You should always plan to throw-away the prototype after development and to re-implement the software, taking account of issues such as security and reliability.

Fare un prototipo è molto conveniente per coinvolgere il cliente nel processo di sviluppo fin da subito, permette di avere un beta testing e di migliorare la visione, quest'ultimo porta però a ripartire un po' più indietro nello sviluppo.

TWO-STAGE PROTOTYPING

Feasibility demonstration You create an executable system that demonstrates the new ideas in your product. The aims at this stage are to see if your ideas actually work and to show funders and/or company management the original product features that are better than those in competing products.

Customer demonstration You take an existing prototype created to demonstrate feasibility and extend this with your ideas for specific customer features and how these can be realized. Before you develop this type of prototype, you need to do some user studies and have a clearer idea of your potential users and scenarios of use.

Il primo permette di mostrare la visione per soddisfare certi bisogni: è utile per che cosa e per chi? Per il nostro team interno e magari per chi ha magari finanziato il progetto. Bisogna dimostrare che i nostri software e visione funzionano, magari per ottenere più soldi.

KEY POINTS 1

Software products are software systems that include general functionality that is likely to be useful to a wide range of customers.

In product software engineering, the same company is responsible for deciding on the features that should be part of the product and the implementation of these features.

Software products may be delivered as stand-alone systems running on the customer's computers, hybrid systems or service-based systems. In hybrid systems, some features are implemented locally and others are accessed over the Internet. All product features are remotely accessed in service-based products.

A product vision should succinctly describe what is to be developed, who are the target customers for the product and why they should buy the product that you are developing.

Domain experience, product experience, customer experience and an experimental software prototype may all contribute to the development of the product vision.

Non solo un customer, ma bisogna convincere un range di customer che vogliamo convincere a comprare il nostro prodotto.

KEY POINTS 2

Key responsibilities of product managers are product vision ownership, product roadmap development, creating user stories and the product backlog, customer and acceptance testing and user interface design.

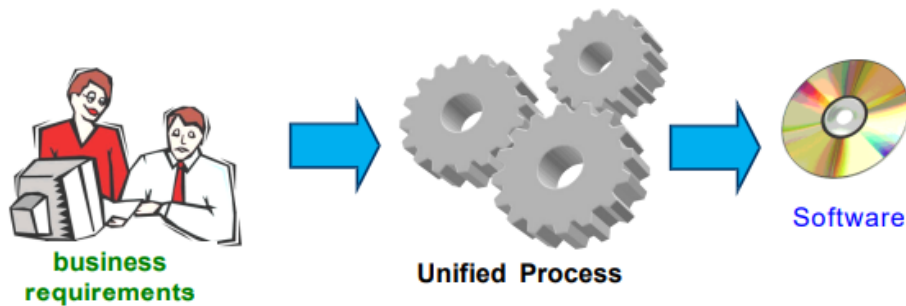
Product managers work at the interface between the business, the software development team and the product customers. They facilitate communications between these groups.

You should always develop a product prototype to refine your own ideas and to demonstrate the planned product features to potential customers

3. UML

1. The Unified Process

- ✓ A Software Development Process (SDP) defines the *who*, *what*, *when*, and *how* of developing software. The Unified Process (UP) is an industry standard SDP from the authors of the **UML** (Unified Modeling Language).

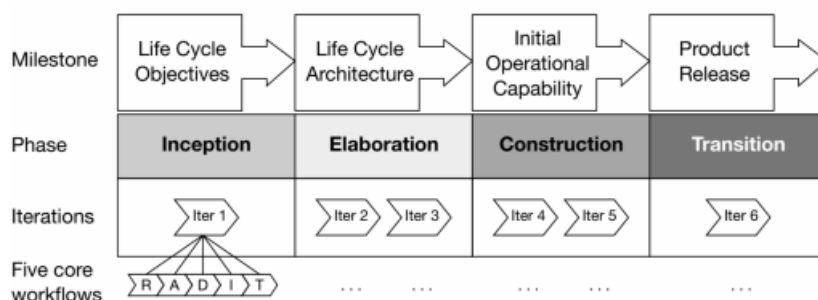


- ✓ UP is *iterative* and *incremental*: a large software development project is broken down into smaller “mini projects” called *iterations*. Each iteration generates a more complete version of the final system. The difference between two consecutive versions is called *increment*.

È un processo standard usato per sviluppare software: è standardizzato, cioè lo stesso costruito viene usato ovunque, è uno **standard internazionale**, per lavorare in contesti internazionali; è **aperto** perché può essere ampliato per includere nuovi componenti.

Iterativo: il modello precedente era *cascade*, ovvero fase dopo fase, mentre questo processo è molto più elastico, hai iterazioni che possono essere fatte più volte in base a come cambia l'ambiente.

- ✓ Each iteration is made by five *core workflows*, with different emphasis:
 - **R: requirements** – capturing what the system should do;
 - **A: analysis** – refining and structuring the requirements;
 - **D: design** – realizing the requirements in system architecture;
 - **I: implementation** – building the software;
 - **T: test** – verifying that the implementation works as desired.
- ✓ In a team work, it is often convenient to schedule iterations in parallel, according to dependencies between the artifacts of each iteration.
- ✓ UP consists of a sequence of four *phases*, terminating with related milestones:



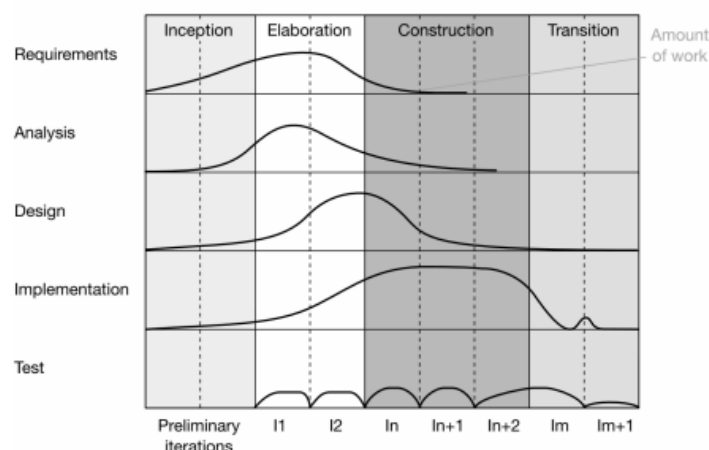
Ogni fase può avere più iterazioni, funziona come “una matrioska”: basta cambiare la grandezza reiterando più volte lo stesso schema. Non tutte queste fasi sono sempre fatte.

La **milestone** è la condizione che indica che la fase è finita, hai soddisfatto tutti i requisiti.

- L'*Inception* permette di capire durante il ciclo di vita **cosa** il mio software deve fare, quindi si parte dalla idea iniziale.
- Nell'*Elaboration* trasformi il software per soddisfare i goal, aggiungendo delle funzionalità. Dunque, prime due ottieni una architettura e un design per il tuo software.
- *Construction*: durante il cambio verso questa nuova fase, la parte di design rallenta, si parte col vero codice, si costruisce veramente il software. Come finisce l'elaborazione? Usando il risultato del processo di elaborazione, ovvero **l'architettura**. Perché è importante in termini di milestone? Perché tutto sarà basato sull'architettura che tu progetti. Alla fine di questa hai scritto il codice, adesso risponde ad ogni bisogno ma è ancora stand-alone, non l'hai ancora installato nei server del customer, nel suo smartphone, perciò devi fare il testing e applicarlo.
- Adesso si può iniziare con la *Transition* che consiste nel prendere il modello e installarlo nell'hw finale del customer.

Ogni fase ha un certo numero di iterazioni, ognuna ha 5 workflow:

1. *Requirements*: capisci cosa il software deve fare
2. *Analisi*: trasformi un requisito in una *specification*. Cosa cambia tra i due? Il linguaggio è diverso, la *specification* è una traduzione da linguaggio formale a linguaggio **tecnico**, di più basso livello, dove è presente meno astrazione.
3. *Design*: sai esattamente cosa e come deve fare, quindi inizi a progettarlo
4. Poi lo puoi *implementare* e *testare*. Il testing è importante, per magari avere il prototipo, che è molto utile per far capire al customer cosa vuole ancora, cosa va bene e cosa no.



UP: core workflows versus phases

- ✓ **Inception**: most of the work in early requirements and analysis
- Elaboration**: the emphasis on requirements and analysis and some design
- Construction**: mostly design and implementation, with related testing
- Transition**: residual implementation and test

Alcune di queste fasi potrebbero mancare o potresti spenderci poco tempo.

In *Inception* e *Elaboration* spendi molto tempo nei workflow di *Requirement*, *Analisi* e *Design*. Perché? Quando il client specifica alcuni bisogni, non essendo un tecnico, ha una idea molto confusa sui bisogni, sono molto di alto livello, non precisi, perciò hai bisogno di “ping pong” tra customer e software developer per definire questi requisiti. Quando sono precisi possono essere tradotti in *specification*. Dipende molto dal livello tecnico del customer. È una sorta di comprensione tra i due.

Quando il ping-pong è finito, dopo l’elaborazione non spendi molto tempo in *Requirement* e *specification*: se vanno bene non devi più cambiarli, per questo ci spendi meno tempo (se ovviamente sono solidi e decisi bene). In *Construction*, nel workflow di design, applichi il design e lo implementi, implementarlo richiede molto tempo. Il picco lo hai nel design di *Elaboration*.

Alla fine, fai molto testing anche se non è finale, ma produci dei prototipi funzionanti, ogni prototipo va testato.

Fino ad ora hai fatto testing interno, alla fine devi testare l’architettura in altri hardware e software, poiché il software deve funzionare nel sistema del client e non deve cambiare il comportamento del sistema del client. Solo durante la fase di *Transition* capisci cosa sia l’hw e il sistema del client. L’implementation alla fine è fatta per le correzioni.

PHASE	GOALS	FOCUS	MILESTONE
Inception	● <i>capturing essential requirements</i> to help scope the system ● <i>feasibility</i>: technical prototype to validate technology, proof of concept to validate business requirements ● business case to demonstrate that the project will deliver quantifiable business benefit	● <i>requirements and analysis workflows</i> ● some design and implementation, to build technical prototype or proof of concept ● no testing – throwaway prototype	● Life Cycle Objectives (requirements/features/constraints, initial use cases) → see conditions and deliverable table
Elaboration	● <i>create executable architectural baseline</i> ● <i>capture use cases</i> to 80% functional requirements; ● refine the Risk Assessment; ● define quality attributes (defect discovery rates, acceptable defect densities, etc.); ● create detailed plan for construction; ● formulate a bid that includes resources, time, equipment, staff and cost.	● <i>requirements, analysis and design workflows</i> ● <i>implementation</i>: build the initial operational capability ● test the initial operational capability (alpha test, internal)	● Life Cycle Architecture
Construction	● <i>complete requirements, analysis and design</i> ● move from architectural baseline to the <i>final system</i> ● maintain the system architecture integrity	● <i>implementation and testing</i> ● build the Initial Operational capability ● test the Initial Operational Capability	● Initial Operational Capability (software system is finished for beta testing in productive environment)
Transition	● starts after beta testing is completed and the system is finally deployed ● correct defects, prepare the user site for the new software; ● create user manuals and other documentation; provide user consultancy; ● conduct a post project review	● <i>no requirements, analysis</i> ● <i>finish implementation and complete test workflows</i> ● modify design if problems arise in beta testing ● user acceptance testing (user community)	● Product Release (the product is accepted into the user community)

Ci sono molti obiettivi per ogni singola fase. Per ognuno di questi goal c'è molta documentazione che dice "ho scelto questo perché..", devi dimostrare che tu stai pagando per qualcosa che viene prodotto. La documentazione spiega che hai fatto alcune attività che hanno portato alcuni risultati.

Tipologie di requisiti:

- *Non functional*: velocità, responsività etc.
- *Functional*: quando faccio il login devo vedere questo e quel menù

Final system: stand-alone, non il finalissimo

Inception: conditions to attain for the Life Cycle Objectives

Conditions of satisfaction	Deliverable
The stakeholders have agreed the project objectives	A vision document that states the project's main requirements, features and constraints
System scope has been defined and agreed with the stakeholders	An initial use case model (only about 10% to 20% complete)
Key requirements have been captured and agreed with the stakeholders	A Project Glossary
Cost and schedule estimates have been agreed with the stakeholders	An initial Project Plan
A business case has been raised by the project manager	Business Case
The project manager has performed a risk assessment	A Risk Assessment document or database
Confirmation of feasibility through technical studies and/or prototyping	One or more throwaway prototypes
An outline architecture	An initial architecture document

Per soddisfare la condizione devi fare qualcosa.

Stakeholder: in qualche modo è interessato al nostro software.

I vincoli sono molto importanti, perché molti requisiti diventeranno *functional requirements*, mentre altri diventano *non functional* (una banca richiede un software di gestione di un ATM, i soldi sono presi entro 3 secondi, è un vincolo che rientra nei *non functional requirements*, specifica quindi **come** non cosa). Quelli rossi sono quelli che ci interessano nel nostro progetto.

Come qualcuno deve usare il software è specificato nello *Use case*: qualcuno interagisce in qualche modo per ottenere un certo risultato (una informazione, lo stato del software...). Vogliamo catturare il 10/20% di tutti i possibili use case e un glossary, nel quale sono salvati termini specifici usati, per esempio per capire il ruolo del customer.

Devi anche preparare un prototipo funzionante che deve essere consegnabile.

Elaboration: conditions to attain for the Life Cycle Architecture

Conditions of satisfaction	Deliverable
A resilient, robust executable architectural baseline has been created The executable architectural baseline demonstrates that important risks have been identified and resolved	The executable architectural baseline UML Static Model UML Dynamic Model UML Use Case Model
The vision of the product has stabilized	Vision document
The risk assessment has been revised	Updated Risk Assessment
The business case has been revised and agreed with the stakeholders	Updated Business Case
A project plan has been created in sufficient detail to enable a realistic bid to be formulated for time, money and resources in the next phases The stakeholders agree to the project plan The business case has been verified against the project plan	Updated Project Plan Business Case and Project Plan
Agreement is reached with the stakeholders to continue the project	Sign-off document

Usiamo tutti i modelli UML.

Construction: conditions to attain for the Initial Operational Capability

Conditions of satisfaction	Deliverable
The software product is sufficiently stable and of sufficient quality to be deployed in the user community	The software product The UML model Test suite
The stakeholders have agreed and are ready for the transition of the software to their environment	User manuals Description of this release
The actual expenditures vs. the planned expenditures are acceptable	Project Plan

Transition: conditions to attain for the Product Release

Conditions of satisfaction	Deliverable
Beta testing is completed, necessary changes have been made, and the users agree that the system has been successfully deployed	The software product
The user community is actively using the product	
Product support strategies have been agreed with the users and implemented	User support plan User manuals

Bisogna pensare ai possibili input e output per ogni funzione del nostro prodotto.

Test suite: testi ogni singola cosa, per ogni funzione devi avere almeno una o due *function test*. Stai doppiando il numero di software per il test in questo modo. Vuoi poi testare tutto il modello quando il software è sull'hw del client.

Lo user manual è importante: il software può essere manipolato anche dal cliente, deve capire come funziona, come testarlo, come cambiarlo.

3.1. Workflow dei requisiti

2. The requirements workflow

- ✓ Requirements: statements on *what* the system should do (functional) and *how* it should do it (constraints, properties, non-functional)



- ✓ Well-formed requirements:

requirements for an automated teller machine (ATM)

functional requirements:

1. The ATM system shall check the validity of the inserted ATM card.
2. The ATM system shall validate the PIN number entered by the customer.
3. The ATM system shall dispense no more than \$250 against any ATM card in any 24-hour period.

non-functional requirements:

1. The ATM system shall be written in C++.
2. The ATM system shall communicate with the bank using 256-bit encryption.
3. The ATM system shall validate an ATM card in three seconds or less.
4. The ATM system shall validate a PIN in three seconds or less.

- ✓ Example:

Hai da stilare una lista, per fare ciò devi chiedere al client quali sono i requisiti che vogliono, ma devi anche capire come vanno implementati. I *functional requirements* sono le funzionalità che devi fornire, i *non functional* dicono in che modo devi implementare una funzionalità, per esempio una soglia di sicurezza.

Questi due tipi provengono da due tipi diversi di interview, i *functional* sono più facili da capire dall'idea del client, mentre i *non functional* vengono dai vincoli dello scenario.

Il punto 4 in figura appartiene ai non functional: è un vincolo di uno scenario più che un non functional requirement, ma viene considerato come tale. In generale, sono meno evidenti rispetto ai primi, sono vincoli di tecnologia e noi li decideremo perché siamo gli esperti. I *non functional* sono molto precisi, vanno sempre discussi a lungo prima di sceglierli. Ci sono delle linee guida per derivarli.

✓ Questions helping to identify actors:

To find actors ask:
"Who or what uses or interacts with the system?"

- Who or what uses the system?
- What roles do they play in the interaction?
- Who installs the system?
- Who starts and shuts down the system?
- Who maintains the system?
- What other systems interact with this system?
- Who gets and provides information to the system?
- Does anything happen at a fixed time?

✓ Questions helping to identify use cases:

- What functions will a specific actor want from the system?
- Does the system store and retrieve information? If so, which actors trigger this behavior?
- Are any actors notified when the system changes state?
- Are there any external events that affect the system? What notifies the system about those events?

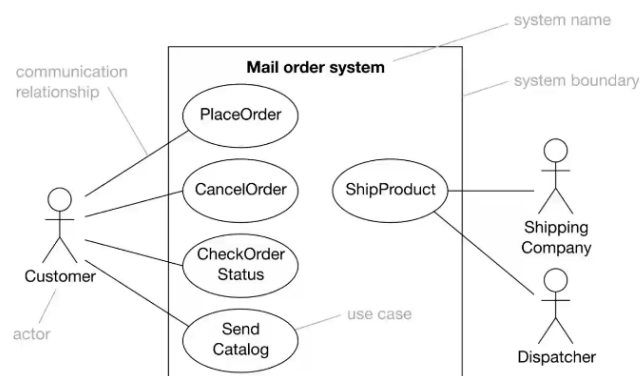
Ogni persona che interagisce col software ha una sequenza di operazioni da fare col software. Dipende chi intervisti, ogni persona avrà una certa idea di cosa fare, poiché ognuno ha differenti *functional requirements*. Da ciò si capisce che tutto dipende dagli attori, perciò è molto importante capire gli attori del sistema.

Perché nella prima domanda è presente anche "What"? Questo perché un attore può avere anche un altro sistema, che comunica con il primo magari.

In generale quando pensiamo agli attori non pensiamo al ruolo tecnico, ma va ricordato che anche i tecnici sono degli attori. Per esempio, le persone che avviano e spengono il sistema sono tecnici, ma possono essere anche non umani, essendo automatico fatto da un software.

Use cases: come gli attori utilizzano il software, molto semplice da derivare una volta conosciuti gli attori. Devi effettuare interview per capire chi sono gli attori e cosa vorrebbero.

✓ The UML use case diagram:

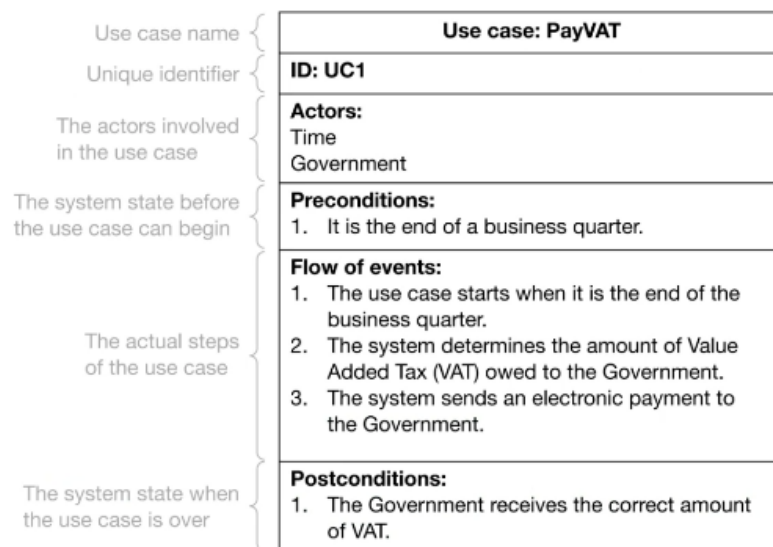


✓ The *project glossary*: is a list of *key business terms*, related *definitions*, *synonyms* (different terms for the same concept → use a unique preferred term) and *homonyms* (same term for different concepts → qualify such terms)

✓ Use case specification: (a) pre/post-conditions, things that must be true before/after the start/end of the use case; (b) flow of events, steps in the use case.

Basta un rettangolo che rappresenta il sistema da progettare, ogni “case” è dentro ad esso, insieme agli attori che usano il sistema attraverso i case, alcuni attori possono condividere gli stessi. Hai bisogno di un verbo e un nome/oggetto: il tutto va tenuto più “corto” possibile in termini di lunghezza delle parole. Anche la descrizione di un attore deve essere corta. Come scegli i nomi? Dal glossario del progetto, che è uno dei deliverable.

✓ Anatomy of a detailed use case:



Se si avessero troppi use case bisogna scegliere solo quelli fondamentali, ogni use case deve essere portato nella forma in figura. Ogni item nella documentazione deve essere numerato e listato.

Ogni use case è *self contained*: ci sono una serie di interazioni che partono da una precondizione e finiscono con post condizioni, ovvero un risultato che cambia lo stato del sistema. Nell'esempio in figura hai una *timing condition*: quando devi pagare le tasse. La condizione post è il cambiamento del sistema per quanto riguarda il pagamento. Hai il *flow*: un certo numero di azioni che portano alla soddisfazione della condizione post.

La prima azione nel flow è sempre qualcosa del tipo “lo use case inizia con/quando”. Poi hai le altre informazioni che riguardano le interazioni per raggiungere le condizioni post

✓ Branching within a flow: **IF**

Use case: ManageBasket
ID: UC10
Actors: Customer
Preconditions: 1. The shopping basket contents are visible.
Flow of events: 1. The use case starts when the Customer selects an item in the basket. 2. If the Customer selects "delete item" 2.1 The system removes the item from the basket. 3. If the Customer types in a new quantity 3.1 The system updates the quantity of the item in the basket.
Postconditions: 1. The basket contents have been updated.

✓ Alternative flows: e.g. for things happening under conditions potentially occurring at any step of the use case

Puoi avere dei rami come *if* e *while* come fosse lo sviluppo del codice. In questo caso in figura la post condition è solo un cambiamento nello stato del sistema, non è una azione o altro. Puoi avere anche grandi cambiamenti, non devi usare *if* ma un flusso alternativo in questi casi.

✓ Repetition within a flow: **FOR**

?

n. For (iteration expression)
n.1. Do something
n.2. Do something else
n.3. ...
n+1.

Use case: FindProduct
ID: UC12
Actors: Customer
Preconditions:
Flow of events: 1. The Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products then 5.1. For each product found 5.1.1. The system displays a thumbnail sketch of the product. 5.1.2. The system displays a summary of the product details. 5.1.3. The system displays the product price. 6. Else 6.1. The system tells the Customer that no matching products could be found.
Postconditions:
Alternative flow: 1. At any point the Customer may move to different page.
Postconditions:

Il *for* si usa quando hai una lista di oggetti. In questo caso in figura non hai precondizioni perché puoi trovare i prodotti quando vuoi.

✓ Repetition within a flow: **WHILE**

n. While (Boolean condition)
 n.1. Do something
 n.2. Do something else
 n.3. ...
 n+1.

Use case: ShowCompanyDetails	
ID: UC13	
Actors: Customer	
Preconditions:	
Flow of events: 1. The use case starts when the Customer selects "show company details". 2. The system displays a web page showing the company details. 3. While the Customer is browsing the company details 3.1. The system plays some background music. 3.2. The system displays special offers in a banner ad.	
Postconditions:	

✓ Requirements tracing: many-to-many relationship between requirements and use cases, how to discover missing use cases or missing requirements.

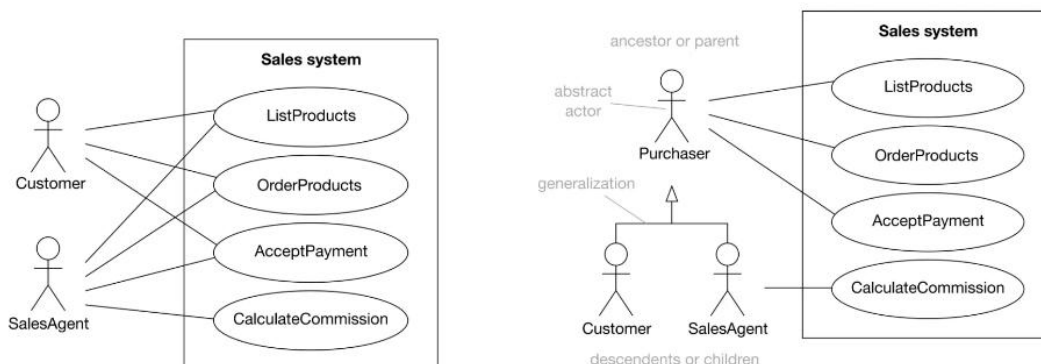
Requirements tracing links requirements in the System Requirements Specification to the use case model.

	Use case			
	UC1	UC2	UC3	UC4
Requirement	R1	X		
	R2		X	
	R3			X
	R4		X	
	R5	X		

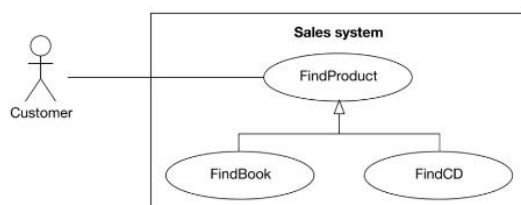
Il *while* è legato ad una condizione booleana. Non stiamo parlando di programmazione, sono solo interazioni col sistema, cose che succedono nel mondo reale. Abbiamo bisogno di una lista numerata, perché in tutta la documentazione ecc devi tracciare le scelte progettuali, dal codice al requisito. Ogni requisito deve essere coperto da almeno uno uso case, può diventare una funzione o una classe.

Con la tabella in fondo vedi che un requisito è coperto da uno use case.

✓ *Actor generalization*: the descendent actors inherit the roles and relationships to use cases held by the parent actor



✓ *Use case generalization*: the child use case inherits features from the parent use case, can add or change (override) inherited features (pre/post condition, steps in flow...)



Feature is ...	Typographical convention
Inherited without change from the parent	Normal text
Overridden	<i>Italic text</i>
Added	Bold text

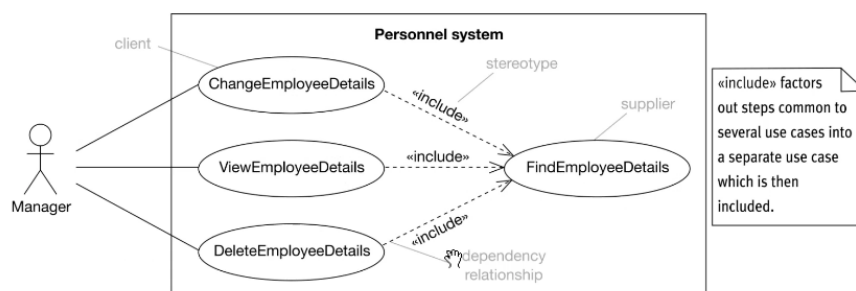
Ci sono due attori in figura: customer e sales agent, condividono l'accesso a 3 use case, quindi entrambi hanno gli stessi bisogni quando usano il sistema con questi 3. In questo caso si può usare una generalizzazione, crei un nuovo attore che li generalizza. Quelli che vengono generalizzati ereditano gli use case del generale.

Nel caso sotto l'unica differenza nelle "Find" è il nome e magari le informazioni che vengono fornite.

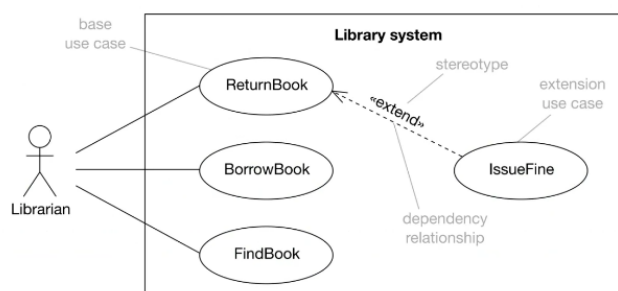
Use case: FindProduct	Child use case: FindBook	Child use case: FindCD
ID: UC12	ID: UC16	ID: UC17
Actors: Customer	Parent Use Case ID: UC12	Parent Use Case ID: UC12
Preconditions: $\emptyset$	Actors: Customer	Actors: Customer
Flow of events: 1. The Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products then 5.1. The system displays a list of the matching products. 6. Else 6.1. The system tells the Customer that no matching products could be found.	Preconditions:	Preconditions:
Postconditions:	Flow of events: 1. The Customer selects "find book". 2. The system asks the Customer for book search criteria consisting of author name, title, ISBN, or topic. 3. The Customer enters the requested criteria. 4. The system searches for books that match the Customer's criteria. 5. If the system finds some matching books then 5.1. The system displays a page showing details of a maximum of five books. 5.2. For each book on the page the system displays the title, author, price, and ISBN. 5.3. While there are more books 5.3.1. The system gives the Customer the option to display the next page of books. 6. Else 6.1. The system redisplay the "find book" search page. 6.2. The system tells the Customer that no matching products could be found.	Flow of events: 1. The Customer selects "find CD". 2. The system asks the Customer for CD search criteria consisting of artist, title, or genre. 3. The Customer enters the requested criteria. 4. The system searches for CDs that match the Customer's criteria. 5. If the system finds some matching CDs then 5.1. The system displays a page showing details of a maximum of ten CDs. 5.2. For each CD on the page the system displays the title, artist, price, and genre. 5.3. While there are more CDs 5.3.1. The system gives the Customer the option to display the next page of CDs. 6. Else 6.1. The system redisplay the "find CD" search page. 6.2. The system tells the Customer that no matching products could be found.
Alternative flow: 1. At any point the Customer may move to a different page.	Postconditions:	Postconditions:
Postconditions:	Alternative flow: 1. At any point the Customer may move to a different page.	Alternative flow: 1. At any point the Customer may move to a different page.
	Postconditions:	Postconditions:

✓ The «include» relationship between use cases includes the behavior of a supplier use case into the flow of a client use case. The client use case is not complete without all of its supplier use cases. The supplier use cases may or may not be complete (behavior fragment, it is not instantiable, it cannot be triggered directly by actors)

Abbiamo un parent use case e due child use case: infatti vediamo come i punti 5.2, 5.3 siano diversi, sono diversi per *findbook* e *findCD*, il resto è uguale.

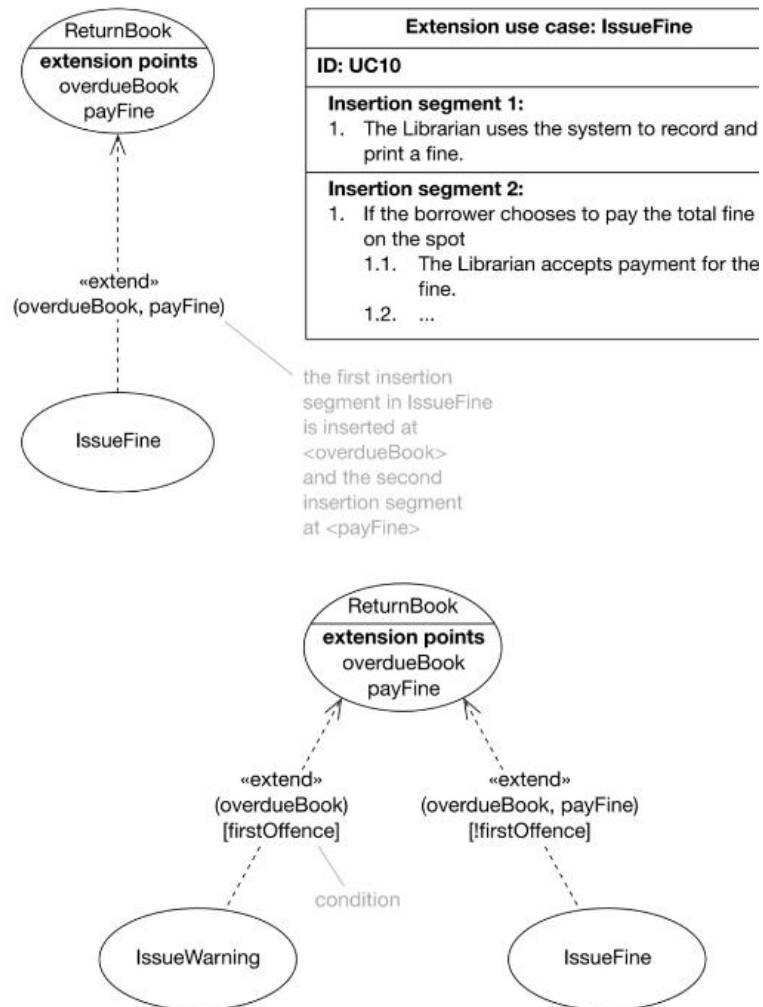


✓ The «extend» relationship between use cases adds new behavior to a base use case. The base use case is complete without its extensions (that usually are not complete).



In entrambi i casi hai una freccia, ma l'inclusione è una forte connessione, devi fare qualcosa obbligatoriamente per finire lo use case. La extend non è una relazione forte, i due use case esistono indipendentemente.

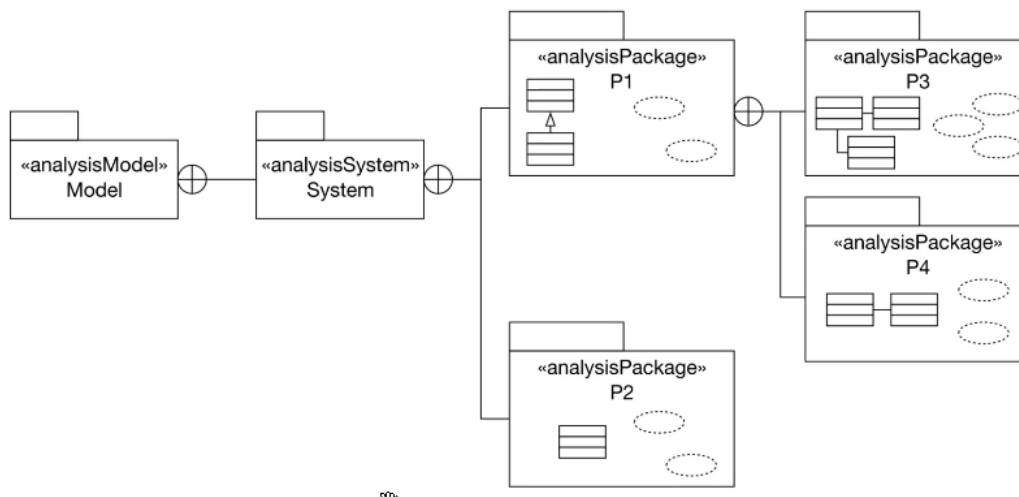
Puoi avere multiple extension (point).



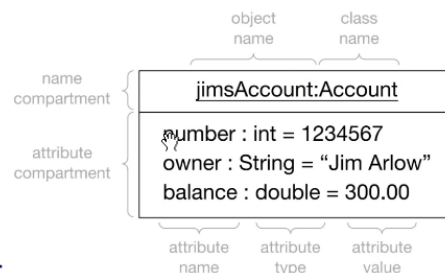
3.2. Workflow di Analisi

2. The Analysis workflow

- ✓ The aim is to produce an analysis model on *what* the system needs to do, leaving details on *how* it will do it to the design workflow
- ✓ Key artefacts produced: *analysis classes* (model key concepts in the business domain) and *use case realizations* (illustrate how instances of the analysis classes can interact to realize system behavior specified by a use case).

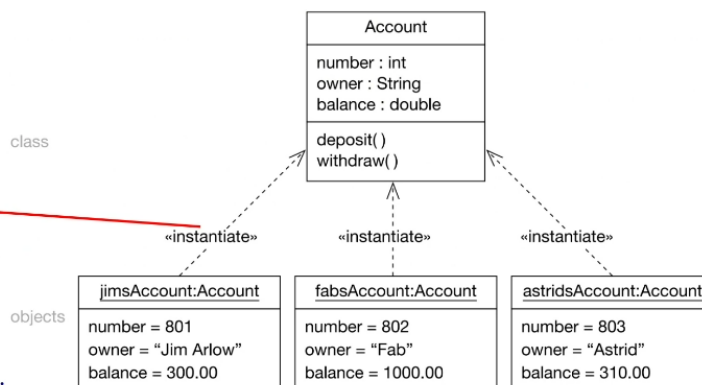


Vuoi unire le informazioni per iniziare a sviluppare il software. Vuoi una iniziale forma di classi che per lo stato corrente del lavoro, perché ad ogni loop vengono refiniti i requisiti e poi le classi.



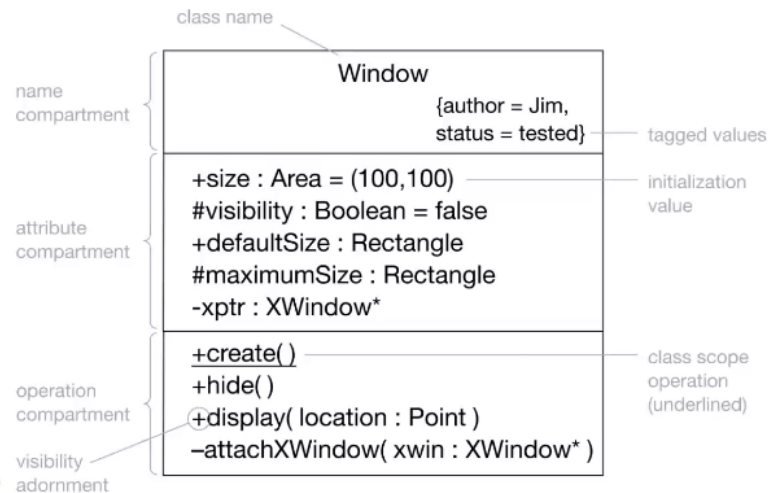
- ✓ UML object notation:

dependency relationship:
a change to the class affects the object



- ✓ UML class notation:

Account è la classe che ha alcuni attributi, poi si ha un numero di account che sono istanze della classe: hai un nome per ogni istanza, ovviamente hai anche la tipologia che è *Account*.



✓ UML class notation:

✓ Class name is CamelCase (no spaces or special symbols because they are used in languages)

✓ Avoid abbreviations of class name



✓ Attribute compartment:

Possiamo aggiungere alcune informazioni, come la **visibilità**, sia per i metodi che gli attributi (privata o pubblica). Non sono le classi finali, servono per refinire i requisiti.

Adornment	Visibility Name	Semantics
+	Public visibility	Any element that can access the class can access any of its features with public visibility
-	Private visibility	Only operations within the class can access features with private visibility
#	Protected visibility	Only operations within the class, or within children of the class, can access features with protected visibility
~	Package visibility	Any element that is in the same package as the class, or in a nested subpackage, can access any of its features with package visibility

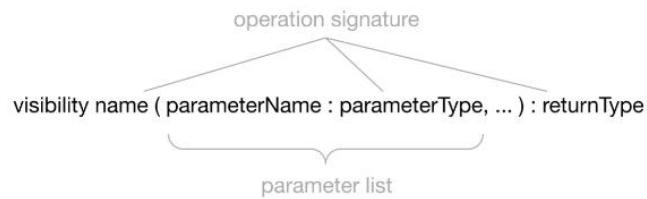
✓ Visibility adornment:

✓ Initial values and visibility are not used in the analysis model.

✓ Multiplicity (number of things) is more used in design, sometimes in analysis:

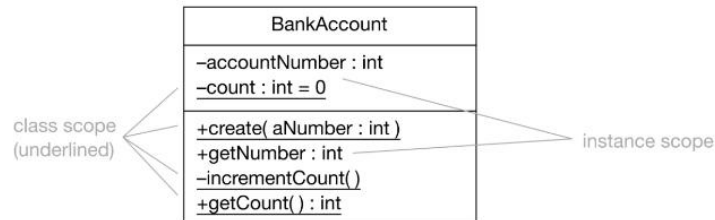
multiplicity expression		
address [3]: String		an address is composed of an array or three Strings
name [2..*]: String		a name is composed of two or more Strings
emailAddress [0..1]: String		an emailAddress is composed of one String or null

Puoi anche esprimere la molteplicità, puoi riferirti con un numero.



✓ Operation compartment:

✓ Instance and class scope (one version shared by all objects):



✓ Activity “analyze a use case”: creating analysis classes and use case realizations

✓ *Analysis class* is in the problem domain (in which the need for the system arises)

Se usi un linguaggio di programmazione “statically typed”, devi decidere il tipo di ogni argomento della funzione. Consideri anche lo scope. Un gruppo di use case possono diventare una classe. Tutte le funzionalità di uno use case deve almeno stare in una classe.

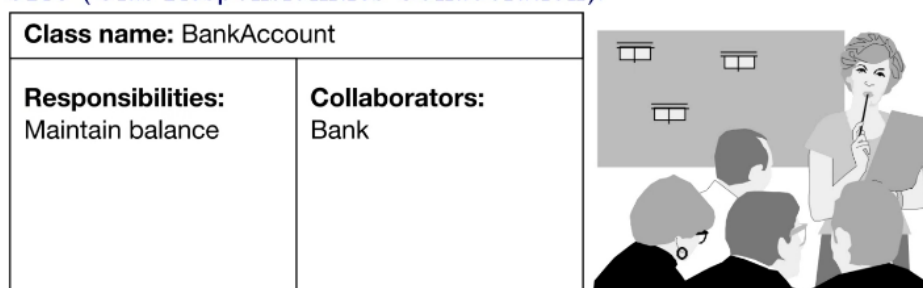
- its name reflects its intent;
- it is a crisp abstraction that models one specific element of the problem domain;
- it maps on to a clearly identifiable feature of the problem domain;
- it has a small, well-defined set of responsibilities;
- it has high cohesion (cohesive set of responsibilities towards the same goal);
- it has low coupling to other classes (number of relationships).

87

✓ Beware of large classes, functoids, omnipotent classes, deep inheritance

✓ How to find analysis classes: noun and noun phrases indicate candidate classes or attributes, whereas verb and verb phrases indicate candidate responsibilities.

✓ CRC (Clas Responsibilities Collaboration):

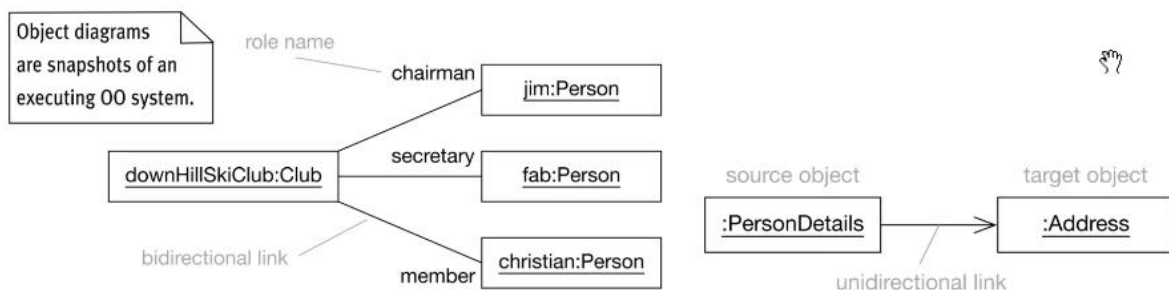


Cosa possiamo implementare come classe? È sempre importante capire quali sono gli obiettivi principali da incorporare nella classe. Partendo dal glossario devi capire le funzionalità che devono essere indirizzate dai metodi della classe.

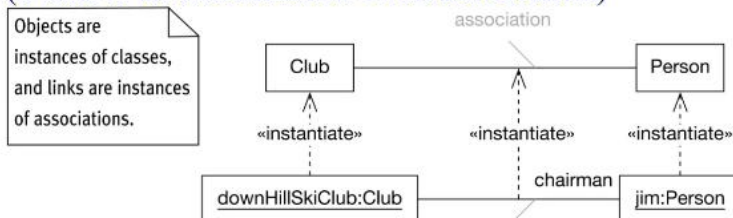
Si parte da piccole classi che risolvono specifici problemi, poi con altre informazioni si creano le *util class*, dove hai tutte le funzionalità: ecco ciò non deve essere fatto, ovvero le classi non devono essere enormi, devi aggiungere solo le necessarie funzionalità che mancano. Gli obiettivi in comune devono essere accomunati in una classe, le funzionalità devono coprire solo quei problemi e quelle responsabilità.

Le classi non devono interagire troppo fra loro, massimo 1 o 2 in comunicazione, sennò specifichereesti responsabilità condivise fra loro. Al massimo si può usare una *super class*, una generalizzazione di altre classi, che magari una che copre un certo set di responsabilità. Ovviamente ogni pezzo del progetto deve comunicare con gli altri.

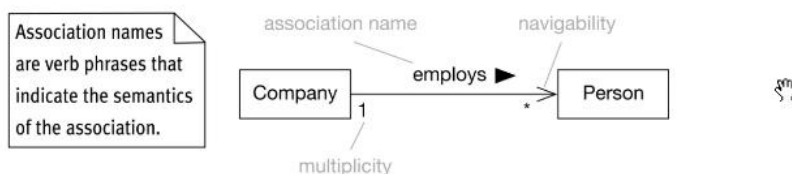
- ✓ *Link* in object diagram: it allows messages to be sent from one object to the other (pointer, references, etc.)



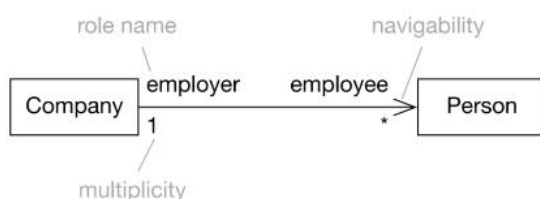
- ✓ *Association* in class diagram: relationship between classes (a link is an instantiation of an association)



- ✓ “A *Company* employs many *Persons* (a black triangle denotes the reading direction), or “Each *Person* works for one *Company*” at any point in time. Over time a *Person* object might be employed by a sequence of *Company* objects.



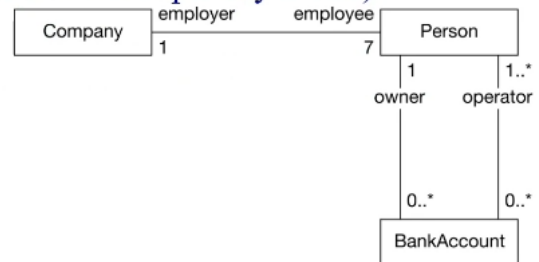
- ✓ Associations can have roles instead of association name:



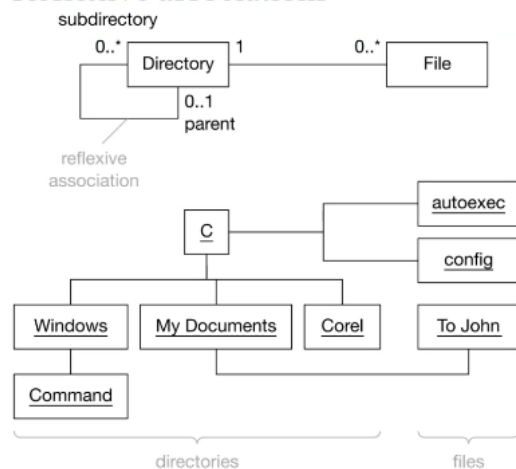
Nella freccia ci puoi mettere anche la molteplicità, puoi avere anche ruoli invece che *association name*, ma è circa la stessa cosa.

✓ Multiplicity (there is not “default” multiplicity if it is not explicitly stated):

Adornment	Semantics
0..1	Zero or 1
1	Exactly 1
0..*	Zero or more
*	One or more
1..*	1 or more
1..6	1 to 6
1..3,7..10,15, 19..*	1 to 3 or 7 to 10 or 15 exactly or 19 to many



✓ Reflexive associations



Non ci serve un numero specifico tipo da 1 a 6.

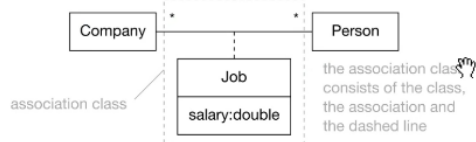


✓ Implementation of association as an attribute

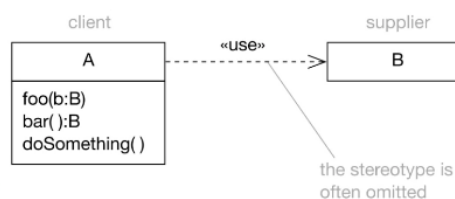
```

public class House
{
  private Address address;
}
  
```

✓ Association class (association that is also a class)



✓ Dependency (between classes, packages, object and classes)

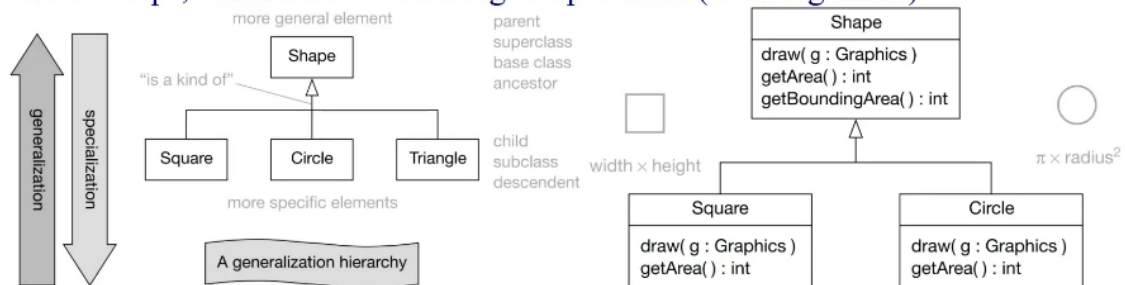


✓ The «use» dependency:

An operation of class A needs a parameter, returns a value, uses an object of class B somewhere in its implementation, but not as an attribute

Abbiamo una associazione che prevede una compagnia che ha una persona e la relazione fra loro è il lavoro.

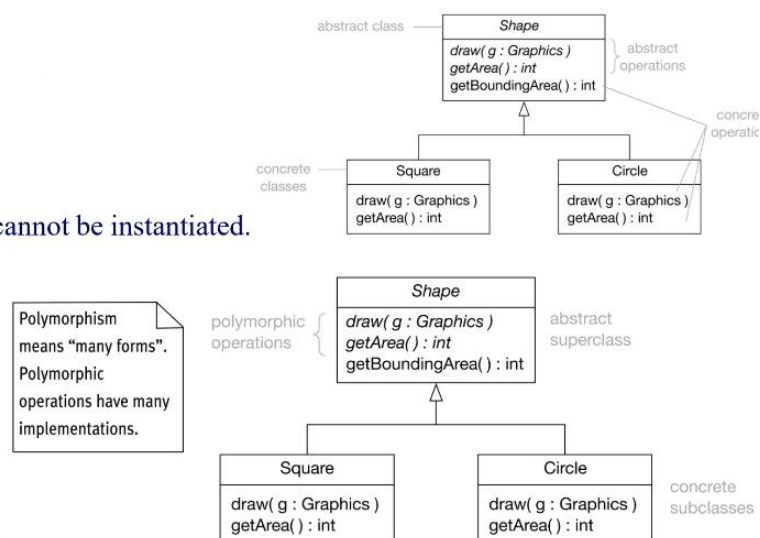
- ✓ The «call» dependency: an operation of class A invokes an operation of class B
- ✓ The «parameter» dependency: in class B, a parameter or returned value of class A
- ✓ The «send» dependency: a class A transfers data to a class B
- ✓ The «intantiate» dependency: an instance of class A
- ✓ The «access» dependency: a package P accesses the public content of package Q (Packages are used in UML to group things)
- ✓ The «import» dependency: the namespace of a package P is merged to the namespace of package Q (you do not need a qualified element name)
- ✓ Generalization: specialized (or extended) classes inherit attributes, operations, relationships, constraints. Overriding of operations (same signature)



La *call dependency* è la più importante e comune.

Non riporteremo i pacchetti importati nella documentazione, vogliamo magari solo la parte testuale non tutto il diagramma.

- ✓ Abstract class cannot be instantiated.



- ✓ Polymorphism:
there are two implementations of the Shape class, i.e., its operations have many forms (polymorphic) depending on the class of its instance (Square or Circle)
- ✓ Overriding concrete operations is considered a bad style.

- ✓ *Dynamic view*: use case realizations show how instances of the analysis classes interact to realize the functionality of the system, via the following elements:

Element	Purpose
Analysis class diagrams	Show the analysis classes that interact to realize the use case
Interaction diagrams	Show collaborations and interactions between specific instances that realize the use case – they are “snapshots” of the running system
Special requirements	The process of use case realization may well uncover new requirements specific to the use case – these must be captured
Use case refinement	New information may be discovered during realization that means the original use case has to be updated

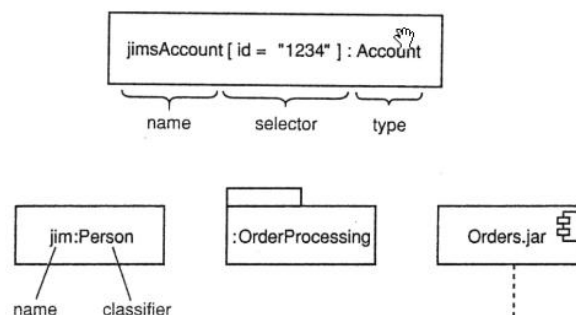
- ✓ Types of interaction diagrams: *communication diagram* and *sequence diagram* (dynamic interaction between instances in terms of *messages*).

Message flow	Semantics
	Procedure call – the sender waits until the receiver has finished This is the most common option
	Asynchronous communication – the sender carries on as soon as the message has been sent; it does not wait for the receiver This is often used when there is concurrency
	Return from a procedure call – the return is always implicit in a procedure call, but it may be explicitly shown using this arrow

Fino ad ora abbiamo visto una visione statica, non vediamo interazioni, le immaginiamo soltanto. Vogliamo qualcosa di più dinamico, questo si fa con gli *interaction diagram*. Andiamo sempre più vicino alla visione del codice. Abbiamo bisogno di entità che comunicano attraverso i *message call*.

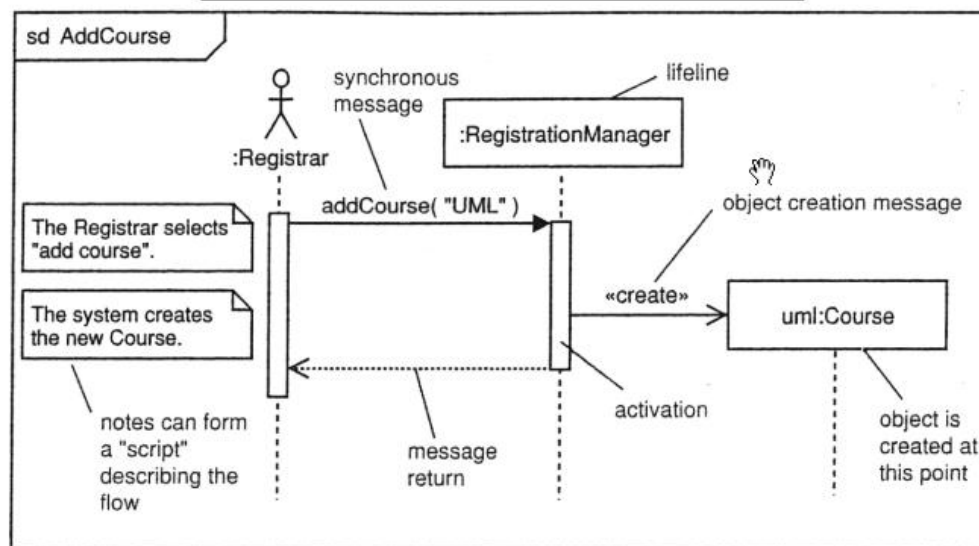
La prima avvia la vita del flow, l'ultima la chiude.

- ✓ *Lifeline*: a participant in an interaction, an instance of a specific classifier (a classifier is a type of thing, such as actor, class, use case; an instance is a concrete example of such thing such as a specific actor, class, use case).



- ✓ *Selector*: a Boolean condition to select a single instance
- ✓ Interaction diagrams are not verbatim transcriptions of a use case, they are illustrations of how the use case behavior is realized by analysis classes
- ✓ Use case and sequence diagram

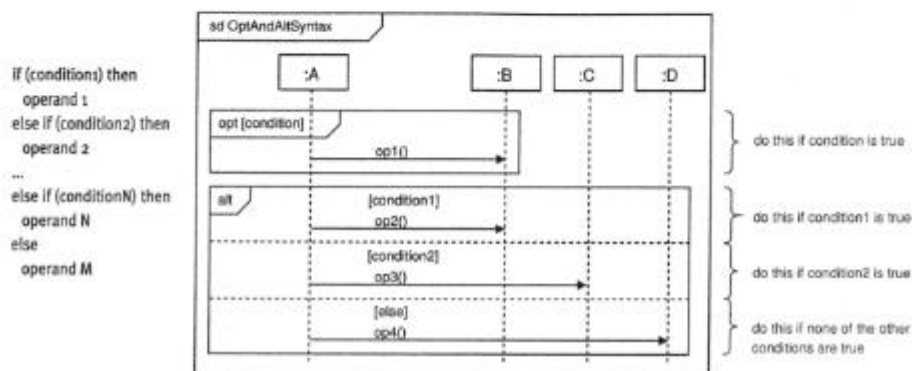
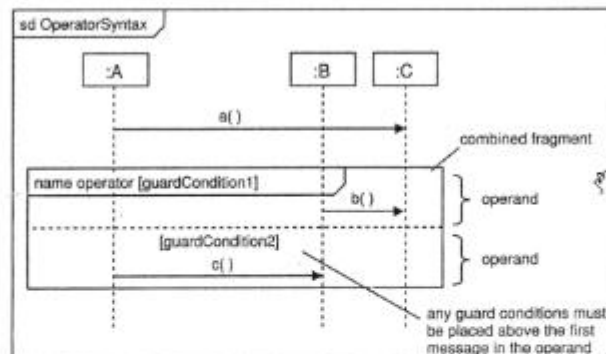
Use case: AddCourse
ID: UC8
Actors: Registrar
Preconditions: The Registrar has logged on to the system.
Flow of events: 1. The Registrar selects "add course". 2. The system accepts the name of the new course. 3. The system creates the new course.
Postconditions: A new course has been added to the system.



Lo use case copre il flusso di eventi, ogni interazione deve diventare una freccia nel *sequence diagram*. Gli use case possono essere tradotti facilmente in un *sequence diagram*.

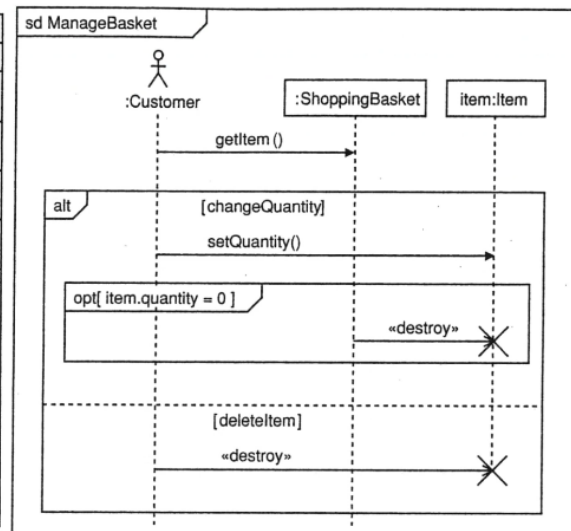
Non vanno usate linee troppo lunghe, bisogna focalizzarsi sulle interazioni principali se ce ne sono troppe. Ogni interazione è un messaggio e può triggerare una nuova chiamata. Il *returning* avviene ma a volte può non essere rappresentato.

- ✓ Combined fragment and operators: combined fragments are areas of the sequence diagram; the operator determines *how* its operands are executed, whereas the guard condition determines *whether* their operand execute.

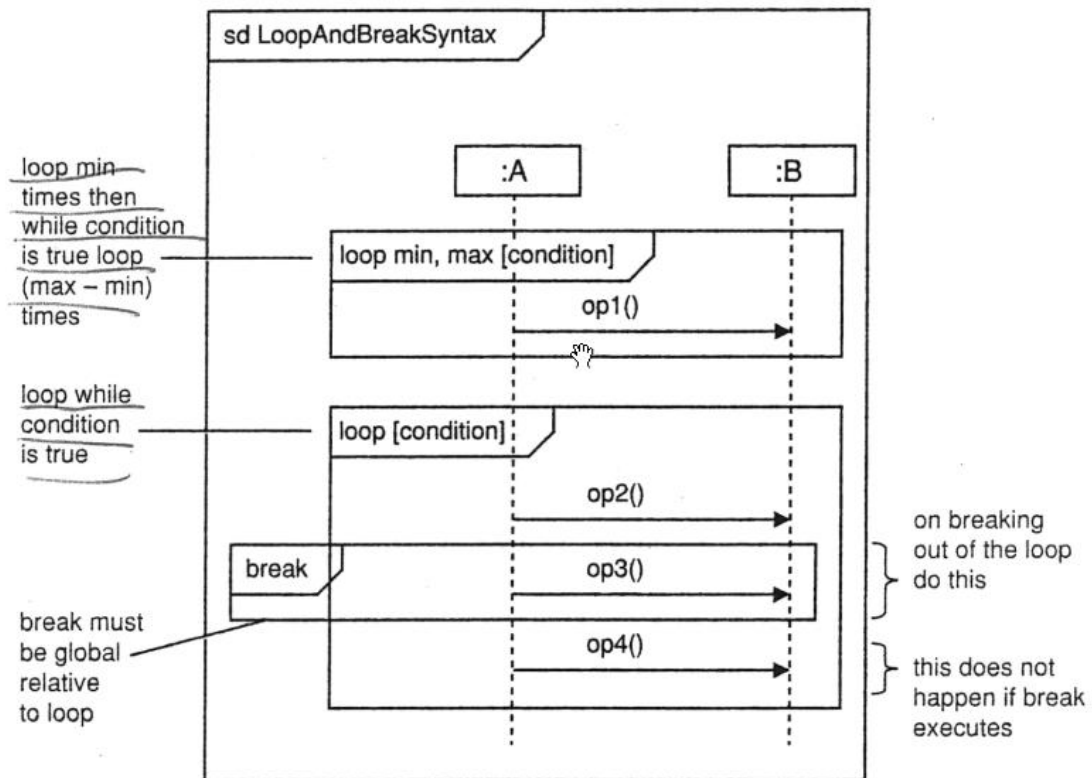


Il primo è un *if else*, il secondo è uno *switch*.

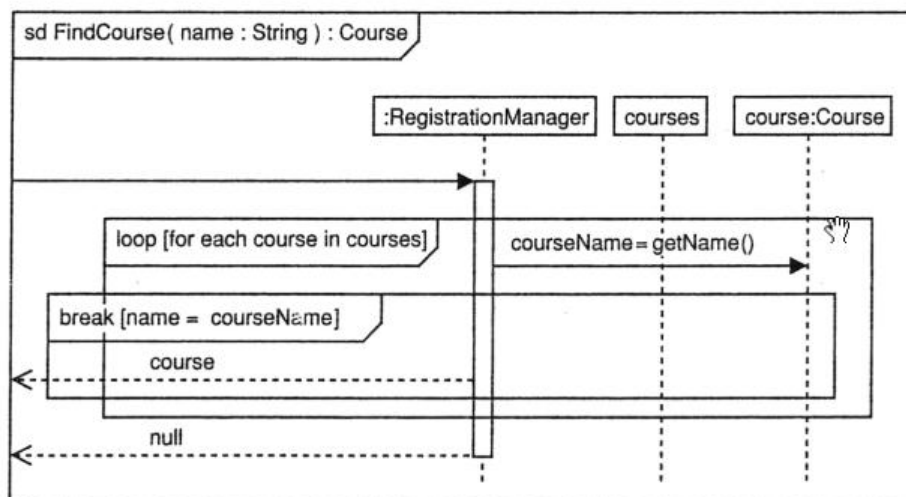
Use case: ManageBasket
ID: 2
Brief description: The Customer changes the quantity of an item in the basket.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The shopping basket contents are visible.
Main flow: 1. The use case starts when the Customer selects an item in the basket. 2. <u>If the Customer selects "delete item"</u> 2.1 The system removes the item from the basket. 3. <u>If the Customer types a new quantity</u> 3.1 The system updates the quantity of the item in the basket.
Postconditions: None.
Alternative flows: None.



La condizione *if* è realizzata in base alle opzioni che occorrono.



È simile al *for* visto prima, questa in figura è la sintassi di un *for* per ogni step. Il *break* parte se c'è una condizione che viene verificata, ma esso va tenuto fuori dal rettangolo del loop.

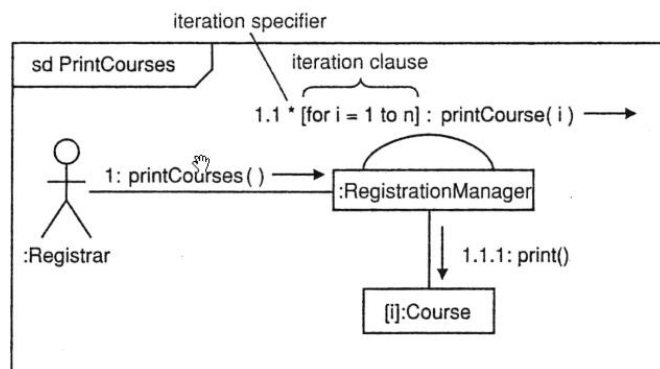
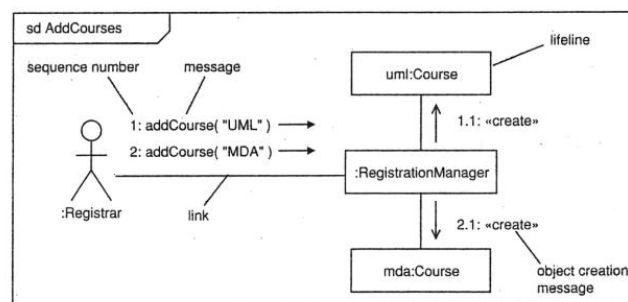


Possiamo vedere il *main loop* e il *break* che è sostanzialmente fuori. È un *for* che viene eseguito per ogni corso e se c'è uno specifico corso interrompi e lo ritorni, altrimenti se non lo trovi ritorni *null*.

Operator	Long name	Semantics
opt	option	There is a single operand that executes if the condition is true (like if ... then) 87
alt	alternatives	The operand whose condition is true is executed. The keyword else may be used in place of a Boolean expression (like select ... case)
loop	loop	This has a special syntax: loop min, max[condition] loop min times, then while condition is true, loop (max – min) times
break	break	If the guard condition is true, the operand is executed, <u>not</u> the rest of the enclosing interaction
ref	reference	The combined fragment refers to another interaction
par	parallel	All operands execute in parallel
critical	critical	The operand executes atomically without interruption
seq	weak sequencing	All operands execute in parallel subject to the following constraint: events arriving on the <i>same</i> lifeline from <i>different</i> operands occur in the same sequence as the operands occur This gives rise to a weak form of sequencing – hence the name
strict	strict sequencing	The operands execute in strict sequence
neg	negative	The operand shows invalid interactions Use this when you want to show interactions that <i>must not</i> happen

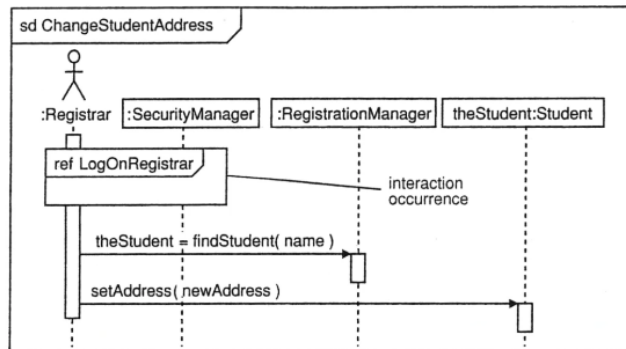
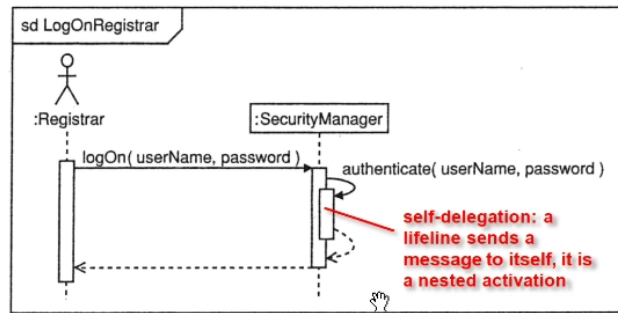
✓ Communication diagram: it is similar to sequence diagram except that there are direct links between lifelines

Nel progetto si usano in generale i primi 4, forse anche *critical* per magari eventuali errori.



Noi usiamo solo i *sequence diagram*, come in figura sopra.

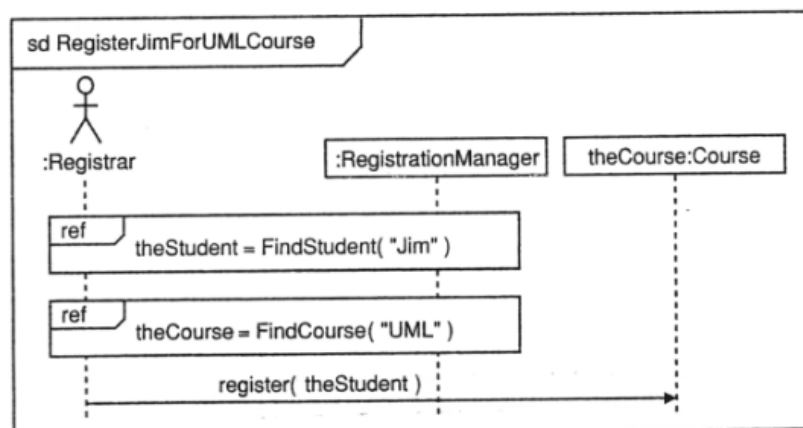
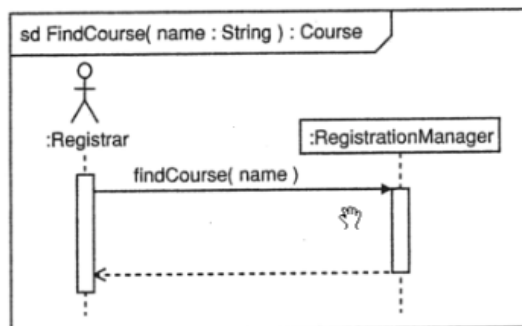
✓ Reusable interaction fragment



La classe *SecurityManager* deve usare la *authenticate* per ritornare uno specifico risultato, come ad esempio se l'autenticazione è andata a buon fine, puoi anche ritornare il risultato all'attore.

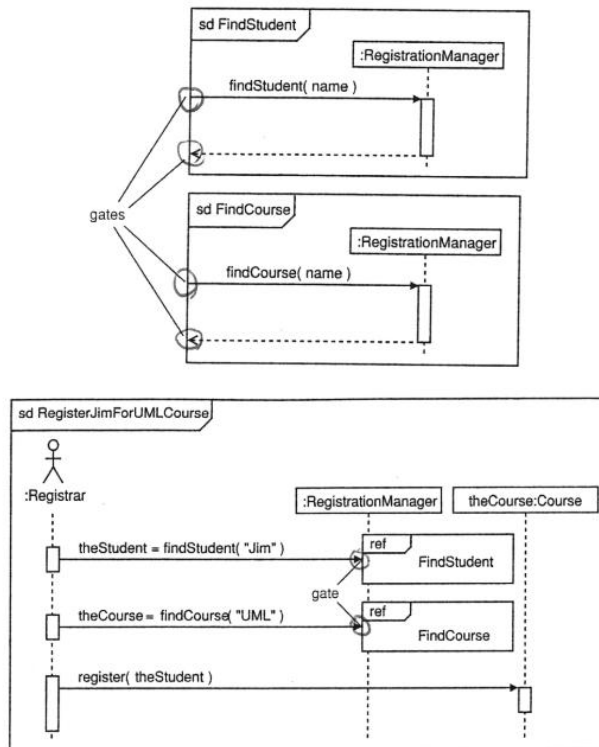
La *lifeline* può essere solo per l'interazione, può essere più corta.

✓ Parameters in reusable interaction fragment



Puoi anche passare alcuni parametri come input alle funzioni.

✓ Gates: inputs and outputs of interactions outside the frame

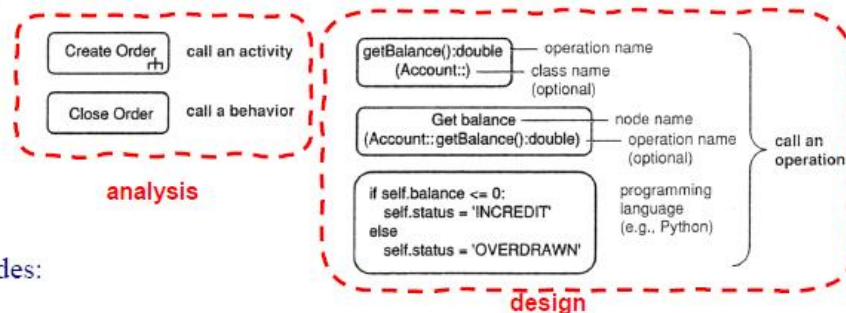


Se vuoi rappresentare l'interazione tra vari use case usi i *gates*: degli input e output point per i *sequence diagram*. Specifichi il grande *sequence diagram* sopra e quello abbozzato sotto, per rappresentare una *self delegation* in modo preciso, quando vuoi ottenere un risultato che è necessario in un grafico.

All'inizio non ti focalizzi sugli errori ma sul main flow, per generare l'analisi, che è la base di tutto. Se vuoi per esempio aggiungere requirements puoi, ripartendo dall'analisi.

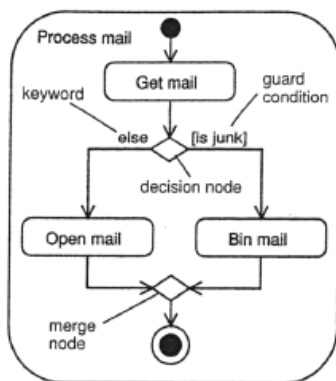
Syntax	Name	Semantics
	Initial node	Indicates where the flow starts when an activity is invoked
	Activity final node	Terminates an activity
	Flow final node	Terminates a specific flow within an activity - the other flows are unaffected
	Decision node	The output edge whose guard condition is true is traversed May optionally have a «decisionInput»
	Merge node	Copies input tokens to its single output edge
	Fork node	Splits the flow into multiple concurrent flows
	Join node	Synchronizes multiple concurrent flows May optionally have a join specification to modify its semantics

✓ Control nodes:

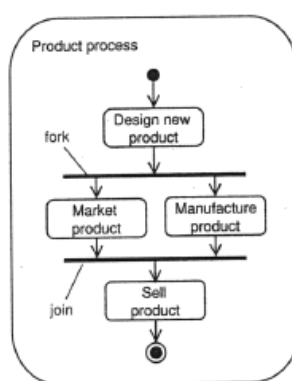


✓ Call action nodes:

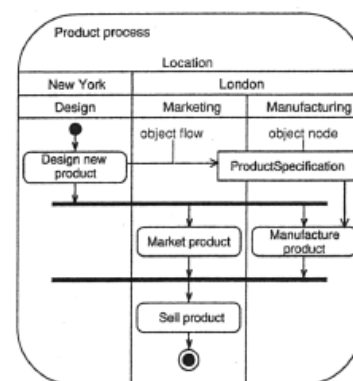
BPMN è un'evoluzione dell'UML. I primi due nodi sono quelli da cui partono le attività. Poi ci sono i nodi per dividere il flow in modo condizionale o non condizionale. Noi cerchiamo di mappare l'intero problema nell'activity diagram.



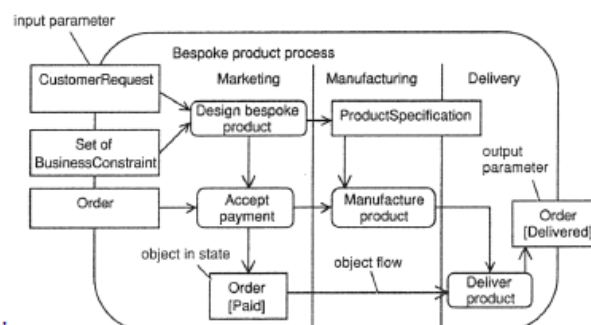
decision/merge nodes



fork/join nodes



object nodes

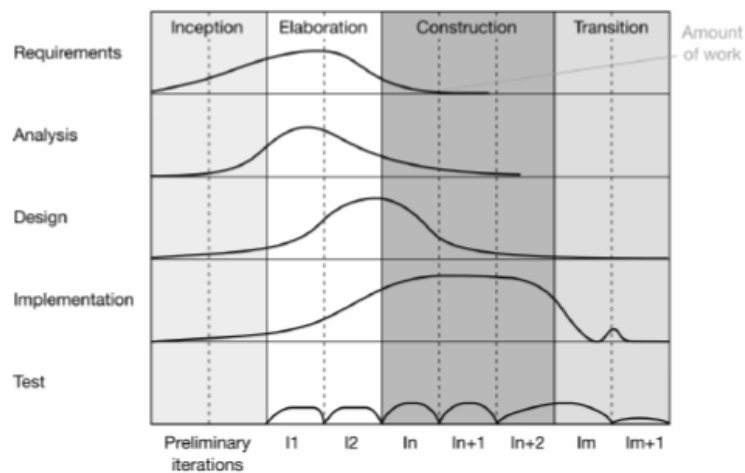


✓ I/O params and object in state:

Nel primo esempio si possono vedere punto iniziale e finale, si può vedere la realizzazione di un if else nel diagramma di attività. Ogni volta che hai un diamante necessiti di un diamante che effettua una fusione delle attività.

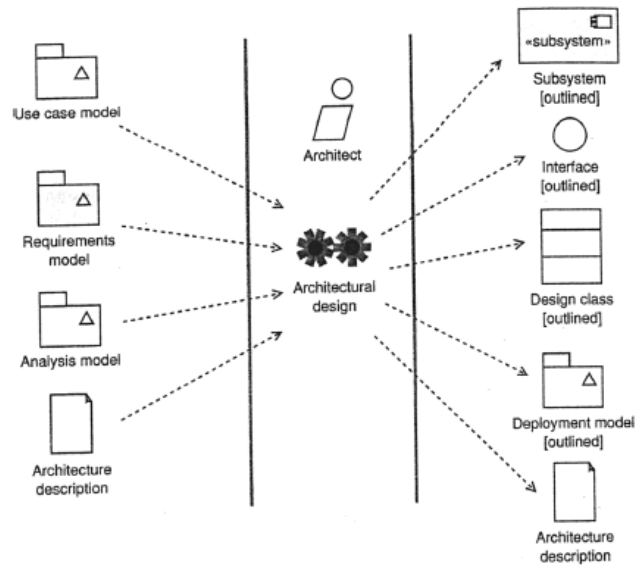
Nel secondo hai un loop.

3. The Design workflow



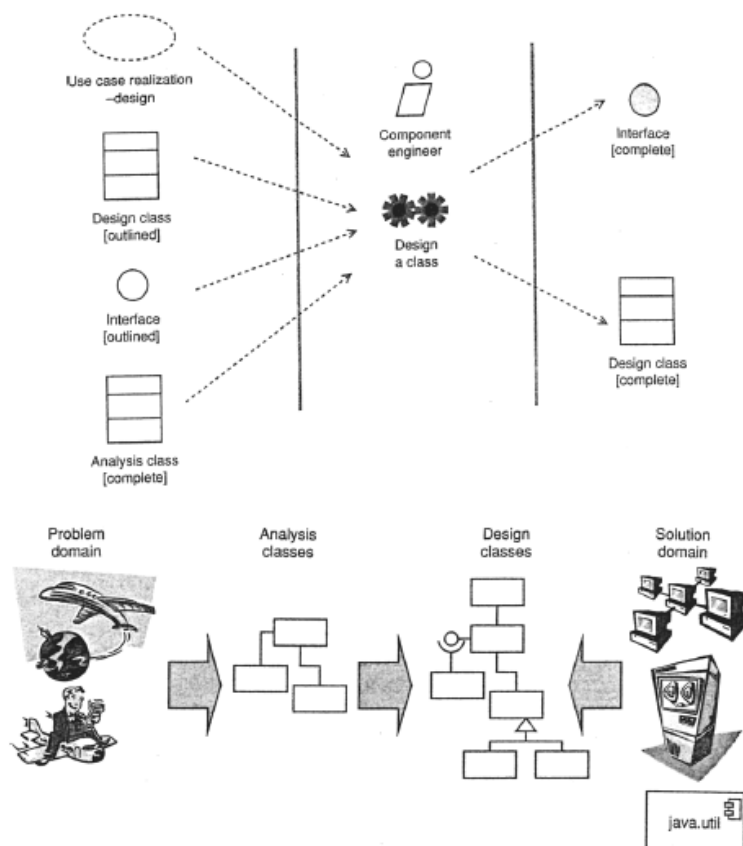
- ✓ While Requirements and Analysis workflows focus on the problem domain from the point of view of the system stakeholders, Design workflow focuses on the solution domain to provide: design subsystems, design classes, interfaces, use case realizations design, deployment diagrams.

L'obiettivo principale del design è arricchire la documentazione, permette di non dover chiedere più nulla al client, diventa molto tecnico e dettagliato.

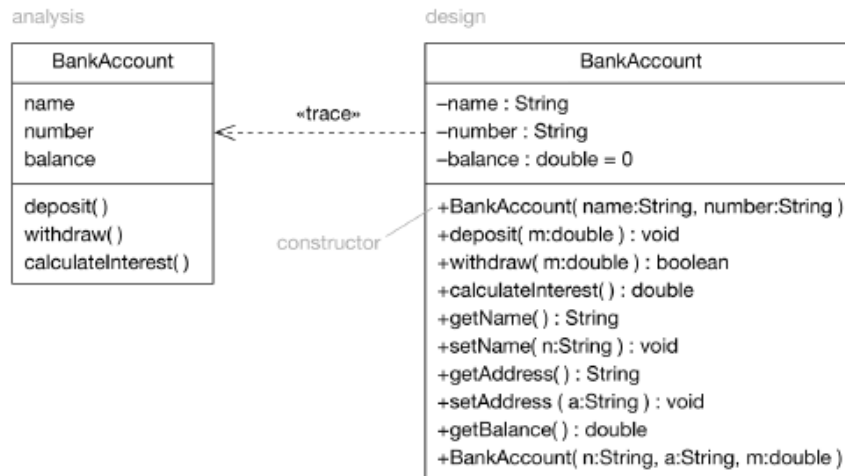


- ✓ Design classes and interfaces are first outlined and then sufficiently detailed to serve as a good basis for creating source code
- ✓ Some design classes are refinements of analysis classes. Other design classes are based on the solution domain (e.g. utility classes, communication middleware, db)

Nel design specifichi le librerie, i vincoli, il linguaggio di programmazione. Queste informazioni le sai te programmatore, non devi chiedere nulla al cliente.



- ✓ Complete the set of attributes and fully specify them including name, type, visibility and (optionally) a default value.
- ✓ Turn the operations specified in the analysis class into a complete set of one or more methods.



- ✓ A cohesive class has a small set of responsibilities that are closely related. Every operation, attribute, and association of the class is designed for the small, focused set of responsibilities.

Qui hai un diagramma appartenente all'analisi e uno al design.

Ogni classe può prestare funzionalità ad altre, nasce una dipendenza ma non ci dovrebbero essere grosse sovrapposizioni.

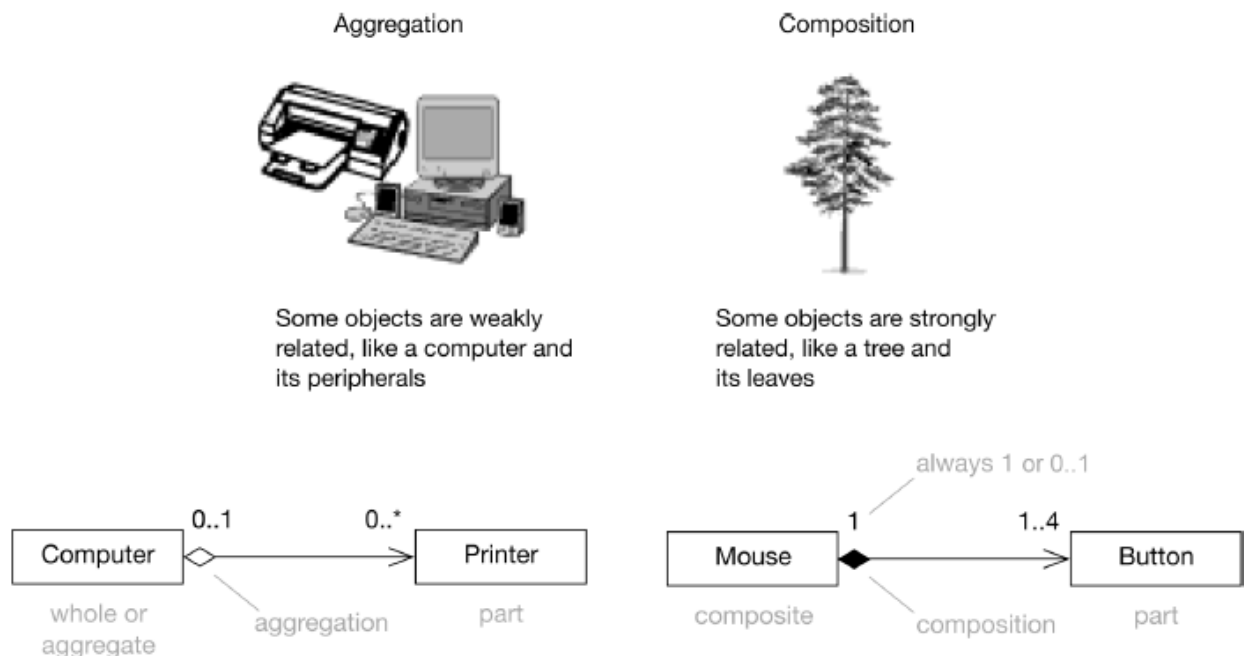
- ✓ The parts can exist (or not) independently of the aggregate, it is possible to share parts between aggregates.
- ✓ The parts can only belong to one composite at a time, no shared ownership; the composite has responsibility the creation/destruction or release of its parts.
- ✓ Multiplicity and constraints, semantics of collection (properties)



Property	Semantics
{sorted}	The collection is sorted according to some key – the key may be specified in the property, e.g. {sorted by name}
{indexed}	Each element in the set is accessible via a numeric index
{set}	Duplicates are not allowed in the collection
{lifo}	"Last in, first out" – a stack where the last element placed on the stack is the first element that can be taken off it
{queue}	A queue where the first element placed on the queue is the first element that can be taken off it

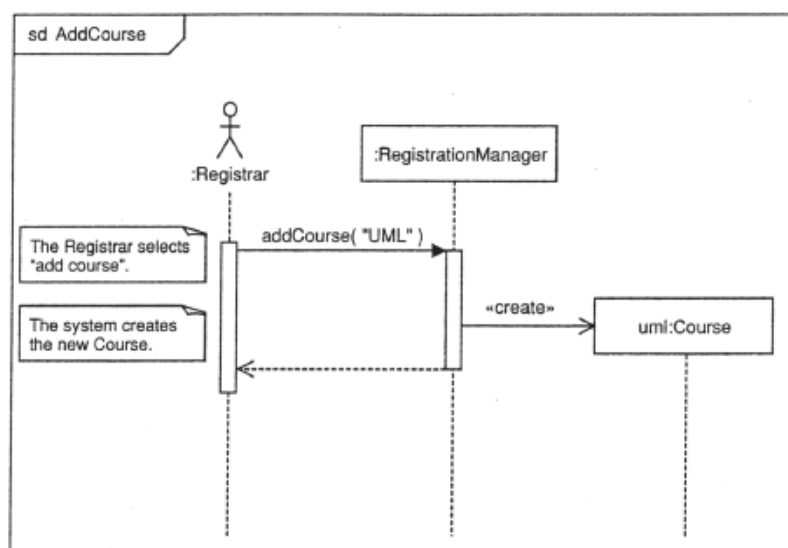
- ✓ Interfaces and components: breaking up the system into subsystems and determining their interactions via interfaces

- ✓ Operations offer a single primitive, atomic service. Do not offer multiple ways of doing the same thing, e.g. BankAccount class with operations for both single and multiple deposits (→maintenance and consistency problems).
- ✓ Refine analysis relationships: type, multiplicities, role names, navigability.



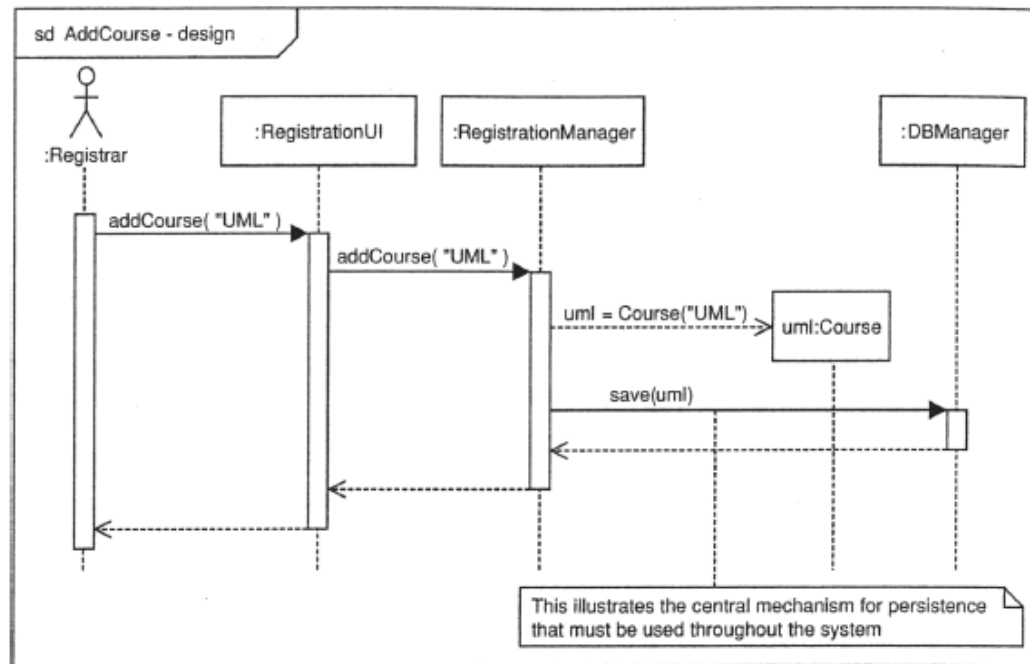
Le classi sono abbastanza per diventare classi di Design?

- ✓ Example of an analysis sequence diagram



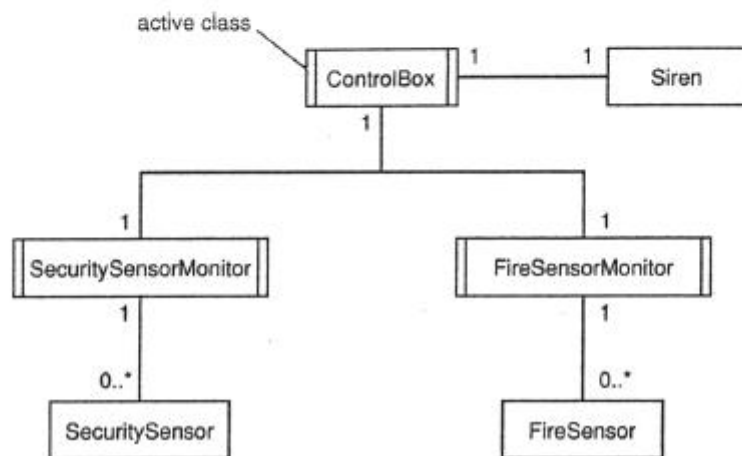
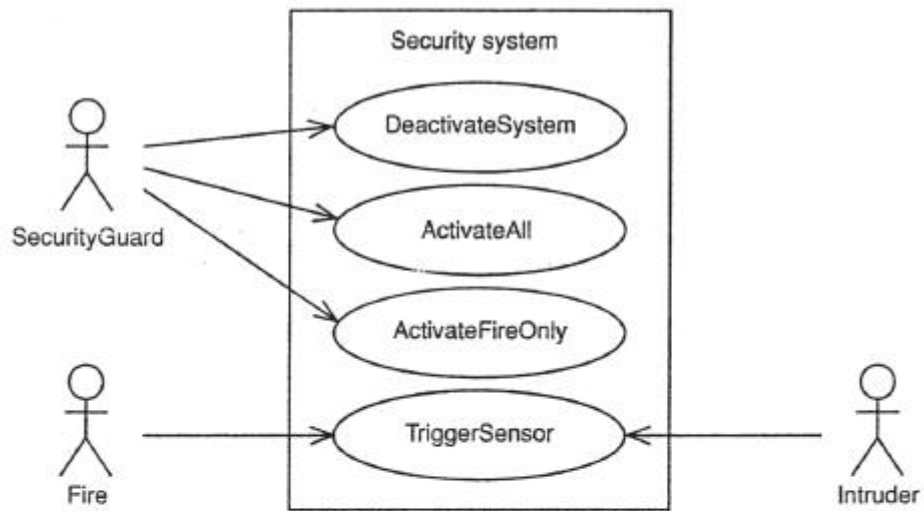
- ✓ In the corresponding design diagram, in the early stage of design, application layers are visible (e.g. front-end/GUI and backend/DB),

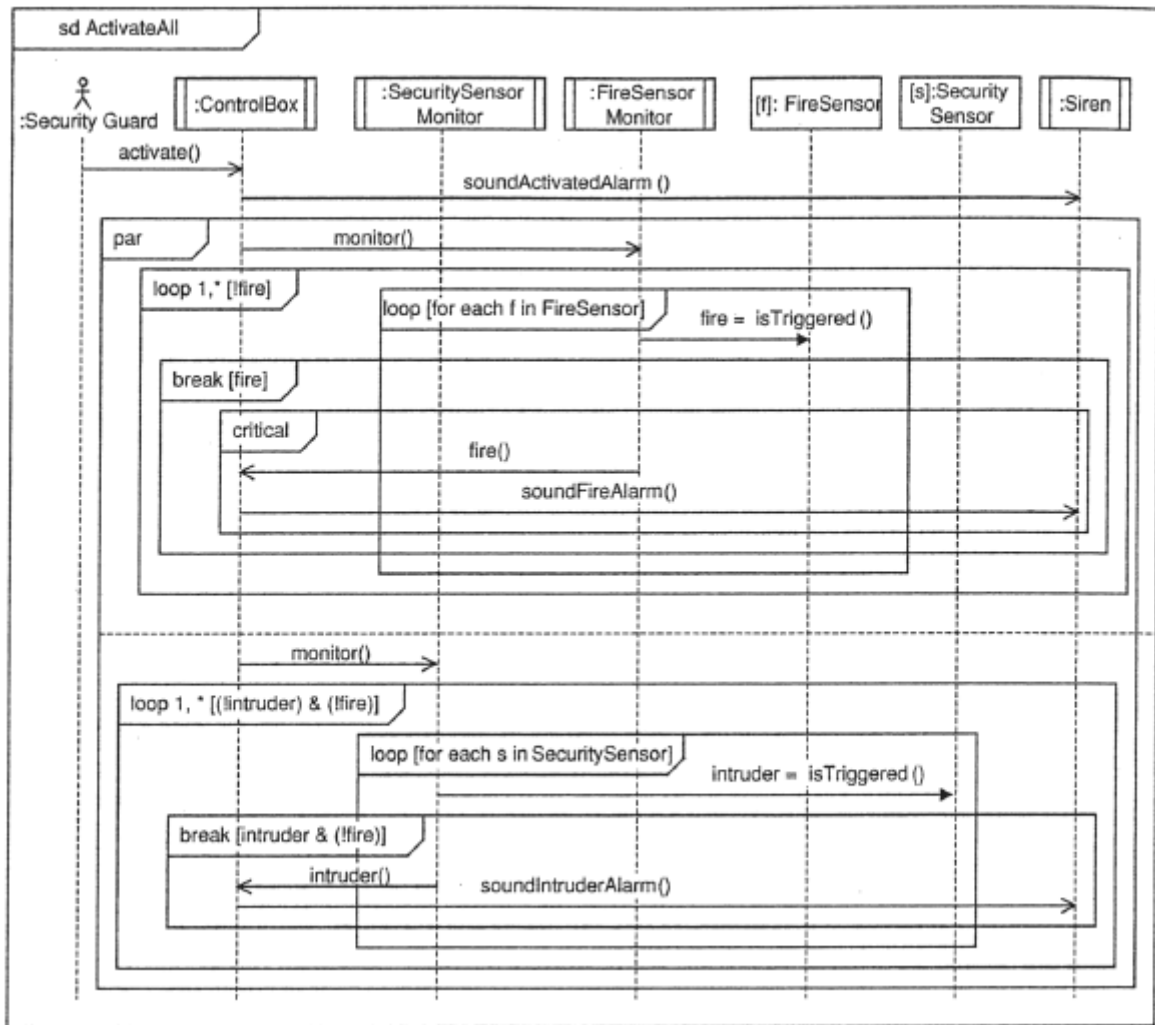
Ogni funzione non deve avere solo il nome, ma anche l'input.



Example of a security system realized with active class (its object encapsulates its own thread of control). It is made by four components: control box, siren, fire sensors, set of security sensors. There is a controller card for each type of sensor. The system is multithreaded.

Example of concurrency in sequence diagrams.

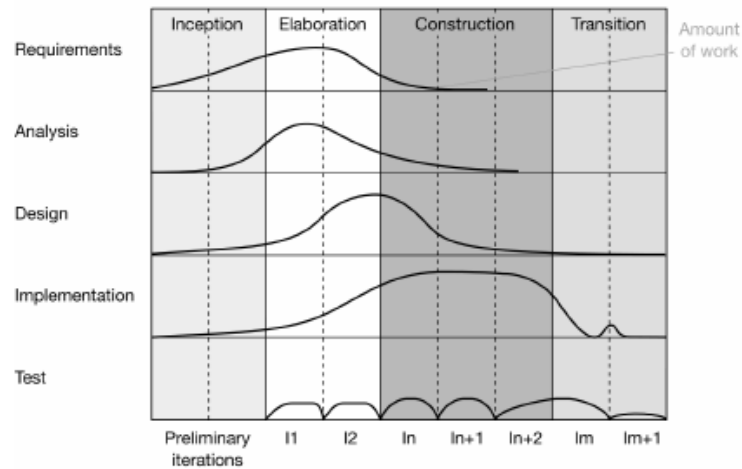




Per costruire il design bisogna fare attenzione alle dipendenze, in particolare quelle di tipo “use”.

Il numero di sequence diagram è lo stesso.

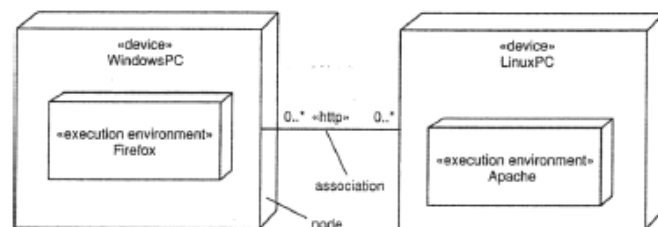
4. The Implementation workflow



- ✓ Implementation: to transform a design model into executable code
- ✓ It begins in the elaboration and is the main focus of the construction phase.
- ✓ Architectural implementation: to identify architecturally significant components and to map them to hw.

Adesso il software può essere sviluppato, abbiamo tutto il necessario per l'implementazione.

✓ Deployment diagram



Device: a physical type of device (PC, Server)

Execution environment: e.g. an Apache web server

- ✓ Types of artifacts: source files, executable files, scripts, database tables, documents, outputs of previous

Dove le componenti sono implementate e installate, questo è l'unico grafico UML di questa fase. Questo può cambiare nel tempo, non ne avrai bisogno per il progetto.

4. Testing Workflow

What is the purpose of the testing workflow?

- The purpose is not to show that the software is bug-free (it is unrealistic)
- The purpose is to find as many errors as possible
- For this purpose, the code should be tested by a different team (testing team) w.r.t. the team that produced it (implementation team)

Il testing è metà dello sviluppo, devi ritestare tutto i moduli che vengono usati da quello dove c'è l'errore ad esempio.

Levels and operations of testing

Testing is carried out at various levels for distinct operations:

- **Unit test**

a unit is the smallest block of software that it makes sense to test, (e.g. a single low-level function that is checked as a stand-alone entity)

- **Module test**

a module is a collection of interdependent units

- **Subsystem test**

a subsystem is a significant aggregate of modules often designed by different teams; there may be interface problems that need to be fixed

- **System or integration testing**

it is the complete product test

Ognuno è diverso, devi fare tutti questi test, ovviamente richiede molto tempo.

Operations of testing

- **α -test**

- the system is developed for a small group of customers
- the system is placed in its final environment and tested with the data on which it will normally have to operate

- **β -test**

- the system is distributed to a community of users
- the system is tested by several users, who use it and provide their observations and the errors found

- **Benchmark**

- the system is tested on standardized data in the public domain for comparison with other equivalent products on the market
- it can be requested by contract

Benchmark: 30% più veloce della versione precedente.

- **Stress testing**

- is checks how the system behaves when it is overloaded, bringing it to the limit
- the stress test allows you to cause an error and verify that the system fails in an acceptable way (fail-soft)

Cerchiamo lo scenario peggiore e vediamo come reagisce il software.

Testing and Debugging

- Testing and debugging are two different concepts
- **Testing** detects the **presence of errors**
- **Debugging** consists in the **localization of the error**, its **analysis** and its **correction**
- **Regression testing** is performed after debugging
- It verifies the behavior of the module that has been corrected w.r.t. the cooperating modules
- It makes sure the correction of the error has not introduced new errors

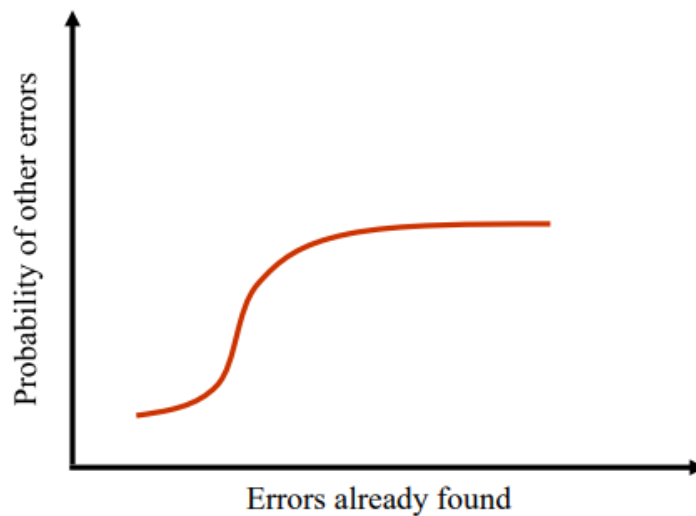
Regression: rifare alcune operazioni per vedere se il debugging ha funzionato. Ricordiamo che il debugging può avere introdotto nuovi errori.

La maggior parte degli errori sono nella control class. Per cui si consiglia di non avere classi con troppe funzionalità o troppe dipendenze.

Error distribution

- Errors tend to concentrate in certain modules, generally the more "complex" modules
- A module with many errors should be checked carefully: it will probably contain more errors
- According to some statistical studies, the probability that there are other "latent" errors increases with the number of errors already found according to a "logistic" curve

- Probability of existence of other errors than those found



Errors introduced by maintenance

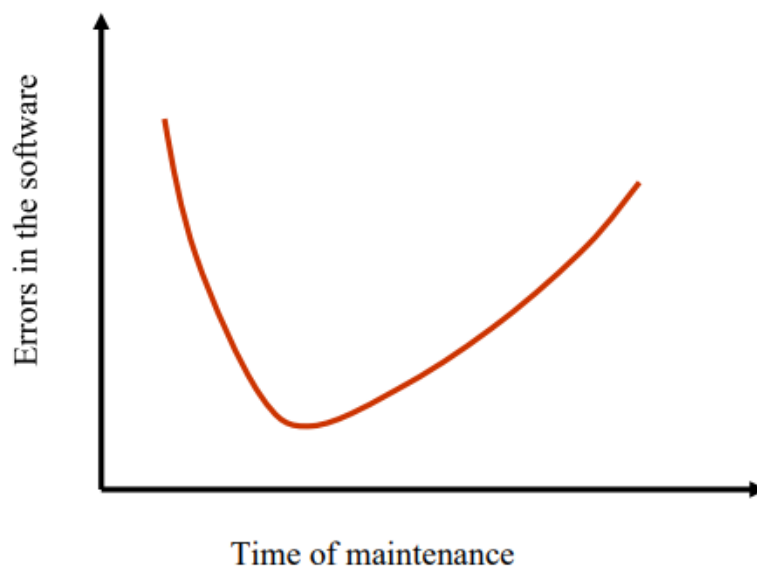
- In the **maintenance** activity, aimed at evolutive changes (new customer needs) and at correcting errors, **new errors** are introduced
- Many decisions initially made by programmers are not documented, they are not evident from the code; other decisions made later in the modification phase tend to be forgotten by authors and unknown to others
- To modify a part of code written by others, or simply "dated", with the aim of eliminating errors, easily leads to introduce further errors
- The typical curve of errors as a function of maintenance time often tends to follow a close-to-**parabolic** curve

Per fare il testing controllo la documentazione per vedere che sia tutto corretto e funzioni tutto come dovrebbe. Bisogna documentare ogni task.

- initially there is a reduction in the number of errors; as maintenance time passes, continuous interventions tend to introduce lots of new errors
- When a module is subject to continuous maintenance, a redesign should be scheduled to optimize maintainability and extensibility

Error distribution

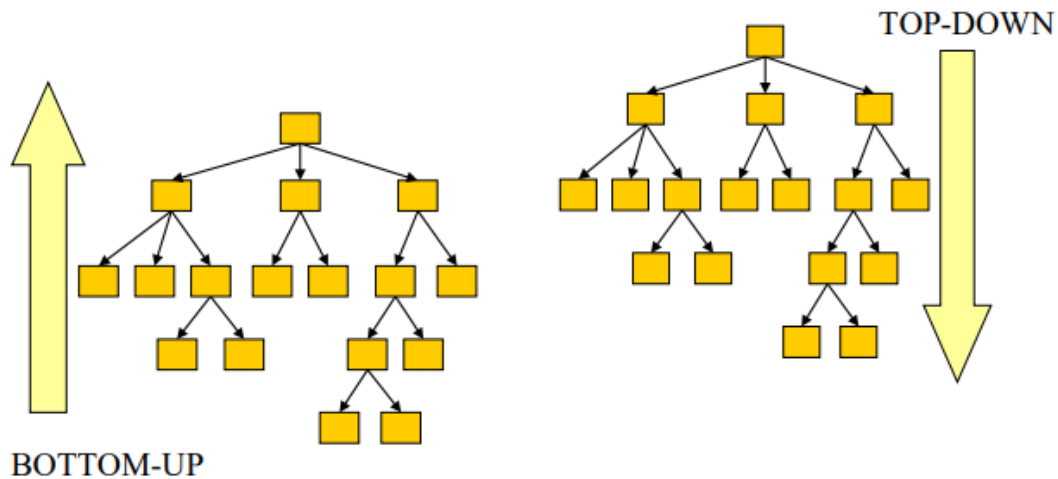
Errors introduced by maintenance



Bisogna cercare di mantenere la manutenzione meno necessaria possibile.

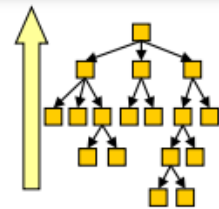
Testing strategy

- Like design, testing can also be carried out top-down or bottom-up



Parliamo di classi che hanno alte dipendenze, utilizzate da molte altre. Bottom-up prevede di partire dalle classe piccole fino a quelle più grosse e viceversa nella top-down.

Bottom-up testing

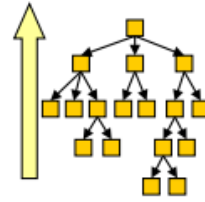


- First, the “lowest level” modules in the hierarchy produced by the design are tested; these are the smallest and most basic units of the program
- When the smallest components are correct, the next level of composition is tested, using the functionality of the previous level
- Progressively, the entire system is tested
- This approach has some issues:
 - **drivers** are needed, i.e. auxiliary software that simulates the call to a module by generating data to be passed as parameters
 - drivers replace the part of the code not yet integrated in the tested portion of the program: the top of the module hierarchy

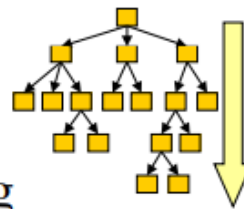
Smallest components = classi più piccole

Drivers = sono simulatori di alto livello, simulano le chiamate da parte di classi di alto livello.

Bottom-up testing: advantages and disadvantages



- The integration and testing process is intuitively **easier** to implement
- The later a bug is discovered, the more expensive it is to fix it, as it requires fixing it usually with a leaf module, and its partial re-testing, up to the modules where the bug was found

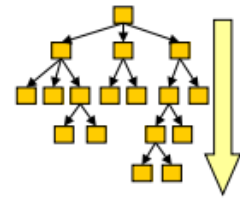


Top-down testing

- Initially, the module corresponding to the root is tested, without using the other modules of the system
- **Simulators**, called **stubs**, are used instead of the other modules of the system
 - these are elements that can return random results
 - request the data from the tester
 - can be a simplified version of the module
- After testing the root module of the hierarchy, its direct children are integrated
 - simulating their descendants by means of stubs
- This continues until all the leaves of the hierarchy are integrated.

I simulatori simulano le componenti a livello più basso. Ritorni risultati randomici per vedere se funziona tutto.

Top-down testing: advantages and disadvantages



- The costliest errors to fix are discovered first
- This is particularly useful if the design is also top-down and the design, implementation and test workflows partially overlap:
 - a design error at the top of the hierarchy can be discovered and corrected before the bottom is designed
- An incomplete but functioning system is available at all times
- Stubs can be **expensive** to create

What is the best testing strategy?

- A compromise solution is usually adopted between the two strategies, to mitigate the drawbacks that each solution presents
- It is not convenient to integrate as many modules as possible at each step
 - for example, in a bottom-up strategy, testing the children of a module and then integrating them all together with the parent to test: this leads to broken code, without a clear idea of where the error is
- It is convenient to **introduce the tested modules one at a time**
 - this way it is possible to locate errors better
- In general it is more convenient to carry out **frequent and short tests**

La maggior parte delle volte non c'è una risposta corretta.

Static checks

- The critical point of the test is represented by the **cost**
 - to reduce it, to improve the relationship between the cost and the power of the test, or to support the test with other verification techniques
- One form of static verification is **code inspection**
- It allows to identify between 60% and 90% of errors that would otherwise only be found during testing with often higher costs
- Inspection can also be performed by a specialized team, having the specification document, whose task is to report the errors to the programmers, who will then fix them
- The coding environment normally provides automatic tools that analyze the code indicating errors (for example, a call to a function with the wrong number of parameters)

Leggi il codice non lo esegui.

Testing

- Code inspection alone is obviously not enough
 - it does not intercept errors due to the execution of programs
- The modules have to be run on **data samples**, observing the resulting behavior
- Testing has two problems
 - It is **never exhaustive**, therefore it does not prove the correctness of the code
 - It is **expensive**, in terms of machine use and human time

Test-cases

- To carry out a test it is necessary to develop a set of **test-cases**, each including:
 - **input** data (test-data) for the module to be tested
 - **description of the function internal** to the module
 - **output** expected from the function
- A test-case represents a characteristic, a requirement, conforming to the specifications and it is used as a unit to construct a test
- It is completed by a set of metadata
 - status (approved, rejected, modified)
 - context
 - link to specification
- – ...

Andrebbe fatto per ogni funzione esistente, per ogni possibile input ecc.

How do you build a test case?

- To build a test-case it is necessary to know the **correct output** for a certain input of the function under examination
- **It should be taken from the specifications and documents produced by the design workflow**
- At the end of the input execution, the output is stored with the test-case and is compared with the expected output
- The test must be **repeatable**
 - for the test phases that occur later in the life of the software

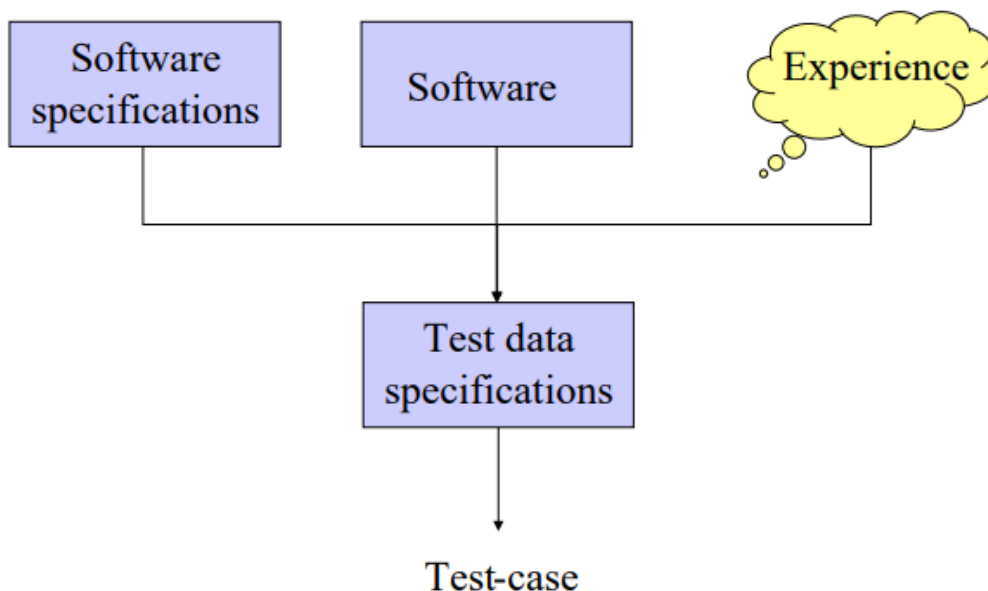
Vuoi fornire al client dei risultati di test che possono essere riutilizzato dal client per vedere se funziona tutto.

How to choose the input?

- The first idea is to generate the input **randomly**
- This technique has the disadvantage of not being very uniform for small samples
 - to have reliability you need to generate a lot of test-data
- Better input selection criteria are needed
- The test team has to decide the characteristics of the test data on the basis of:
 - – the **program**,
 - – the **specifications**,
 - – the **experience**

Bisogna fare attenzione che i valori random generati siano utilizzabili davvero, abbiano davvero senso. Le specification sono quelle viste in fase di Design (dove abbiamo definito il tipo e il numero di input).

Test-case generation -



Testing methodologies

- **Black-box testing**
 - The specifications of the input data are achieved only from the specifications of the program
 - Without considering the code
- It is normally used for the first tests
- It has the advantage of being accessible to the customer (who does not see the code)
- **Structural testing**
 - You derive the specifications of the test data by looking at the code
- It is generally better than the black-box testing

Il codice viene visto come una black box, bisogna capire come il sistema lavora senza guardare all'interno. Per esempio fornisci il sito web ad una certa quantità di persone che possono interagirvi.

Black box testing

- The choice of input data is based only on the specifications of the program
 - each function is matched with a more or less formal description of its correct inputs, the output that should be emitted and the function itself
 - for example, a function that returns the date accepts as input the day, the month in the respective ranges of 1:31, 1:12
- The idea of black-box testing is to **partition the set of admissible inputs into equivalence classes** so that within each class, each data has the same testing power
- Experience determines the selection criteria:
 - **at least one value inside each class**
 - **all boundary values**

L'input deve essere random ma in un certo range di valori per avere senso nel dominio della applicazione.

Black-box testing and equivalence classes

- Some criteria for determining the equivalence classes
- For a variable v defined in an interval $a \dots b$ there are three classes:
 $v < a$; $a \leq v \leq b$; $v \geq b$;
- For integer variables there is only one class
- – experience suggests to use also the value 0 and a negative value
- For alphanumeric strings, the determination of classes and boundary values depend on the restrictions on the format of the string
– for example: the empty string, the one containing only characters, the one containing numbers, etc.
- One problem with this method
- – the combination of test data for all input components leads to a **huge number of test cases**

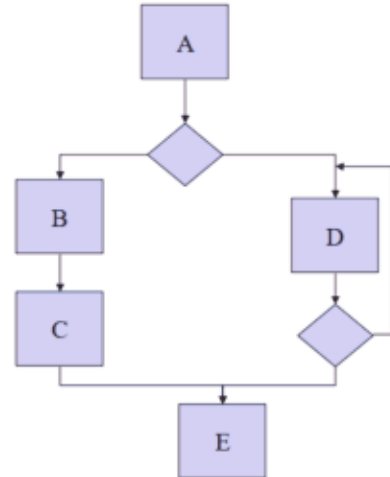
Structural testing

- The choice of input is based on the structure of the program
– the control flow of the program is determined and expressed in the form of a flow-chart
- The idea of Structural Testing is to use test data that leads **to all possible computations**
- Data combinations are searched for all path combinations in the flow-chart
- In the case of cycles, at least two test data are taken
– one who runs the loop
– and one that doesn't do it

Types of Tests

example of structural test

- It is conducted according to the structure of the system
 - the flow of control is determined in the form of a flow-chart
 - using test data leading to all possible combinations of paths in the flow-chart
- E.g. it can be used in the test of web sites
 - applied to pages
 - verifying all the paths through the links



In figura hai un flow chart, vuoi usarlo per un structural test

Automatic tools (Test Automation)

- Testing can be streamlined using **automated testing tools**
- These tools provide several supports, essential for the repeatability of the tests
 - Management of test cases
 - Test-data management
 - Storage of the tests carried out
 - Analytical reporting
- Automated testing cannot always fully replace manual testing
 - some checks are more effective if performed manually (verification of the graphic layout and usability of the product)

Consigliato per lo unit test. Per testare una funzione o classe di base, perché tu sai la struttura. Puoi creare tanti test case.

Test Automation cases

- **Functional tests** of a product
 - need to repeat the same test many times
 - different values of the input data
 - different configurations
 - **New product releases**
 - check that the changes made have not introduced regressions
 - Manual testing makes the process particularly expensive
 - cost and time constraints lead to non-execution of tests compromising the quality of the product
 - Automating testing drastically reduces its cost
 - increases the initial design cost, which is however returned by the time savings in the numerous automatic re-runs of the test cases
- repetitive verification operations easily lead to human errors; the automatic test performs the same operations several times with accuracy and precision

Simulators

- The most difficult software to test concerns exceptional or dangerous situations: in these cases, simulators of the environment in which the software will operate are used
- A simulator is an auxiliary program
(which will then not be used at the end of the development process)
that mimics the actions of another program or hardware or environment behavior:
 - it is used to test products whose malfunction can cause damage or (e.g. a program to operate a nuclear reactor)
 - also serves to test the system under particular load conditions (stress-testing), difficult to obtain if not in particular conditions of the final environment
- The construction of simulators obviously has a significant impact on the cost of the finished product

5. Esempio Modeling da linguaggio naturale

Modeling from natural language

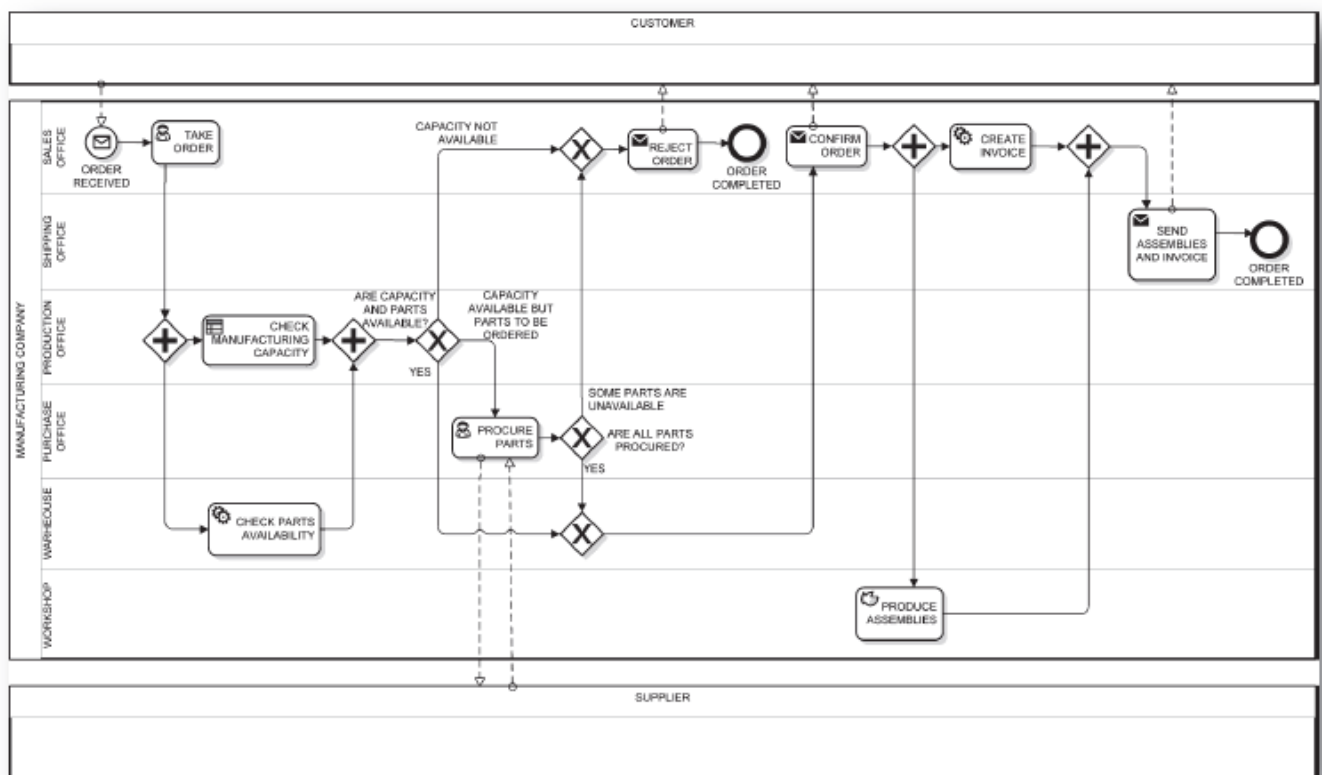
12 of 14

Example: create a handoff diagram

Manage orders in a manufacturing company

Order processing in a manufacturing company takes place after an order has been received from a customer. The order is first taken by the sales office. Both manufacturing capacity and parts availability are then checked, via appropriated business rules and an automated information service, by the production office and the warehouse, respectively. Subsequently, an order rejection is sent to the customer by the sales office if capacity is not available. Otherwise, if parts are also available, an order confirmation is automatically sent to the customer by the sales office. Therefore, both assemblies and invoice are produced, by human operatives at the workshop and by the sales office via an automated information service, respectively. Finally, assemblies and invoice are sent to the customer by the shipping office. In contrast, if there are parts to be ordered, the purchase office procures such parts, purchasing them from suppliers, so as to allow the aforementioned process to continue once such parts are available. However, if some parts are still unavailable (they cannot be procured), a rejection is then sent to the customer by the sales office.

The BPMN model of the manufacturing company at the handoff level



Dal testo si può notare che il Customer è un attore, perciò creiamo il pool del customer, ma è una black box perché non ne conosciamo i dettagli, quindi ci focalizziamo sulla compagnia.

L'altro pool è la *manufacturing company*, la quale prevede un altro box. All'inizio di questa abbiamo un trigger, che è il primo evento indicato dalla prima riga: "un ordine è stato ricevuto da un customer". Quindi abbiamo *order received* che è l'evento. Vediamo anche che c'è un *order processing*, la sequenza è: ricevere l'ordine e poi prendere l'ordine. Non ci sono altri elementi nella prima frase.

Nella seconda frase compare il sales office che è l'entità che prende l'ordine. In questo caso l'icona suggerisce che è un task svolto dall'umano grazie all'aiuto di IT.

NB: Task va indicato con il verbo al presente.

Terza frase: la parola "Both" indica multiple attività fatte in parallelo, ci sono due attività di verifica, ognuna è fatta in due modi diversi. Dopo il task di prendere l'ordine, abbiamo un *parallel gateway* utilizzato per dividere il flusso in due token, essendo fatte in parallelo. Legato ad esso è presente una business rule, infatti vediamo l'icona della scheda. L'altra icona è l'ingranaggio perché è automatizzato.

In generale si un database gli ordini. Devi verificare che tutte le componenti necessarie siano disponibili.

Check manufactory capacity: se hai risorse umane che assemblano l'ordine, hai bisogno di umani. Perché usiamo una business rule? Poiché specifichi: una persona a può prendere questo ordine, in un certo orario, per fare ciò assegni le risorse. Va verificato che hai abbastanza persone per processare l'ordine. Devi gestire le regole con sistemi di informazioni. Quando invece abbiamo un database con componenti, basta usare query differenti per vedere se sono disponibili o no. Ci sarà un albero di componenti, con una lista di query per capire se sono nel magazzino.

Abbiamo due lane: *production office* e il *warehouse*. Non hai un unico database ma hai differenti sistemi, ci saranno prodotti e ordini differenti. Le due attività menzionate prima sono fatte in parallelo sui due lane appena citati.

Se si segue il flusso bisogna rispondere ad una domanda, ma per rispondervi necessitiamo che entrambe le attività siano finite. Ora però focalizziamoci solo sulla capacità: è disponibile? Se non lo è, vai in un ramo in alto (specificato dalla quarta frase). Il "full envelope" significa invia un messaggio. Dopo il *reject* il flusso è finito, dato che non abbiamo persone che possono completare l'ordine.

Se ci sono sia capacity e che parti disponibili, possiamo seguire la freccia in giù, questa parte è spiegata dalla quinta riga.

Quando confermiamo l'ordine esso va creato l'ordine, nel rombo che precede la creazione abbiamo un "+" perché c'è "both" nella frase relativa.

Invoice essendo calcolata differentemente, porta ad avere un altro sistema di informazioni, un altro database.

Bisogna aggiungere un nuovo lane per gestire i “produce assemblies”. Poi arriveremo al “synchronization point”, un “+” nuovamente.

Dopo aver inviato gli invoice il token viene distrutto, altrimenti avresti un deadlock, il processo non verrebbe completato.

Poi c'è una parte che indica se la capacità è disponibile ma non le parti che devono essere ordinate. Usiamo solo le condizioni specificate dalla azienda, la logica va espressa come dichiarata dalla compagnia.

Supplier è un altro attore, una entità esterna.

Customer relationship management: quando abbiamo degli appuntamenti, riceviamo alcuni email ad esempio, esse sono fatte dal CRM, che è un sistema di informazione specializzato. Abbiamo anche un SRM (supplier relationship management).

Abbiamo diversi software per sviluppare il nostro, fatti da altre aziende.

Procure parts: quando abbiamo bisogno di questo serve un umano supportato, che ordina le componenti mancanti.

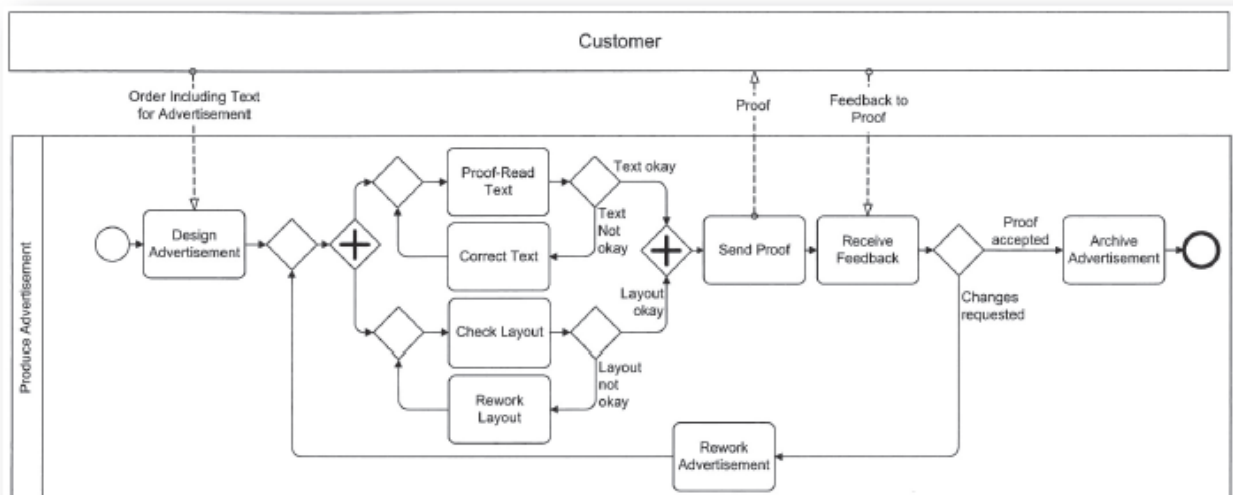
Gli ordini possono essere rifiutati per una delle due condizioni, infatti abbiamo la X.

BP Modeling: semi-formal specification and scenarios

8 of 17

Homework: advertising agency

- ❑ Describe in semi-formal natural language the following process model, describing how an advertising company produces an advertisement, and which messages it exchanges with the customer.
- ❑ Given 100 starting tokens, determine the number of ending tokens for each scenario (path), considering the following branching proportions at each gateway: text not okay (10%), Layout not okay (20%), changes requested (1%).
- ❑ At each scenario, assume "happy" cases maximizing completd orders.



1. The Company is ready to receive orders from customers
2. A Customer sends an order to the Company including the text for advertisement
3. The Company receives the order from the Customer
4. The Company designs the Advertisement
- 5.a.1. The Company proofreads the text
- 5.a.2.1. If the text is okay
- 5.a.2.1.1. → 6
- 5.a.2.2. If the text is not okay
- 5.a.2.2.1. The Company corrects the text
- 5.a.2.2.2. → 5.a
- 5.b.1. The Company checks the layout
- 5.b.2.1. If the layout is okay
- 5.b.2.1.1. → 6
- 5.b.2.2. If the layout is not okay
- 5.b.2.2.1. The Company reworks the layout
- 5.b.2.2.2. → 5.b
6. The Company waits until both text and layout are okay
7. The Company sends a proof to the Customer
8. The Customer receives the proof from the Company
9. The Customer sends to the Company a feedback on the proof
10. The Company receives the feedback on the proof from the Customer
- 11.1. If the Customer requests changes
- 11.1.1. The Company reworks the advertisement
- 11.1.2. → 5
- 11.2. If the Customer accepts the proof
- 11.2.1. The Company archives the advertisement
- 11.2.2. End

Dopo design continui, c'è un merge node, poi dividi in parallelo. (bianco indica a X).

5.a 5.b perché è in parallelo. Poi abbiamo 5.a.1

Vediamo lo splitting multiplo dopo proof read. Internamente c'è un altro livello di numbering.

Internamente c'è anche un loop, dove la compagnia corregge il testo, e magari ci possono essere più loop.

Quando li hai entrambi puoi continuare dopo il + (sia layout che testo), c'è una sincronizzazione in quel nodo. Poi viene inviata una prova al customer.

A data factory (DataOps) for statistical classification

Mario G.C.A. Cimino, Dept. Information Engineering, University of Pisa

A classifier receives an input session, classify that session, and delivers a class label. The classifier lifecycle consists of three phases. Development phase: the classifier is under construction; human-provided labels and related sessions are collected. Production phase: the classifier is developed and provides classes (labels) for a given session. Production & evaluation phase: during production, the classifier is periodically evaluated. An evaluation of the classifier performance involves a comparison of corresponding human-provided classifier-provided labels. E.g., every 5000 production sessions, 50 sessions of evaluation.

A session is a set of prefixed of records. Each record is provided by a different client-side system (e.g., IoT sensor, wearable device, annotator). A record is made by samples. It follows some activities carried out in the factory. Prepared sessions are collected and checked to generate a learning set. After receiving an order, client-side and server-side computer systems are configured, to be operative. Records are collected and joined in a raw session, and then transformed into a prepared session (feature vector). The quality of the classifier is evaluated over a set of labels. Learning sets are separately used to train, validate, and test the classifier. Validation is performed via many training instances

La data factory è basata su *dataops*, gestisci i dati e fornisci un servizio. L'obiettivo è uno *statistical classifier*: vengono effettuate classificazioni per esempio per capire se c'è un rischio o no ecc.

Aggiungi del valore con il classifier, risparmi soldi. Esso prevede la *session* in l'input e il *label* è l'output.

Hai due tipi di parametri: parametri e hyperparametri. Nella fase di sviluppo devi avere molti input per aggiustare il classifier, quindi tante session e tanti label (un insieme di input-output). Si parte dal calibration set. Si hanno due classi di parametri. Gli Optimization parameters sono chiamati *calibration* perché servono per la calibrazione, nel validation invece cerchi un set ottimo di hyperparameters.

Metti immagine

Dobbiamo fare un *landscape*, partiamo dal generale. Abbiamo una descrizione di una factory. Si parte dal testo, si capisce, si fa passo passo tutto il lavoro. Si analizza frase per frase, si crea prima la sequenza nominale come nella compagnia di telefonia.

Questions

Suppose that the tasks of proof-read, check, and feedback must provide the same result when applied to the same content (text and/or layout):

1. The Customer can request changes to the layout, and subsequently to the text.
☐ true ☐ false
 2. Even if the Company reworks the advertisement by applying exactly all changes requested by the Customer, the Customer can subsequently request new changes.
☐ true ☐ false
1. Falso perché se dici che il test è buono una volta non puoi più cambiare perché il testo è uguale, la consegna dice che è deterministico.

Answers

1. **False:** because the task *rework advertisement* will change the layout only, and then the tasks *proof-read text* and *feedback to proof* will return again *text ok* (as they must provide the same result when applied to the same text).
2. **True:** because the activities of proof-read, check, and feedback can provide a different result when applied to a new content.

6. Software factory

A data factory (DataOps) for statistical classification

Mario G.C.A. Cimino, Dept. Information Engineering, University of Pisa

A classifier receives an input session, classify that session, and delivers a class label. The classifier lifecycle consists of three phases. Development phase: the classifier is under construction; human-provided labels and related sessions are collected. Production phase: the classifier is developed and provides classes (labels) for a given session. Production & evaluation phase: during production, the classifier is periodically evaluated. An evaluation of the classifier performance involves a comparison of corresponding human-provided classifier-provided labels. E.g., every 5000 production sessions, 50 sessions of evaluation.

A session is a set of prefixed of records. Each record is provided by a different client-side system (e.g., IoT sensor, wearable device, annotator). A record is made by samples. It follows some activities carried out in the factory. Prepared sessions are collected and checked to generate a learning set. After receiving an order, client-side and server-side computer systems are configured, to be operative. Records are collected and joined in a raw session, and then transformed into a prepared session (feature vector). The quality of the classifier is evaluated over a set of labels. Learning sets are separately used to train, validate, and test the classifier. Validation is performed via many training instances

Prima riga: Il software, per esempio, darà una classe per ogni input (brain signal) e l'output è una direzione.

Terza riga: il Development è quando generi il classificatore. Ad esempio, pensi e invierai i segnali del cervello con un label associato.

Quarta riga: Il classificatore è nella fase di produzione, dopo il Development c'è la production phase.

Sesta riga: Gli umani guardano i dati e associano i label, si utilizza l'annotator per fare ciò. Bisogna collezionare dati in un time frame, non istantaneamente. Hai differenti dati in aggiunta a quelli, provenienti da diverse applicazioni. I dati sono collezionati in un certo tempo, per cui le *data annotation* devono essere schedate. Hai un cloud dove collezioni i dati e sono poi gestiti lì. Ci sarà un esperto che annota i dati. Nella production, però, non usi l'esperto, risparmi soldi. "Valutato" significa vedere se è ancora buono, per esempio, se un paziente ha una malattia degenerativa, vedi se il segnale al cervello è cambiato e, nel caso, devi allenare di nuovo il classificatore. Ogni tot sessioni chiedi all'esperto di mettere i label e vedi se il classificatore è accurato o meno.

Nona riga: un sample, ad esempio, può essere costituito da uno o più segnali del cervello, bisogna modellare i sample perché si "perdono" a causa del rumore, i dispositivi wireless sono spesso soggetti a perdita di dati. Bisogna controllare la qualità dei dati che arrivano. Per esempio, ci sono tecniche per eliminare il rumore nei dati.

Le attività non sono ordinate, poi si ordinano successivamente. Bisogna capire il significato delle attività per ordinarle.

Undicesima riga: quando collezioniamo le sessioni, alla fine otteniamo un learning set.

Dodicesima riga: hai un nuovo customer, come un nuovo paziente per la sedia a rotelle, devi connetterlo al cloud environment. Configurazione: ad esempio, devi connettere la sedia a rotelle al cloud, devi assegnare un IP. Devi configurare il sistema per collezionare correttamente i dati. Ricevi ordini per il nuovo classificatore. Si completa un *configuration service* per un ordine ricevuto.

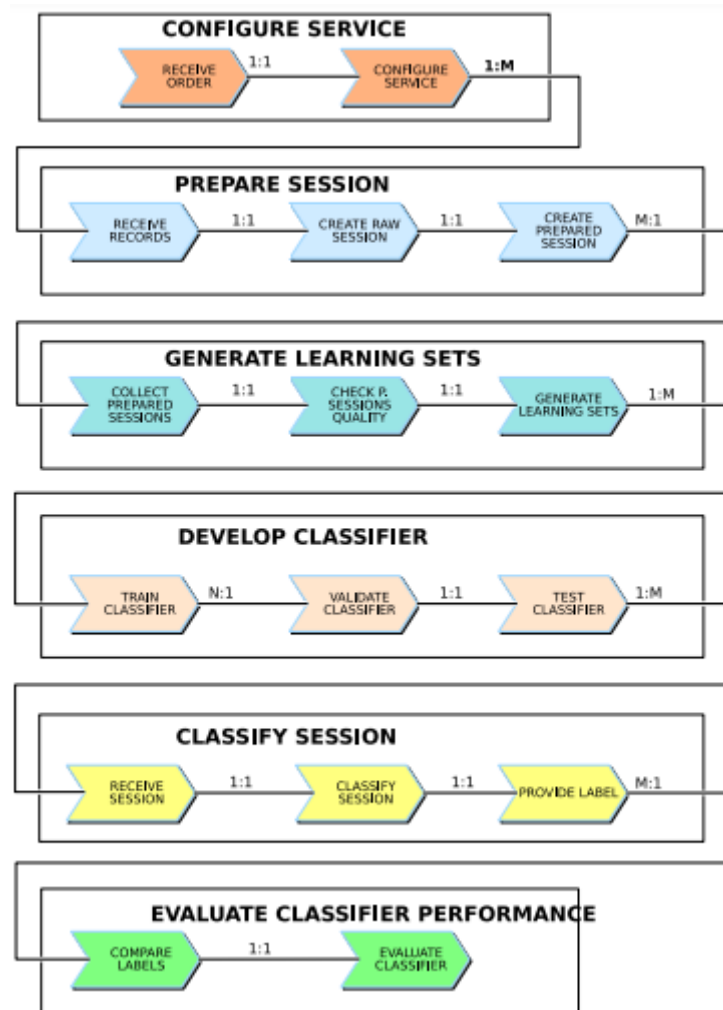
Tredicesima riga: un comando nella sedia a rotelle permette di collezionare record, per poi metterli nella raw session. Da questa si ottiene una *prepared session*: puoi estrarre feature statistiche per compattare le informazioni e ottenere valori significativi per il tuo classificatore.

- Raw = non processati, solo collezionati e messi insieme.
- Compari label generati da umani e dalla macchina.
- Lo sviluppo è fatto da train, evaluate e test.
- Il prossimo step è ordinare le attività.
- Il diagramma creato è il life cycle.

Bisogna spiegare come hai creato la tua factory.

Ricorda che il diagramma che vogliamo creare rappresenta l'happy case, non il workflow.

Per creare learning set devi avere più sessioni, devi fare la collect.



La Receive è fatta una sola volta, 1 esecuzione di *receive record* corrisponde a 1 create raw session.

Per ogni cambiamento di cardinalità cambi il colore. Cambiamento di cardinalità → cambiamento di token. Assegna un nome quando cambia cardinalità.

L'attività è fatta da molteplici task, il task è una operazione atomica.

Milestone: è un importante traguardo, ad esempio, quando la data collection è finita, completi uno step importante, invece, ricevere un record non è importante, devi riceverne tanti per avere un set di dati significativo. Quando hai cambiamento di cardinalità, se l'attività non è milestone, puoi continuare con lo stesso colore/processo. Raggruppiamo le attività con le stesse cardinalità. Cambiamento di cardinalità ma nello stesso loop, se è un technical change, se non vai in un'altra fase o in un altro traguardo metti nello stesso group. Tutto nello stesso sotto processo.

Come sviluppare una NN? Una NN non si sa se è grande o piccola, dipende dalla complessità dei dati, devi stabilire il numero di layer e di neuroni, fai molto training ma con gli stessi dati, per gli stessi dati si fanno più esecuzioni. Nella validation trovi la migliore.

Train → sviluppi il tuo classificatore

INFORMATION SYSTEMS INVOLVED:

- Messaging System (used for service communications)
- Ingestion System (collect records from client-side systems, analyze records/samples flow, not the s
- Preparation System (preprocess samples, correct samples values, extract features)
- Segregation System (collect prepared sessions and create learning sets)
- Development System (train, validate, test the classifier)
- Production System (the classifier)
- Evaluation System (periodically evaluates the performances of the classifier)
- Client-side systems (provide various input records)

BPMN MODELING

- for each process, create the handoff level first, and then modify it as a service level workflow
- by default, the task type is script
- an asterisk denotes a parameter to configure

Service communication: interazioni fra membri della *factory* per parlare della fase in corso. Bisogna comunicare su Microsoft Teams per annunciare di aver fatto qualcosa, che qualcosa è pronto. Si possono automatizzare i *service messages*. Il cambiamento di fase di solito è comunicato con i *service messages*.

Ingestion system → sviluppato con Python, colleziona dati da differenti applicazioni. È capace di processare il flusso di dati e i valori dei messaggi.

Preparation system → è per il preprocessing: se hai un *outlier*, cioè un valore non possibile, allora significa che c'è del rumore, dunque va cambiato, preprocessando i valori.

Ingestion e Preparation lavorano sessione per sessione.

Segregation → system colleziona la sessione.

Development system → è usato per generare il classifier.

Avremo use case differenti, dopo aver generato il BPMN genereremo differenti interfacce per i vari utenti.

Il Production è piccolo, viene fatto insieme a l'*Evaluation*, a parte questo viene assegnato 1 sistema per persona.

All'inizio si hanno molte classi, anche in condivisione: gli altri usano magari le classi che te hai già fatto, bisogna scambiare le componenti fra i membri.

Handoff version: solo le attività in generale, poi si sviluppano quelle specifiche

```

Process CONFIGURE SYSTEMS ("human-glue" version)
1 configuration received (Messaging system, human)
2.a.1 configure (Ingestion System, human) → 3
2.b.1 configure (Preparation System, human) → 3
2.c.1 configure (Segregation System, human) → 3
2.d.1 configure (Development System, human) → 3
2.e.1 configure (Production System, human) → 3
2.f.1 configure (Evaluation System, human) → 3
2.g.1 configure (client-side systems)
3 wait for the end of 2.a, 2.b, ..., 2.g (Messaging system, human)
3 configuration completed (Messaging system, human)

```

Human → non è script

"*" significa che non hai un parametro configurato, il parametro deve essere configurato prima di inizio del processo, useremo i *json* per configurare i parametri.

```

Process PREPARE SESSION
(Ingestion System)
1 record received
2 store record
3 if insufficient record (*) then END
4 create raw session
5 remove records
6 mark missing samples
7 if raw session invalid (*) then END
8 if evaluation phase then send label to evaluation system
9 raw session sent (*)

```

Per ogni sistema abbiamo il suo nome, il *prepare session* è fatto dall'Ingestion system.

- Present tense → activity
- Past tense → ?
- Ogni client side system manda un messaggio
- Collezioniamo i record e poi continuiamo
- Data flow è internal
- Message flow è quello tra i sistemi
- If → gateway

(mettere grafico)

Li rimuovo perché il buffer è poco pesante, così quando facciamo i test stressando il sistema sarà comunque responsivo.

Se abbiamo dei dati mancanti vanno segnati, il sistema *sis* segnerà i problemi relativi al flusso, in modo tale che possiamo chiedere un'altra generazione ad esempio.

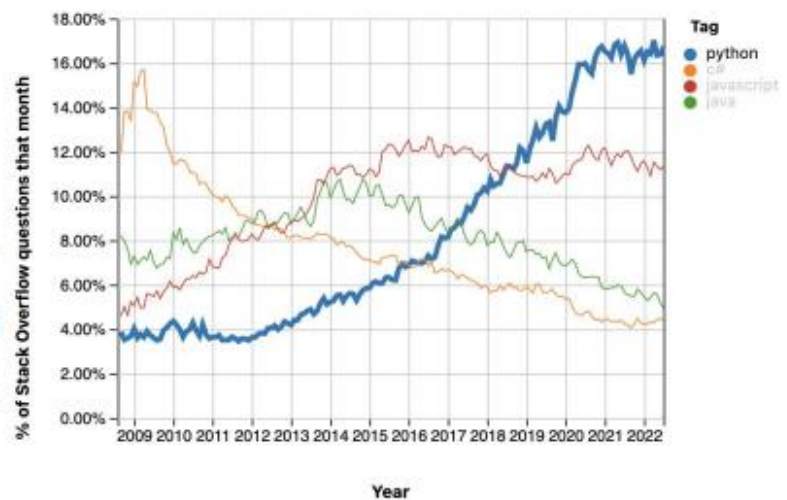
Il numero di record non è statico, in base alla fase puoi avere più o meno record.

7. Python (Alfeo)

WHY PYTHON?

Created by **Guido van Rossum** in the 1980s, to be a general-purpose language with a simple and readable syntax. Today is more and more required since it is:

- High level
- Open source
- Portable: write once run everywhere
- Extensible in C/C++
- Easy to learn
- With a mature and supportive community
- With hundreds of thousands of libraries, packages and frameworks, supported by big players (Google, Facebook, Amazon) and non-profit organizations



VIRTUAL ENVIRONMENT

- In each project, a number of Python packages are imported and used. Each of them may require a given version of other packages and Python as well.
- A **virtual environment** is a self-contained directory tree that contains a Python installation for a particular version of Python, plus a number of additional packages
- In this way, the project-wide dependencies are stored, easily snapshotted and retrieved
- Create a Virtual Environment with Python X.Y in folder env

```
virtualenv --python=pythonX.Y env
```

- Activate the Virtual Environment

```
source ./env/bin/activate
```

```
./env/bin/activate
```

- Deactivate the Virtual Environment

```
deactivate
```

BASIC PACKAGES MANAGING

- Install package

```
pip install package
```

- Uninstall package

```
pip uninstall package
```

- Snapshot installed packages in *requirements.txt*

```
pip freeze > requirements.txt
```

- Install all packages snapshotted in *requirements.txt*

```
pip install -r requirements.txt
```

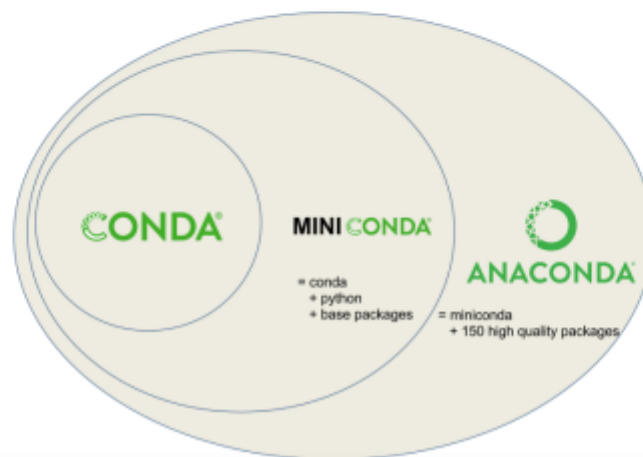
- Now you can import and use these packages in your project

ANACONDA

Anaconda is a distribution of Python that aims to simplify package management and deployment, suitable for Windows, Linux, and macOS.

Package versions in Anaconda are managed by **conda**, an open source, cross-platform, language-agnostic package manager and environment management system

Conda analyses the current environment including everything currently installed, works out how to install/run/update a compatible set of dependencies



6

Anaconda è consigliato, è più automatica a livello di dipendenze ecc. Automatizzeremo tante procedure e tante verifiche.

PYTHON IDE + MINICONDA = PYCHARM

Pycharm offers configurable python interpreter and virtual environment support.



Based on previous lecture by A. L. Alfeo

INDENTATION

In python **code blocks are not surrounded by curly brackets**. Just use the correct **indentation**!

```
for(int i = 0; i < n; i++){  
    int k = i % 3  
    if(k == 0){  
        // stuff...  
    }  
}
```



```
for i in range(0, n):  
    k = i % 3  
    if k == 0:  
        # stuff...
```

BUILT-IN TYPES

Python is **dynamically typed**, i.e. types are determined at runtime.

Variables can **freely change type during the execution**.

In python there are a lot of built-in types, the most notables are:

- **Boolean** (*bool*)
- **Strings** (*str*)
- **Numbers** (*int, float*)
- **Sequences** (*list, tuple*)
- **Mapping** (*dict*)

VARIABLES

Variables can be assigned to given values...

```
pi = 3
```

```
name = "Pippo"
```

...Even multiple variables at once via **iterable unpacking**. [Click for more on iterable data structures \(e.g. lists, tuples...\)](#)!

```
pi, name = 3, "Pippo"
```

```
first, second, third = SomeSequence
```

In Python everything is stored and passed as **reference** with the only exception of Numbers.

```
a = [1, 2, 3]
```

```
b = a
```

```
b[0] = 5 # now a = [5, 2, 3]
```

CONDITIONAL INSTRUCTION 1/2

Simple conditional instruction with the **if** keyword.

```
if someConditions:
    someActions()
    someOtherActions()
```

Python uses **and** and **or** as logical operators instead of && and ||

```
if (C1 and C2) or C3:
    someActions()
    someOtherActions()
```

Inline conditional instructions can be used as it follows

```
value if Condition else otherValue
```

CONDITIONAL INSTRUCTION 2/2

The **if-else** statement is used as it follows

```
if Condition:
    someActions()
else:
    someOtherActions()
```

There is no switch case statement in Python. Just use **if** and **elif**

```
if C1:
    A1()
elif C2:
    A2()
elif C3:
    A3()
else:
    A()
```

LOOPS 1/2

There is no do-while loop, just **while** and **for** loops

```
while Conditions:                for element in elements:
    Stuff()                       doStuff(element)
    otherStuff()
```

Tuple unpacking can be used with loops iterations

```
for x, y in SequenceOfTuples:
    doStuff(x, y)
```

LOOPS 2/2

zip() combines one-by-one the elements of two or more iterables

```
L1 = [1, 2, 3]
L2 = [4, 5, 6]
for x, y in zip(L1, L2):
    print(x, y)
```

enumerate() returns a list of *(index, element)* tuples

```
names = ["Federico", "Mario", "Giovanni"]
for i, name in enumerate(names):
    print(i, name)
```

For efficient loops an iterable object can leverage the [itertools](#) packages.

The simplest iterable can be a **list** created via **list comprehension**

```
[someOperation(element) for element in elements]
squares = [i**2 for i in range(0, N)]
```

FUNCTIONS: DEFINITION

A new function can be defined by using the keyword **def**

```
def getCircleArea(r):  
    return pi*r**2
```

Default argument are indicated with **=**

```
def getCircleArea(r, isEngineer=True):  
    pi = 3 if isEngineer else 3.1415  
    return pi*r**2
```

Inline functions can be created by using the **lambda** keyword

```
lambda comma, separated, arguments : expression
```

For example

```
norm2D = lambda x, y: math.sqrt(x**2 + y**2)
```

FUNCTIONS: POSITIONAL ARGUMENTS

A **variable** number of arguments can be defined via the ***** symbol

```
def sumOfSquares(*args):  
    squares = [arg**2 for arg in args]  
    return sum(squares)  
  
result = sumOfSquares(1, 2, 3)
```

A **sequence** can be passed as positional arguments as it follows

```
def norm2D(x, y):  
    return math.sqrt(x**2 + y**2)  
  
vec = [2, 3]  
norm = norm2D(*vec)
```

FUNCTIONS: KEYWORD ARGUMENTS

A function using keyword arguments needs to have the **** symbol as last argument**.

```
def greet(language = "en", **kwargs):
    if language == "it":
        print("Ciao "+kwargs["name"]+" "+kwargs["surname"])
    else:
        print("Hello "+kwargs["name"]+" "+kwargs["surname"])

greet("it", surname="Galatolo", name="Federico")
greet(name="Mario", surname="Cimino")
```

You can also pass a **dict** of keyword arguments using the symbol ****** while calling the method

```
person = dict(name="Federico", surname="Galatolo")
greet("it", **person)
greet(**person)
```

CLASSES: METHODS

In python classes are defined with the **class** keyword. Class methods are defined with the **def** keyword. **Class method** have the first argument equal to **self**, in contrast with **static method**.

```
class Person:
    def getName(self):
        return "Federico"
    def greet(self):
        return "Hi! I am "+self.getName()

class Person:
    greeting = "Hi!"
    def getGreeting():
        return Person.greeting

g = Person.getGreeting()
```

However, unless you want to use a **decorator**, Python does not know about static/non-static methods, **it is all about notation!**

```
p = Person()
p.greet() # ok
Person.greet(p) # still ok
```

CLASSES: ATTRIBUTES

In python you can create, modify and retrieve **instance attributes** using the dot (.) selector on the instance reference. You can create and assign an instance attribute everywhere in a class method.

```
class Person:
    def setName(self, name):
        self.name = name
    def greet(self):
        return "Hi! I am "+self.name
```

You can create **class attributes** specifying them after the class declaration. You can modify and retrieve class attributes using the dot (.) selector on the class reference

```
class Person:
    greeting = "Hi!"
    def setName(self, name):
        self.name = name
    def greet(self):
        return Person.greeting+" I am "+self.name
```

CLASSES: VISIBILITY

In python there is no such thing as a private method or attribute. **Everything is public.** The naming convention for “private” methods and attributes is to precede their name with the `_` symbol.

```
class Person:
    def setName(self, name):
        self._name = name
    def greet(self):
        return "Hi! I am "+self._name
```

CLASSES: CONSTRUCTOR

In python the construct function is named `__init__` and it is called at object instantiation.

You can specify one or more arguments. The first argument is the object instance reference.

```
class Person:
    def __init__(self, name):
        self._name = name
    def greet(self):
        return "Hi! I am "+self._name
p = Person("Federico")
```

CLASSES: INHERITANCE

You can **extend** a base class with another specifying the base class between the parenthesis at class definition. In order to get the base class reference you need to use the `super()` function.

```
class Person:
    def __init__(self, name):
        self._name = name
    def greet(self):
        return "Hi! I am "+self._name
```



```
class Student(Person):
    def greet(self):
        return "Leave me alone, I have to study"
```

MORE ON CLASSES: DATA MODEL

There are a lot of built-in functions provided by the base class of all the classes object. Each of which provide a specific behavior, a few are:

- `__len__ (self)`
Returns the "length" of the object (called by `len()`)
- `__str__ (self)`
Returns the object as a string (called by `str()`)
- `__lt__ (self, other), __lt__ (self, other), __eq__ (self, other)`
Called when the object is used in a comparison
- `__getitem__ (self, key), __setitem__ (self, key, value)`
Called in square brackets access

7.1. Code quality

CODE QUALITY: PEP8 STANDARD

The primary focus of **PEP8** (Python Enhancement Proposal) standard is to improve the readability and consistency of Python code. Examples:

Use 4 spaces per indentation level. Use spaces, not tabs.

Limit all lines to a maximum of 79 characters.

Imports should be as specific as possible

Surround top-level functions and classes with two blank lines.

Surround method definitions inside classes with a single blank line.

Use blank lines sparingly inside functions to show clear steps.

Surround the following binary operators with a single space on either side:

- Assignment operators (`=`, `+=`, `-=`, and so forth)
- Comparisons (`==`, `!=`, `>`, `<`, `>=`, `<=`) and (`is`, `is not`, `in`, `not in`)
- Booleans (`and`, `not`, `or`)

Queste sono regole per aderire ad uno standard internazionale.

PEP8 - NAMING

Function: Use a lowercase word or words. Separate words by underscores to improve readability. I.e. `function`, `my_function`

Variable: Use a lowercase single letter, word, or words. Separate words with underscores to improve readability. I.e. `x`, `var`, `my_variable`

Class: Start each word with a capital letter. Do not separate words with underscores. This style is called camel case. I.e. `Model`, `MyClass`

Method: Use a lowercase word or words. Separate words with underscores to improve readability. I.e. `class_method`, `method`

Constant: Use an uppercase single letter, word, or words. Separate words with underscores to improve readability. I.e. `CONSTANT`, `MY_CONSTANT`, `MY_LONG_CONSTANT`

Module: Use a short, lowercase word or words. Separate words with underscores to improve readability. I.e. `module.py`, `my_module.py`

Package: Use a short, lowercase word or words. Do not separate words with underscores

Pycharm ti avvisa con notifiche in giallo se hai errori in PEP8.

AUTOMATIC QUALITY CHECKING

Pylint is a source-code, bug and quality checker for the Python programming language. It checks:

if the code is compliant with Python's **PEP8** standard

if each module is properly imported and used

if declared interfaces are truly implemented

if there is duplicated code

Pylint plugin is fully customizable: modify your **pylintrc** to customize which errors or conventions are important to you.

7.2. Pacchetti Python

WORKSPACE FOLDER ORGANIZATION

The project folder and subfolder organization is up to you, but needs to be simple and clear, and allow to understand the processing put in place by each module.

Considera di usare differenti cartelle per ogni cosa.

Panda permette di gestire i dati tabulari

DATA ANALYSIS IN PYTHON

So many packages available:

- [Pylint](#): to produce readable code
- [Pandas](#): read data from a broad range of sources like CSV, and Excel files.
- [Json](#) and [Joblib](#): to save/read configurations and models to/from [files](#).
- [Sci-kit learn](#): machine learning library with a broad range of classification algorithms. It provides also data pre-processing, [pipelining](#) and performance analysis modules.
- [Matplotlib](#): graphic visualization libraries.

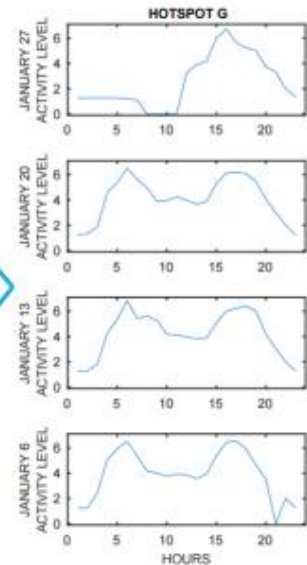
AN EXAMPLE...

Let's start with a very simple example ([click to download the data](#)).



HOTSPOT SET

By looking at mobility dynamics in the hotspot it's easy to distinguish between weekdays and weekend, or even to detect traffic anomalies (i.e. an parade, a storm).



Hotspot è una area di una città che è molto trafficata. Guardando il traffico nell'area puoi capire il traffico in tutta la città. Nei dati sono presenti dei dati del traffico in un certo periodo passato in una specifica ora, per inferire poi quanto traffico ci potrebbe essere in futuro. Magari potremmo capire che giorno è solo guardando ai dati attuali.

Abbiamo vari picchi di traffico, quando le persone vanno a lavoro, questi sono pattern regolari durante la settimana, ovviamente ciò cambia nel weekend, il pattern viene shiftato a destra verso le ore notturne.

Abbiamo giorni feriali, giorni festivi (venerdì e sabato) e domenica (non c'è vita notturna, poiché il giorno dopo si lavora). Voglio capire il tipo di giorno guardando ai dati attuali.

Gennaio 26 2015 → forte nevicata, questo è un outlier, è una anomalia, quindi avremo una colonna "anomalous" con 1 se lo è.

UPLOAD/SAVE TABULAR DATA WITH PANDAS.DATFRAME

```
from pandas import read_csv
```

```
# Read csv
```

```
data1 = read_csv('data/hotspotUrbanMobility-1.csv')
```

```
# Print dataframe shape
```

```
print(data1.shape)
```

```
# Read csv
```

```
data2 = read_csv('data/hotspotUrbanMobility-2.csv')
```

```
# Print dataframe shape
```

```
print(data2.shape)
```

```
# Append dataframes
```

```
data1 = data1.append(data2, ignore_index=True)
```

```
print(data1.shape)
```

```
# Save the new dataframe as csv
```

```
data1.to_csv('data/completeDataset.csv', index=False)
```

Path of the csv to read

Dataframe shape consist of its numer of rows and column

Append 2 dataframes and ignore the indexes that pandas provides

Save the dataframe

Possiamo avere una somma dei dati dei due dataset, tramite la funzione *append*. Invece che *append* si può usare *concatenate* perché *append* è stato deprecato.

Il progetto ha una cartella per i dati dove sta il dataset, va memorizzata la locazione del dataset.

DATAFRAME MANIPULATION WITH PANDAS.DATFRAME

```
from pandas import read_csv
```

```
# Read csv
```

```
data = read_csv('data/completeDataset.csv')
```

```
# Check the content of the dataframe
```

```
print(data.describe())
```

```
# Remove column with constant values
```

```
data = data.drop('h24', axis=1)
```

```
# Remove anomalous instances
```

```
data = data.loc[data['Anomalous'] < 1]
```

```
print(data.describe())
```

```
# Save as csv
```

```
data.to_csv('data/preprocessedDataset.csv', index=False)
```

Provides statistics for each column in the dataframe

Drop the column (axis=1) whose label is 'h24'

Loc select instances by using a boolean array.

Voglio togliere le colonne che hanno solo gli stessi valori, non servono al programma di machine learning.

Con la loc rimuovo ogni riga contenente un outlier, escludi tutti gli *anomalous* = 1.

TRY THE LAST 2 SLIDES YOURSELF!

```
(198, 28)
(167, 28)
(365, 28)

```

	Anomalous	Cluster	Day	...	h22	h23	h24
count	365.000000	365.000000	365.000000	...	365.000000	365.000000	365.0
mean	0.101370	0.569063	15.720548	...	0.472210	0.384617	1.0
std	0.302232	0.729045	8.800321	...	0.373362	0.373992	0.0
min	0.000000	0.000000	1.000000	...	0.000000	0.000000	1.0
25%	0.000000	0.000000	8.000000	...	0.137218	0.130619	1.0
50%	0.000000	0.000000	16.000000	...	0.403655	0.225237	1.0
75%	0.000000	1.000000	23.000000	...	0.998552	0.783687	1.0
max	1.000000	2.000000	31.000000	...	1.000000	1.000000	1.0

```
[8 rows x 28 columns]

```

	Anomalous	Cluster	Day	...	h21	h22	h23
count	328.0	328.000000	328.000000	...	328.000000	328.000000	328.000000
mean	0.0	0.530408	15.640244	...	0.678014	0.455213	0.370639
std	0.0	0.716121	8.654346	...	0.279946	0.371232	0.374739
min	0.0	0.000000	1.000000	...	0.000000	0.000000	0.000000
25%	0.0	0.000000	8.000000	...	0.467768	0.134583	0.129144
50%	0.0	0.000000	15.500000	...	0.702473	0.381617	0.222655
75%	0.0	1.000000	23.000000	...	1.000000	0.998032	0.774066
max	0.0	2.000000	31.000000	...	1.000000	1.000000	1.000000

```
[8 rows x 27 columns]
Process finished with exit code 0
```

198 righe per il primo, 167 per il secondo, per un totale di 365. Dopo aver rimosso le anomalie, si hanno 328 righe, abbiamo rimosso 40 anomalie. h24 l'abbiamo rimossa perché la standard deviation era 0 o 1.

CREATING DATABASES WITH SQLITE3

SQLite is a database engine that makes it simple to store and work with relational data. In Python it can be used via [sqlite3](#)!

```
import sqlite3

con = sqlite3.connect('example.db')
cur = con.cursor()
# Create table and insert a row of data
cur.execute('CREATE TABLE <table_name> (<col_1>, <col_2>, <...>)' )
cur.execute('INSERT INTO <table_name> VALUES ((<val_1>, <val_2>, <...>)' )
# Save the changes
con.commit()
# Return all results of query
cur.execute('SELECT <...> FROM <...> WHERE <...> ')
cur.fetchall()
# Return first result of query
cur.execute('SELECT <...> FROM <...> WHERE <...> ')
cur.fetchone()
# Close the connection
con.close()
```

18

Qui viene spiegato come importare e creare un database sql lite.

PANDAS FOR SQLITE3 DATABASES

Once you have a working connection to the SQL database, you can use [read_sql](#) to query it or obtain a dataframe out of it and [to_sql](#) to save a dataframe as an SQL table.

```
import sqlite3
import pandas as pd
from sklearn.datasets import load_iris
from pandas import read_csv

con = sqlite3.connect('example.db')
# Obtain a dataframe
data = read_csv('data/preprocessedDataset.csv')
# Save it to the database
data.to_sql("mobility", con, if_exists="replace")
# Check the database
print(pd.read_sql("select * from mobility;", con))
# Save the changes and close the connection
con.commit()
con.close()
```

Il codice va mantenuto semplice ma ben documentato.

MORE ON TABULAR DATA WITH PANDAS

- Normalize JSON and convert them into dataframe
- Load csv as a dataframe, save a dataframe as csv
- Shape, head and quick description of a dataframe
- Dataframe concatenation and append
- Data sort by index or by value
- Data resampling and augmentation
- Dataframe selection by value or index
- Dataframe columns/row addition and elimination
- Replace by value, drop duplicate rows, group by value
- Drop NAN values , fill NAN value
- Data interpolation and signal smoothing via moving average

DATA SEGREGATION WITH PANDAS AND SCIKIT-LEARN

```
from pandas import read_csv
from sklearn.model_selection import train_test_split

# Read csv
data = read_csv('data/preprocessedDataset.csv')
print(data.describe())

# Random split 75% as training and 25% as testing
training_data, testing_data, training_labels, testing_labels =
    train_test_split(data.iloc[:, 4:len(data.columns)], data.iloc[:, 1])

# Save as csv
training_data.to_csv('data/trainingData.csv', index=False)
testing_data.to_csv('data/testingData.csv', index=False)
training_labels.to_csv('data/trainingLabels.csv', index=False)
testing_labels.to_csv('data/testingLabels.csv', index=False)
```

Split randomly 75% and 25% of the dataset (default), once it knows which are the target and the data.

iloc provides integer-location based indexing

Scikit-learn allows you also to discretize and normalize your data via minmax or standard scaler procedure. **Those are very important if you are going to use an artificial neural network!**

Data segregation: dati del dataset divisi per training e per il machine learning, spesso anche validation. In generale sono set divisi, non ci deve essere sovrapposizione tra i database.

Train test split si differenziano i data frame a partire da un dataset iniziale.

Labels = sono i valori reali per il training e per il testing, quindi per capire quanto il modello ha azzeccato.

Iloc → partizioni i dati in base a specifici gruppi di colonne.

I dati vanno sempre discretizzati/normalizzati, da 0 ad 1. Qui è già fatto, nei nostri dati invece andrà fatto, è obbligatorio.

Min max → tutto ciò che è vicino a 1 viene messo a 1, viceversa a 0, è una trasformazione lineare. Se hai outlier la normalizzazione viene sballata, è importante rimuoverli prima del training.

MODEL DESIGN 1\2

Scikit-learn provides an easy-to-use **classifier** based on artificial neural networks. Those use features, information extracted from an input data instance, to classify the instance itself. [Here you can find the documentation to use them.](#)

```
from sklearn.neural_network import MLPClassifier
from pandas import read_csv
from numpy import ravel
# Read the data
training_data = read_csv('data/trainingData.csv')
training_labels = read_csv('data/trainingLabels.csv')
testing_data = read_csv('data/testingData.csv')
testing_labels = read_csv('data/testingLabels.csv')
# Build and train the classifier
mlp = MLPClassifier(max_iter=100).fit(training_data, training_labels)
mlp.predict(testing_data) # produce the predictions
mlp.score(testing_data, testing_labels) # compute the accuracy of the model
```

Ci sono tanti altri *hyperparameters*.

Si fa il training, poi effettui la predizione e calcoli la percentuale di precisione.

MLP non è obbligatorio.

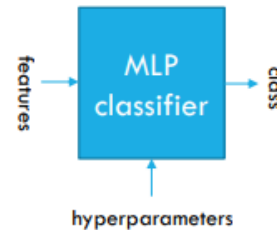
MODEL DESIGN 2\2

However, there is a number of **hyperparameters** to tune, such as:

- Hidden layer number and size

- Batch size

- Max iterations



Sometimes some recommendation can help you, but not always...

Batch size → numero di sample per ogni iterazione del training

Più il batch size è piccolo più iterazioni devi mettere per garantire un certo numero di informazioni.

HYPER-PARAMETRIZATION VIA GRID SEARCH

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV
from pandas import read_csv
from numpy import ravel
```

```
# Read the data
training_data = read_csv('data/trainingData.csv')
training_labels = read_csv('data/trainingLabels.csv')
# setup Grid Search for MLP
mlp = MLPClassifier()
# values to test
parameters = {'max_iter': (100, 200, 300)}
# apply grid search
gs = GridSearchCV(mlp, parameters)
gs.fit(training_data, ravel(training_labels))
print(pd.DataFrame(gs.cv_results_))
final_model = gs.best_estimator_
```

If "cv=None", GridSearch uses Cross-fold validation. Explicitly pass test and training sets with PredefinedSplit

The ML model

Max_iter is the name of the hyperparameter of MLPclassifier

Use ravel to transform dataframe to narray

Access all the results of each trial and the best model

Si cambiano i parametri in base ad un range di valori possibili che tu configuri. In questo caso si modifica solo *max_iter*, il quale assumerà vari valori. Alla fine, mi dirà il migliore modello in base

alle combinazioni di parametri, puoi avere un csv di ogni parametrizzazione. Non vuoi magari considerare solo il best estimator, ma magari i migliori 3 o migliori 5.

MLP classifier dopo la fit e la grid search sono lo stesso oggetto.

CLASSIFIER EVALUATION

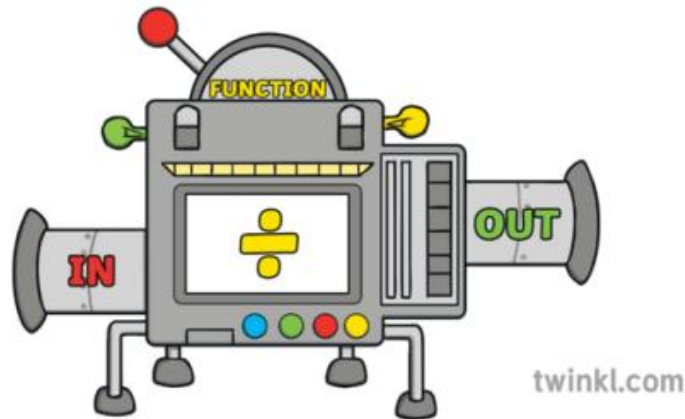
```
# compute labels on testing data
labels = final_model.predict(testing_data)
# evaluate accuracy score
score = accuracy_score(ravel(testing_labels), labels)
```

You can use different
performances measures

MORE ON MODEL EVALUATION...

- A lot of algorithms and performance metrics available on scikit-learn
- How much time does an instruction take to execute?
- Performance visualization with scikit-learn, and how to write them on files

INPUT/OUTPUT OF FUNCTIONAL MODULES



JSON TO SAVE THE BEST MODELS PARAMS

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV
from pandas import read_csv
from numpy import ravel
import json

# Read the data
training_data = read_csv('data/trainingData.csv')
training_labels = read_csv('data/trainingLabels.csv')
# setup Grid Search for MLP
mlp = MLPClassifier()
# values to test
parameters = {'max_iter': (100, 200, 300)}
# apply grid search
gs = GridSearchCV(mlp, parameters)
gs.fit(training_data, ravel(training_labels))
# save best configuration
config_path = 'config/modelConfiguration.json'
with open(config_path, 'w') as f:
    json.dump(gs.best_params_, f)
```

The ML model

Max_iter is the name of the hyperparameter of MLPclassifier

Use ravel to transform dataframe to narray

Save the best hyperparameter as JSON

29

Dump serve per scrivere i migliori parametri in un Json, è l'azione di scrivere in un Json file.

Puoi trasformare una istanza di una classe in Json.

FROM JSON TO OBJECT

```
import json

class Params():
    def __init__(self, M):
        self.max_iter = M

best_par = Params(60)
out_file = open("config/modelConfiguration.json", "w")
json.dump(best_par, out_file)
out_file.close()

# load best configuration
config_path = "config/modelConfiguration.json"
with open(config_path, "r") as f:
    params = json.load(f)

params_object = Params(**params)
```

Use a simple class to keep the model's parameters. You can use `json.dump()` to store it in a file.

Load the JSON. Remember, it should be compliant to the expected json schema i.e. pass the validation

Create a new object, and pass the JSON dictionary as a map to convert JSON data into a custom Python Object

Hai una classe con dei parametri, puoi salvarla come un Json file. Si crea un nuovo Params object, si salva in json. Praticamente si mandano informazioni via Json, l'altra parte la riceve e istanzia oggetti (comunicazione fra componenti).

JSON TO LOAD AN OBJECT

```
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
from pandas import read_csv
from numpy import ravel
import json

data1 = read_csv('data/hotspotUrbanMobility-1.csv')
data2 = read_csv('data/hotspotUrbanMobility-2.csv')
data = data1.append(data2, ignore_index=True)
data = data.drop('h24', axis=1)
data = data.loc[data['Anomalous'] < 1]
data.to_csv('data/preprocessedDataset.csv', index=False)
data = read_csv('data/preprocessedDataset.csv')
training_data, testing_data, training_labels, testing_labels = \
    train_test_split(data.iloc[:,
        4:len(data.columns)], data.iloc[:, 1])
# ...try to use the default 70-30 split

mlp = MLPClassifier(random_state=0)
parameters = {'max_iter': (100, 200, 300)}
# ...try to use all the hyper-params at slide 63
gs = GridSearchCV(mlp, parameters)
#...try PredefinedSplit...
gs.fit(training_data, ravel(training_labels))
#...or try to pick the first runner up config
config_path = 'config/modelConfiguration.json'
with open(config_path, 'w') as f:
    json.dump(gs.best_params_, f)
with open(config_path, 'r') as f:
    params = json.load(f)
model = MLPClassifier(**params).fit(training_data,
    ravel(training_labels))
labels = model.predict(testing_data)
score = accuracy_score(ravel(testing_labels), labels)
print(score)
```

What's the accuracy score of your classifier?

MODEL DEPLOYMENT WITH JOBLIB

```
from pandas import read_csv
from sklearn.neural_network import MLPClassifier
from numpy import ravel
import joblib

# Read the training data
training_data = read_csv('data/trainingData.csv')
training_labels = read_csv('data/trainingLabels.csv')
# train the model
initial_model = MLPClassifier(random_state=0).fit(training_data, ravel(training_labels))
# save the model
joblib.dump(initial_model, 'config/fitted_model.sav')
#[...]
# Read new data
testing_data = read_csv('data/testingData.csv')
testing_labels = read_csv('data/testingLabels.csv')
# load the model
model = joblib.load('config/fitted_model.sav')
# Evaluate the initial model on new data
print('Evaluation score:')
print(model.score(testing_data, ravel(testing_labels)))
```

Random_state
for
reproducibility

Save a fitted
model to file

Load a fitted
model from file

Use the model
with new data

Alla fine, hai il modello trainato.

Con Joblib hai sempre la dump e la load, per salvare e leggere un modello di machine learning allenato. Puoi leggere i training data, i training label ecc. infine si salva il modello e si carica, in modo tale da avere uno score, stai spezzando il training e il testing.

Ogni machine learning approach ha una pseudo randomization. Se abbiamo ottime performance e avere sempre queste, si fissa un random state, che è composto da un random seed, va fissato per rendere il tutto “ripetibile”, altrimenti prenderai sempre dati diversi per il training e il test.

PRESENT YOUR RESULTS



Vogliamo presentare i risultati in maniera grafica.

MATPLOTLIB

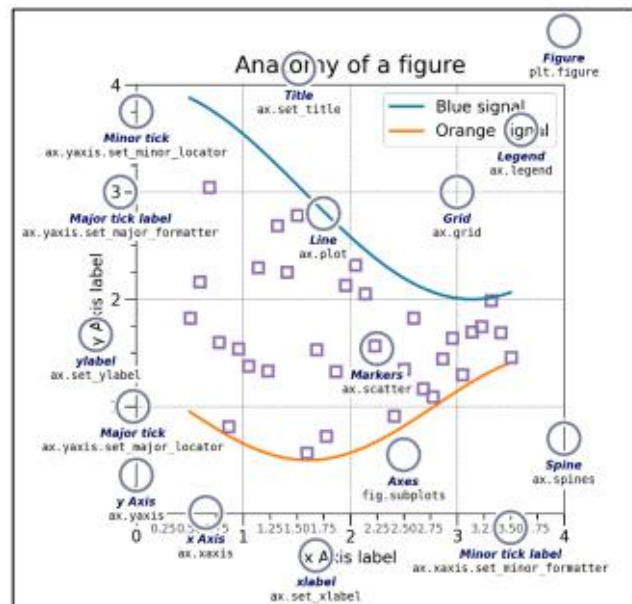
Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

Each **element** in a figure can be addressed and modified.

There are two ways to use Matplotlib:

1. **Explicit:** explicitly create Figures and Axes, and call methods on them (the "object-oriented (OO) style").
2. **Implicit:** Rely on pyplot to implicitly create and manage the Figures and Axes, and use pyplot functions for plotting.

See [Matplotlib Application Interfaces \(APIs\)](#) for an explanation of the trade-offs between the implicit and explicit interfaces.



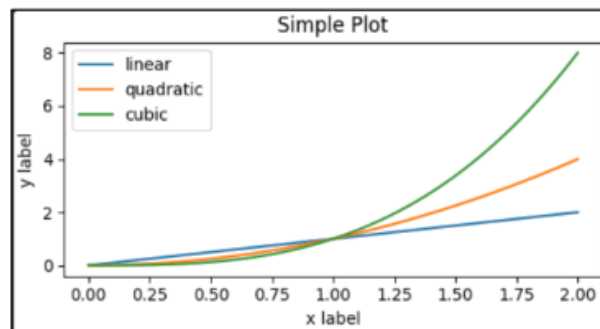
In figura abbiamo le varie proprietà di un grafico prodotto tramite la libreria matplotlib.

MULTIPLE LINES GRAPH

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2, 100) # Sample data

plt.figure(figsize=(5, 2.7), layout='constrained')
plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadratic')
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend()
plt.show()
```



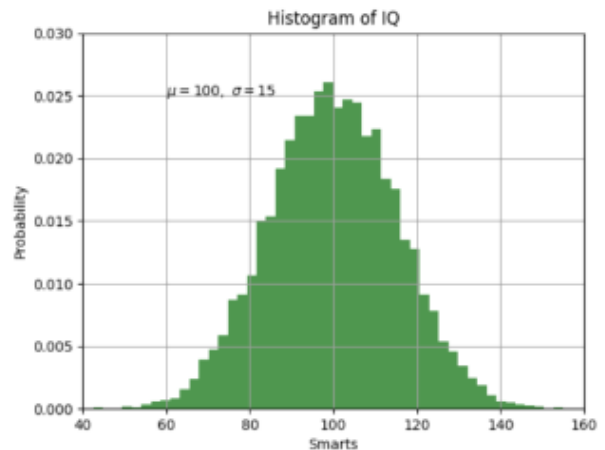
HISTOGRAMS

```
import matplotlib.pyplot as plt
import numpy as np

mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50,
density=True, facecolor='g', alpha=0.75)

plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)
plt.show()
```

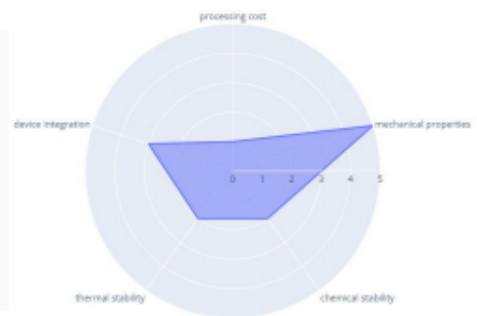


PLOTLY AND RADAR CHART

A [Radar Chart](#) (also known as a spider plot or star plot) displays multivariate data in the form of a two-dimensional chart of quantitative variables represented on axes originating from the center.

Use [plotly.express](#) to generate them!

```
import plotly.express as px
import pandas as pd
df = pd.DataFrame(
    ( r=[1, 5, 2, 2, 3],
      theta=['processing cost',
             'mechanical properties', 'chemical stability',
             'thermal stability', 'device integration']))
fig = px.line_polar(df, r='r', theta='theta', line_close=True)
fig.update_traces(fill='toself')
fig.show()
```



...MORE ON GRAPHS AND [MATPLOTLIB.PYPILOT](#)

- [Plot](#) and [save a figure](#)
- [Histogram](#) and barplot
- [Boxplots](#)
- [Scatterplot](#)
- Create single and [multiple graphs](#)
- Find out more on this [TUTORIAL!!!](#)

7.3. Module testing and communication

CODE CORRECTNESS CHECKING

PyCharm supports **pytest**, providing many testing **features** such as:

1. Dedicated test runner.
2. Code completion for test subject.
3. Code navigation.
4. Detailed failing assert reports.
5. Multiprocessing test execution.

..A QUICK NOTE

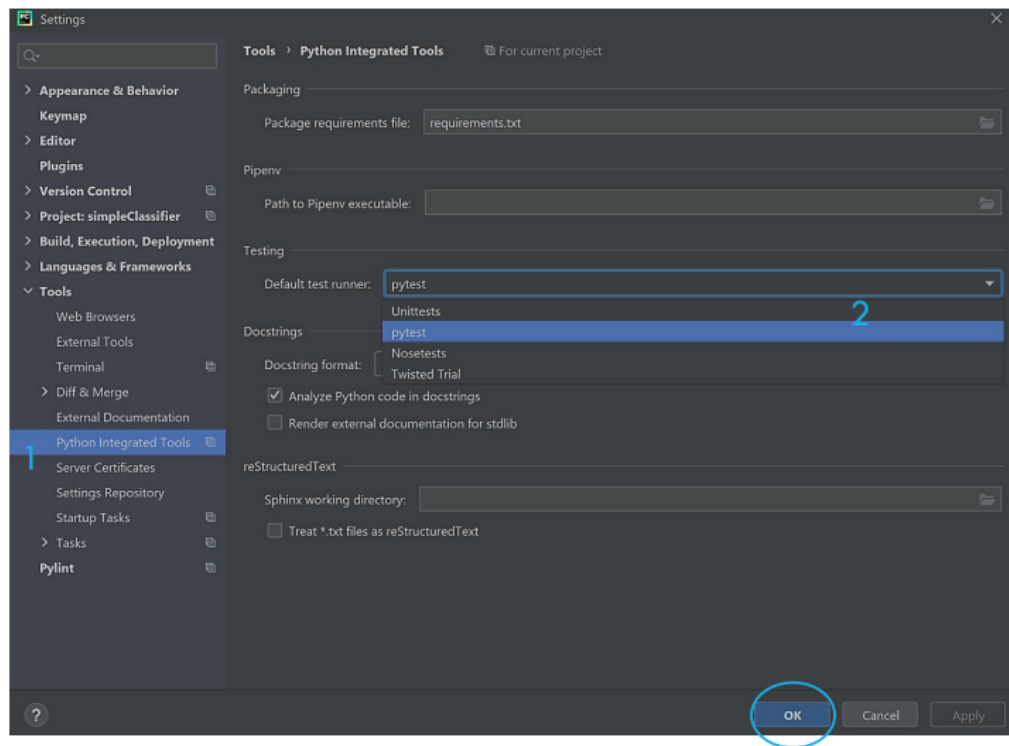
In the next slides, the functions described in the previous lessons are represented by a hypothetical class "irisClassifier" and its methods:

1. **Ingestion**: allows data to be uploaded via .csv and combine them
2. **Segregation**: prepare training and testing sets
3. **Train**: train the machine learning (ML) module
4. **Evaluate**: assess the performance of the ML module

In order to perform the next steps, prepare a "evaluate" function in which your trained model can be tested to obtain its score as result.

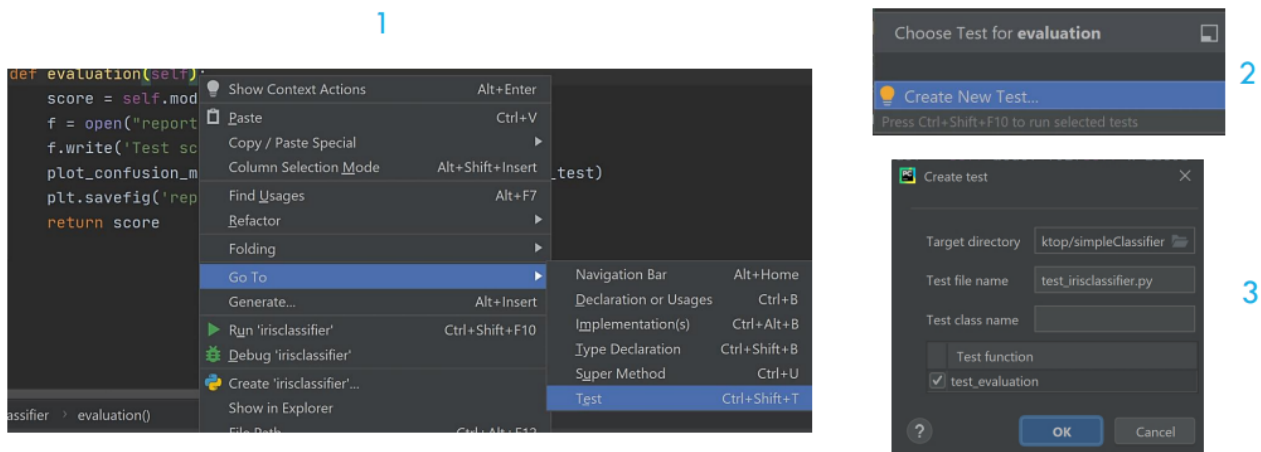
ENABLE PYTEST

1. File > Settings > Python Integrated Tools
2. Set “pytest” as the “Default test runner”



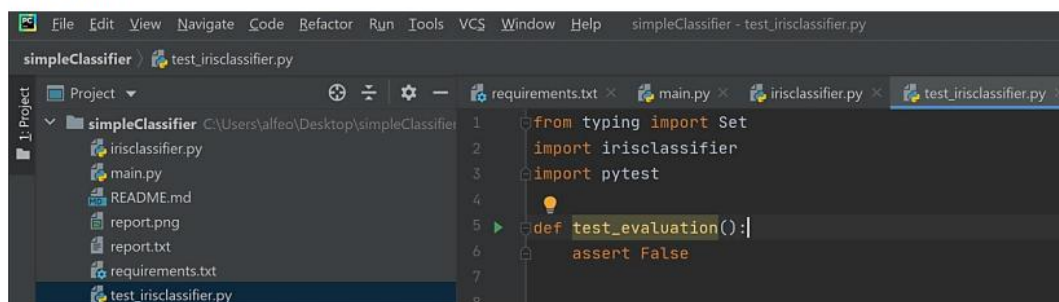
CREATE A NEW TEST

1. Put the cursor on the bracket of a **method declaration**, right-click and select **Go To > Test**
2. Create a new test script
3. Give it a directory, a name, and a method to test



BUILD AND RUN A TEST 1/3

1. Fill the test function and the **assert** statement. If this statement is true the test is PASSED!



2. Click "Edit pytest for ..."



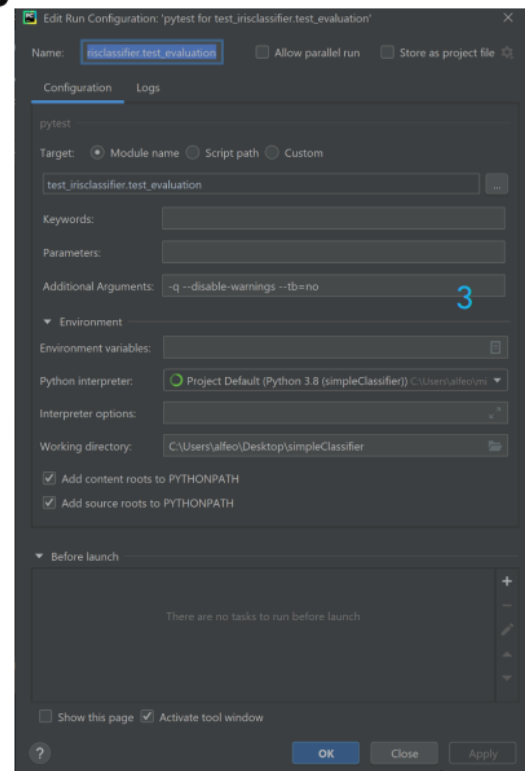
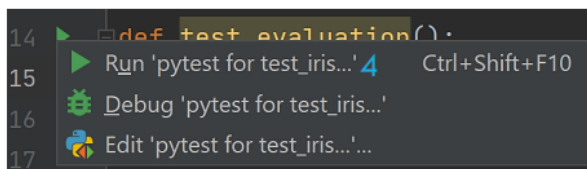
BUILD AND RUN A TEST 2/3

3. Reduce the verbosity of the pytest outcome to improve readability. Put the following Additional Arguments:

“ -q --disable-warnings --tb=no ”

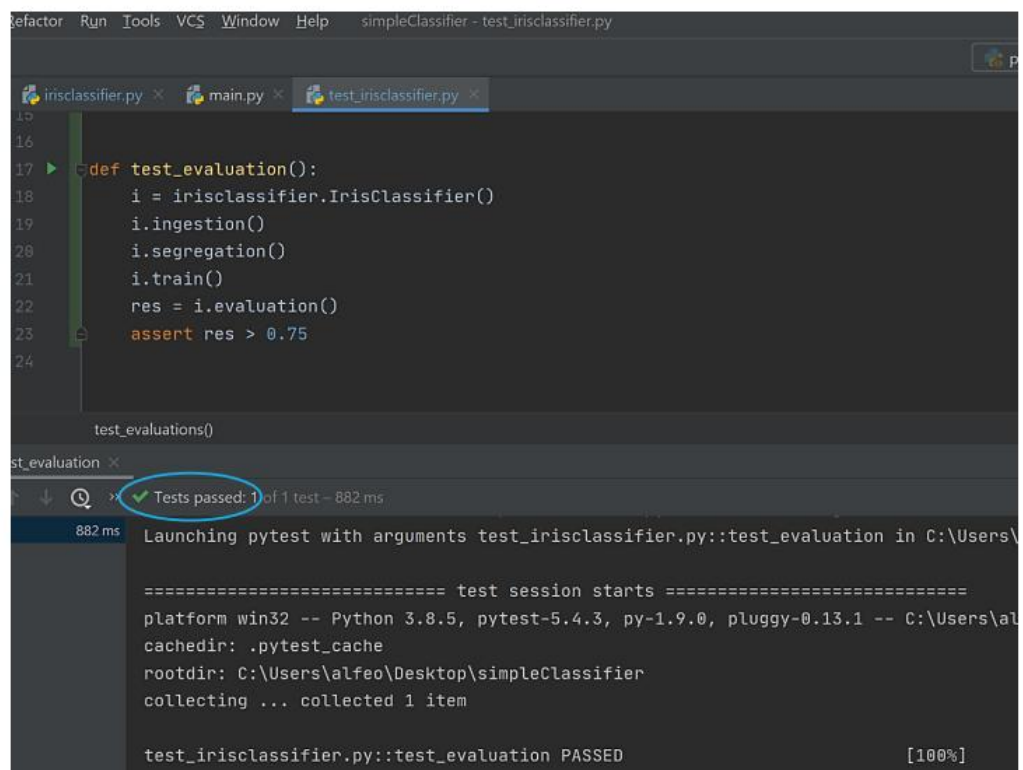
- q, quiet output
- disable-warnings, don't show most of the warnings
- tb=no, don't show any traceback

4. Click **OK** and Run the test



Vogliamo solo controllare gli errori pesanti non gli warning.

BUILD AND RUN A TEST 3/3



La valutazione dà un accuracy score, se supera una certa soglia il test è passato.

Bisogna solo inserire del codice nella funzione di test, il resto è fatto automaticamente.

0.75 può diventare un parametro attraverso l'uso di un *decorator*.

TEST PARAMETRIZATION

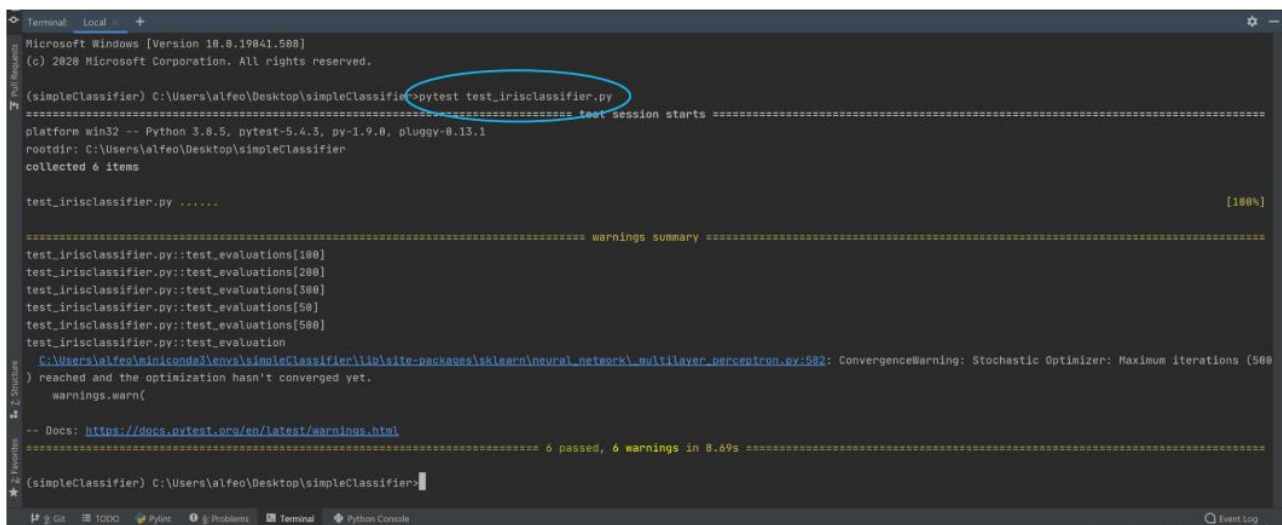
The **Pytest** framework makes it easy to write tests to support multiple and complex functional testing for applications and libraries. Pytest enables test **parametrization** at several levels, such as:

@pytest.fixture provide a fixed baseline (e.g., an instance of a class) upon which tests can reliably and repeatedly execute. Moreover, via **scope** controls it is possible to check how often a fixture gets called.

@pytest.mark.parametrize allows the definition of multiple sets of arguments and fixtures for testing purpose.

Dato che molte classi sono procedure, ci focalizziamo sul decorator *parametrize*.

USE PYTEST VIA COMMAND LINE



```
Microsoft Windows [Version 10.0.19041.588]
(c) 2020 Microsoft Corporation. All rights reserved.

(simpleClassifier) C:\Users\alfeo\Desktop\simpleClassifier>pytest test_irisclassifier.py
===== test session starts =====
platform win32 -- Python 3.8.5, pytest-5.4.3, py-1.9.0, pluggy-0.13.1
rootdir: C:\Users\alfeo\Desktop\simpleClassifier
collected 6 items

test_irisclassifier.py ..... [100%]

===== warnings summary =====
test_irisclassifier.py::test_evaluations[100]
test_irisclassifier.py::test_evaluations[200]
test_irisclassifier.py::test_evaluations[300]
test_irisclassifier.py::test_evaluations[50]
test_irisclassifier.py::test_evaluations[500]
test_irisclassifier.py::test_evaluation
C:\Users\alfeo\miniconda3\envs\simpleClassifier\lib\site-packages\sklearn\network\multilayer_perceptron.py:582: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500)
reached and the optimization hasn't converged yet.
  warnings.warn(

-- Docs: https://docs.pytest.org/en/latest/warnings.html
===== 6 passed, 6 warnings in 8.69s =====

(simpleClassifier) C:\Users\alfeo\Desktop\simpleClassifier>
```

BUILD AND RUN MANY TESTS

1. Prepare a set of values to test
2. Test those by passing them to the test function
3. Now you have one evaluation for each test value!

```
performance_thresholds = {0.50, 0.75, 0.95} 1

@pytest.mark.parametrize('th', performance_thresholds)
def test_evaluations(th):
    i = irisclassifier.IrisClassifier()
    i.ingestion()
    i.segregation()
    i.train()
    res = i.evaluation()
    assert res > th 2
```

```
===== 3 passed, 3 warnings in 1.30s ===== 3

Process finished with exit code 0
PASSED [ 33%]Test score: 0.8947368421052632
PASSED [ 66%]Test score: 0.8947368421052632
PASSED [100%]Test score: 0.9736842105263158
```

Invece che 0.75 si passa direttamente un parametro, che è una collezione di numero diversi. Si possono quindi runnare 3 differenti test.

Si vuole testare una situazione media, una situazione estrema ecc., questo si traduce in semplici parametri.

7.3.1. Comunicazione con i moduli

Ogni modulo inizia con delle righe specializzate per comunicare con gli altri moduli. Ogni modulo comunica con gli altri, ognuno sviluppa una parte (un modulo) della pipeline.

REST API AS INTERFACES 1\3

REST APIs are becoming the standard de-facto for machine-to-machine communication, and can be used to communicate among the different modules of your project. For instance you can:

- Generate some **end-points** for the functionalities to expose
- Use **POST** method to create a new resource (e.g. build the training set, pre-process it)
- Use **GET** method to retrieve a resource from different components of the application (e.g. the configuration file, or the JSON version of a configuration object)
- Implement **DELETE** and **PUT** functionalities, that's up to you!

Si possono usare giusto i POST e GET, bastano questi perché i moduli comunicano con JSON, files e ricevono delle risposte.

Le REST API necessitano degli end point, ottenuti da POST e GET.

REST API AS INTERFACES 2\3

In Python you can use [Flask](#) and [Flask-Restful](#) (install them via pip) to build and use a simple REST server. For instance you can define the end-point «/» as it follows

```
from pandas import read_csv
from flask import Flask, request
from flask_restful import Resource, Api

app = Flask(__name__)
api = Api(app)

class HelloWorld(Resource):
    def get(self):
        data = read_csv('data/preprocessedDataset.csv')
        data = data.to_dict()
        return {'data': data}, 200

    def post(self):
        some_json = request.get_json()
        return {'you sent ': some_json}, 201

api.add_resource(HelloWorld, '/')

if __name__ == '__main__':
    app.run(debug=True)
```



```
Run: main
* Serving Flask app 'main' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a
  production environment. Use a production WSGI server instead.
* Debug mode: on
* Restarting with stat
* Debugger is active!
* Debugger PIN: 977-993-967
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
127.0.0.1 - - [13/Oct/2021 16:37:27] "GET / HTTP/1.1" 200 -
```

15

Si usano quelle due librerie in figura.

Abbiamo una classe HelloWorld che ha una funzione get e post, la get prende il dataset salvato localmente e ritorna un dizionario specifico, il dataset trasformato in un dizionario.

Api.add → aggiungo un endpoint, ritorno la classe HelloWorld.

TRY IT YOURSELF USING REQUEST

1. Install **requests** package (e.g. via pip) from the **Terminal**
2. Send a request via **Python Console**

```
Python Console ->
PyDev console: starting.

Python 3.8.11 (default, Aug 6 2021, 09:57:55) [MSC v.1916 64 bit (AMD64)] on win32
>>> import requests
>>> r = requests.post('http://127.0.0.1:5000/', json={"key": "value"})
>>> r.json()
{'you sent ': {'key': 'value'}}
>>> r = requests.get('http://127.0.0.1:5000/')
>>> r.json()
{'data': {'Anomalous': {'0': 0, '1': 0, '2': 0, '3': 0, '4': 0, '5': 0, '6': 0, '7': 0, '8': 0, '9': 0, '10': 0, '11': 0, '12': 0, '13': 0, '14': 0, '15': 0, '16': 0, '17': 0, '18': 0, '19': 0, '20': 0, '21': 0, '22': 0, '23': 0, '24': 0, '25': 0, '26': 0, '27': 0, '28': 0, '29': 0, '30': 0, '31': 0, '32': 0, '33': 0, '34': 0, '35': 0, '36': 0, '37': 0, '38': 0, '39': 0, '40': 0, '41': 0, '42': 0, '43': 0, '44': 0, '45': 0, '46': 0, '47': 0, '48': 0, '49': 0, '50': 0, '51': 0, '52': 0, '53': 0, '54': 0, '55': 0, '56': 0, '57': 0, '58': 0, '59': 0, '60': 0, '61': 0, '62': 0, '63': 0, '64': 0, '65': 0, '66': 0, '67': 0, '68': 0, '69': 0, '70': 0, '71': 0, '72': 0, '73': 0, '74': 0, '75': 0, '76': 0, '77': 0, '78': 0, '79': 0, '80': 0, '81': 0, '82': 0, '83': 0, '84': 0, '85': 0, '86': 0, '87': 0, '88': 0, '89': 0, '90': 0, '91': 0, '92': 0, '93': 0, '94': 0, '95': 0, '96': 0, '97': 0, '98': 0, '99': 0, '100': 0, '101': 0, '102': 0, '103': 0, '104': 0, '105': 0, '106': 0, '107': 0, '108': 0, '109': 0, '110': 0, '111': 0, '112': 0, '113': 0, '114': 0, '115': 0, '116': 0, '117': 0, '118': 0, '119': 0, '120': 0, '121': 0, '122': 0, '123': 0, '124': 0, '125': 0, '126': 0, '127': 0, '128': 0, '129': 0, '130': 0, '131': 0, '132': 0, '133': 0, '134': 0, '135': 0, '136': 0, '137': 0, '138': 0, '139': 0, '140': 0, '141': 0, '142': 0, '143': 0, '144': 0, '145': 0, '146': 0, '147': 0, '148': 0, '149': 0, '150': 0, '151': 0, '152': 0, '153': 0, '154': 0, '155': 0, '156': 0, '157': 0, '158': 0, '159': 0, '160': 0, '161': 0, '162': 0, '163': 0, '164': 0, '165': 0, '166': 0, '167': 0, '168': 0, '169': 0, '170': 0, '171': 0, '172': 0, '173': 0, '174': 0, '175': 0, '176': 0, '177': 0, '178': 0, '179': 0, '180': 0, '181': 0, '182': 0, '183': 0, '184': 0, '185': 0, '186': 0, '187': 0, '188': 0, '189': 0, '190': 0, '191': 0, '192': 0, '193': 0, '194': 0, '195': 0, '196': 0, '197': 0, '198': 0, '199': 0, '200': 0, '201': 0, '202': 0, '203': 0, '204': 0, '205': 0, '206': 0, '207': 0, '208': 0, '209': 0, '210': 0, '211': 0, '212': 0, '213': 0, '214': 0, '215': 0, '216': 0, '217': 0, '218': 0, '219': 0, '220': 0, '221': 0, '222': 0, '223': 0, '224': 0, '225': 0, '226': 0, '227': 0, '228': 0, '229': 0, '230': 0, '231': 0, '232': 0, '233': 0, '234': 0, '235': 0, '236': 0, '237': 0, '238': 0, '239': 0, '240': 0, '241': 0, '242': 0, '243': 0, '244': 0, '245': 0, '246': 0, '247': 0, '248': 0, '249': 0, '250': 0, '251': 0, '252': 0, '253': 0, '254': 0, '255': 0}}
```

Si può avere una richiesta ad un server locale, si ottiene un JSON. Con la POST invio, con la GET ricevo. Nel caso in figura viene eseguito il codice precedente, si ottiene il dataset con la GET.

REST API AS INTERFACES 3\3

You can also use an **argument** with your request as it follows

```
from flask import Flask, request
from flask_restful import Resource, Api

app = Flask(__name__)
api = Api(app)

class Multi(Resource):
    def get(self, num):
        return {'result': num * 10}

api.add_resource(Multi, '/multi/<int:num>')

if __name__ == '__main__':
    app.run(debug=True)
```

La GET si aspetta uno specifico input.

Si aggiunge un endpoint con *api.add_resource*, si lascia anche lo spazio per un argomento chiamato *num*.

TRY IT YOURSELF USING REQUEST

Send a request via Python Console

```
>>> r = requests.get('http://127.0.0.1:5000/multi/4')
>>> r.json()
{'result ': 40}
```

Passiamo 4, questo numero verrà moltiplicato poi per 10.

SENDING FILES

```
from flask import Flask, send_file

app = Flask(__name__) # always use this

# the file in "file_path" will be sent due to a GET request on the endpoint "/get_file"
@app.route("/get_file", methods=['GET'])
def test():
    file_path = 'file.sav'
    return send_file(file_path, as_attachment=True)

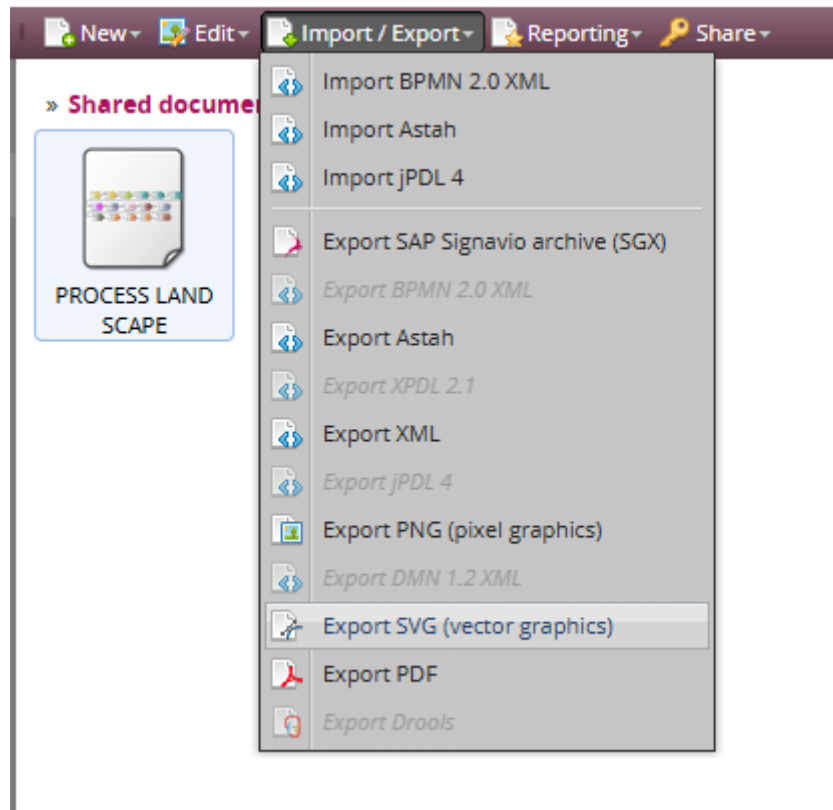
app.run()
```

Siamo anche capace di inviare i file, oltre ai già visti JSON e numeri. Anche qui devi definire una specifica root e un metodo.

EXERCISE

Can you send the trained model you saved (via joblib) in the last laboratory to your closest colleague?

8. Consigli



Esportare e convertire le immagini dei diagrammi da *svg* in *emf* con il link: <https://cloudconvert.com/svg-to-emf>.

Non importare *emf* nel browser ma aprilo sempre nel relativo word.

4 o 5 righe riguardo la motivazione del landscape. Perché hai messo quella cardinalità in quel punto? “Questo è 1 a M perché ... “. Servono 1 o 2 motivazioni per membro per attività. Motivare 1 esempio di cardinalità. “Questo è un milestone per questo .. “ , “questo non è un milestone anche se c’è 1 a M perché ho deciso di stoppare il processo qui perché.. “

9. Dettagli generali sul progetto

- SUBMISSION: via the project submission system, a member of the team submits the report in pdf format (generated from the shared docx file) and a zip file downloaded from the Github;
 - TRELLO AND CLOCKIFY: the Trello board tasks and the Clockify dashboard are also discussed; each member should spend 90 hours;
 - WF REQUIREMENTS: Process Landscape (Signavio), BPMN diagram (Signavio);
 - WF ANALYSIS: Use case, mockup; detailed use case; class diagram; sequence diagram;
 - WF DESIGN: class diagram; sequence diagram;
 - WF IMPLEMENTATION: Deployment diagram;
 - WF TEST: report with non-automation, non-responsiveness, non-elasticity, non-resiliency, non-interoperability; a live automation test and a live maintenance will be required;
 - SETUP AND USE MANUAL;
 - CODE QUALITY: formal (pylint) and inspection.
- WF REQ: generiamo process landscape e BPMN diagram
 - WF ANALYSIS: aggiungiamo le analysis card, sviluppiamo lo use case, il mockup class diagram e sequence diagram. Questo è quello che facciamo solo per gli human task.

Obiettivo: sviluppiamo una pipelined application che è fatta da diverse applicazioni *Python* organizzate insieme (ingestion system, ..) che scambiano messaggi. Alcuni sistemi hanno il loro server che ascolta le richieste e alcuni uno client che manda i messaggi. Progettiamo l'architettura e poi alla fine misuriamo differenti indicatori come *responsiveness*, ecc.

Caratteristiche generali:

- La documentazione è fatta principalmente di diagrammi. Possiamo aggiungere alcuni commenti nel documento, motivazioni ecc. per aiutarci prima dell'esame.
- Potremmo poi aggiungere alcune project management card.
- Useremo la pylint code analysis.
- Useremo JSON format anche per i file di configurazione. Nello sviluppo abbiamo pycharm, sviluppiamo png e JSON. Il test è fatto nell'ambiente di sviluppo., non svilupperemo reali interfacce, abbiamo bisogno di un portale web per questo.
- La factory è del classifier, infatti il classifier ha diverse facce.
- Potremmo muoverci nella produzione del classifier (se abbiamo tempo), ma noi rimaniamo solo nella fase di sviluppo.
- Avremo una dashboard con, ad esempio, un balancing bar diagram per vedere il balancing dei dati, per il quale verrà prodotto un png. Le immagini png saranno mantenute localmente, stessa cosa per i JSON (il JSON per i dati in json, il png per i grafici).
- Durante l'esame dobbiamo motivare il risultato del test, tipo la velocità ecc., ma non importa la *accuracy* del classifier, dobbiamo solo motivare i nostri risultati.
- Ci sarà richiesto di cambiare il codice.
- La configurazione viene mandata ad una persona quindi al messaging system che non è un vero e proprio sistema ma è un messaggio su teams di una persona che ha visto che il

classificatore non è buono, quindi chi si occupa della configurazione deve magari cambiare il numero di label da 3 a 4 o viceversa, poi spegnere il sistema e riaccenderlo. Non è detto che tutti lo debbano fare, magari solo alcune persone.

10. Modellazione task

Dobbiamo capire come modellizzare i task, creando un UML class diagram. Usiamo visual paradigm per UML. Dobbiamo aggiungere una card per ogni human task, ma solo quelli human presenti nel BPMN. Abbiamo una user interface per ogni human task.

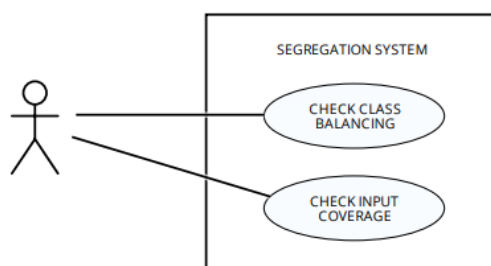
Dobbiamo sviluppare 3 differenti componenti di questo file.

TASK MODELING

Task level modeling concerns mainly human tasks. Often a human task is a use case, but sometimes a fragment of workflow with tied human tasks is a unique use case. Use **Visual Paradigm** for use cases, detailed use cases, class diagrams, but for mockups use **Diagrams.net**

For each use case, **add an analysis Trello card**, with title: **"TASK <name of the task>"**, description: **"use case, detailed use case, mockup"**.

First step: for each system, determine its use cases.



Abbiamo differenti sistemi, abbiamo uno use case per ogni human task, ci si muove verso un system oriented vision. Ci sono sistemi complessi e semplici, ci possiamo aiutare fra noi se alcune cose non troppo complicate. Possiamo bilanciare il work load scambiandoci informazioni, questo sarà fatto scambiandoci cards.

For each human task, design a user interface (mockup) suitable for the decision-making task; design a detailed use case for both the interaction and for the decision logic.

For mockups use **Diagrams.net** see the file “From performance to productivity evaluation” for starting examples of mockups.

It follows a textual description of the most important human task. It is strongly recommended to model a standard basic factory in the AS-IS version, and to focus on improvements in the To-BE process.

Check class balancing: to check whether classes are balanced or not, on the basis of a balancing

tolerance parameter; consider a **bar chart** with the number of instances for each class; e.g. 5% of balancing tolerance means that the numbers of instances can be different for at most 5% with respect to their average. Consider that a number of sessions to collect is set in the development phase, as well as a number of sessions needed for training.

Check input coverage: to check whether the input space of the network is definitively covered for each feature. It means that by collecting other sessions, the distribution does not change. Consider a **radar scatter** (for continuous features) or **radar bubble** (for discrete features) chart.

<https://tinyurl.com/radar-scatter>

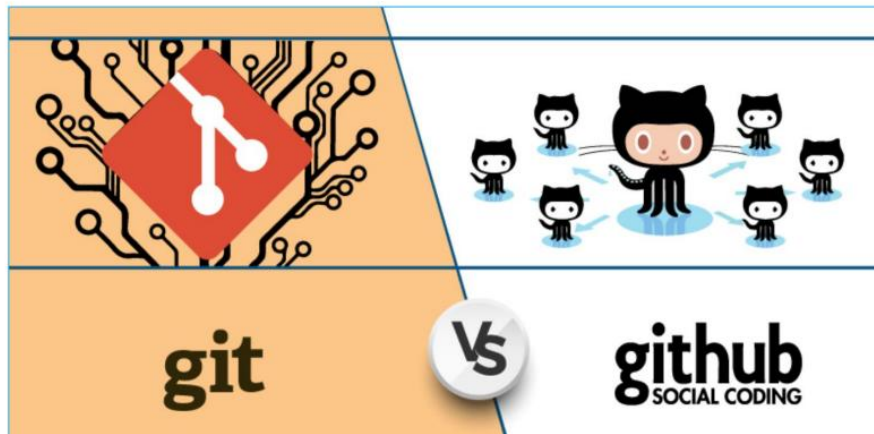
<https://tinyurl.com/radar-bubble>

Mockup: è una foto della dashboard, lo creiamo con *draw.io*. Per esempio, il diagramma a barre si crea prima della implementazione.

Si parte dalla descrizione narrativa, si sviluppa lo use case semplificato e poi quello dettagliato per ogni human task.

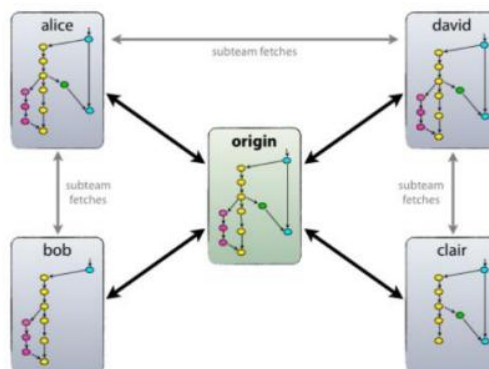
11. Project Versioning (Alfeo)

GIT AND GITHUB

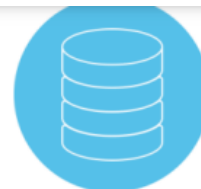


GIT

Git is a **distributed Version Control System** in which each participant can have different view of the project.



REPOSITORY



A Repository (or “**repo**”) is a data structure containing all project’s **files and history**. You can **clone** a remote repo with the clone command:

```
git clone <repo url>
```

In a repo some files starting with “.git”, those are special files and folders used to store the project history and to instruct git.

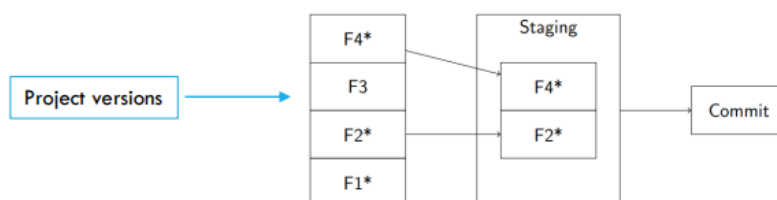
- .git folder containing the project history (*do not touch!*)
- .gitignore file containing ignoring rules
- .gitmodules file containing submodules information

Ogni volta che fai una commit salvi la storyline delle modifiche, per recuperare magari le versioni precedenti.

COMMIT

A **commit** is immutable and atomic units of modification within a project, a snapshot of the project in a given time, it is uniquely identified by its hash and can be commented by a message. A git commit is a two-stages process.

- Add files to the staging environment `git add file/folder`
- Commit the changes `git commit -m "commit message"`



HEAD is a Git variable that points to the most recent commit. HEAD can be changed as it follows, the second option change also the files in your repo:

```
git reset <NEW_HEAD>
```

```
git reset --hard <NEW_HEAD>
```

Tags allows pointing to specific commits that represent milestones.

Ogni cambiamento è legato ad una commit, la quale ha anche altri dettagli (owner della commit, descrizione,...).

La prima procedura è commit, la seconda è chiudere la modifica e pusharla.

BRANCH...

A **branch** is a semantically significant collection of commits. A commit is **always** in a branch. As a commit represent an atomic unit of modification, a branch represent a continuous flow of modifications.

- **git branch**

- Show all the existing branches and the active one (denoted with an asterisk)

- **git branch <branch>**

- Create a new branch called <branch>

- **git checkout <branch>**

- Switch working on the branch <branch>

...AND MERGE

Different branches (or even single commits) can be **merged** in order to transfer all the changes developed in different branches.

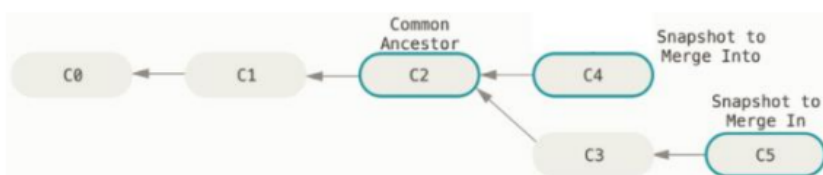
All you have to do is **check out** the branch you wish to merge into and then run the git merge command:

- **git merge <branch>**

- Merges <branch> in the active branch

- **git merge --abort**

- Abort the merge and restore everything as it was before git merge



Il branch è una collezione di fotografie. Puoi creare un flow parallelo di commit, si possono avere tanti branches e possono essere fusi se non ci sono conflitti.

HANDLING MERGE CONFLICTS

If the commits changed different files or different sections of the same files the merge is **without conflicts**, as it may happen in your project since each one of you should work on a different module!

If the commits changed the same sections of the same files (at least once) the commit is **with conflict**.

Handling conflicts is pretty easy. When a conflict is detected git puts in the place of the conflict:

```
Some non-conflicting text
<<<<<<< HEAD
CONFLICTING PORTION COMING FROM HEAD
=====
CONFLICTING PORTION COMING FROM bugfix
>>>>>>> bugfix
Some other non-conflicting text
```

Is up to you keep the portions that you want and then git add and git commit the changes

Nel nostro progetto non ci saranno conflitti, ogni modulo sarà indipendente, tranne la classe padre che tutti avremo in comune.

REMOTES

Git is distributed. A **remote** is a reference to a remote instance of the repository. The default remote is called **origin**: if you clone a repository then origin points to the cloned repo.

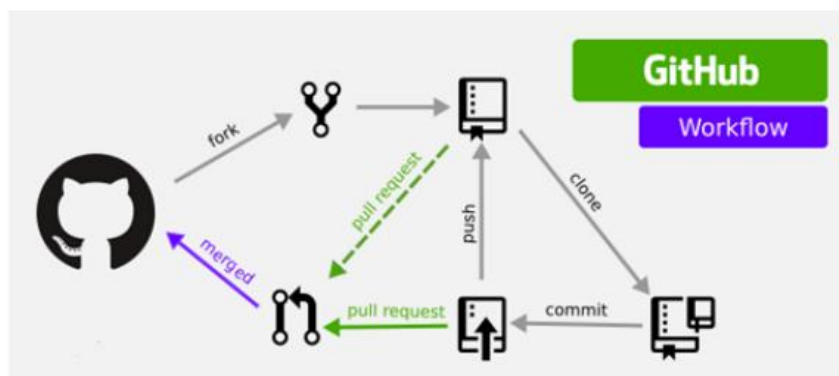
- **git fetch <remote> <branch>**
 - Fetch the commits from branch <branch> of <remote> in the local branch <remote>/<branch>
- **git pull <remote> <branch>**
 - Fetch the commits from branch <branch> of <remote> and merge them in the local <branch>
- **git push <remote> <branch>**
 - Push the state of the local branch to the origin branch <branch>

GITHUB



- GitHub is Git server.
- Used by over 65 Million Users
- Contains over 200 Million Repositories
- Largest source code host in the world

MORE ON GITHUB: FORK AND PULL



In GitHub any public repository can be **Forked**. After a fork you own an exact copy of that repository into yours (you are the administrator). A fork is not a Git clone.

If you want to ask for the integration of your changes in the forked repository you can create a **Pull Request**. In the pull request you have to specify the destination branch (original repository) and the source branch (your repository).

Fork significa che tu diventi l'owner di una visione privata tua. Fare una pull request significa chiedere la tua modifica può essere fusa con il software originale.

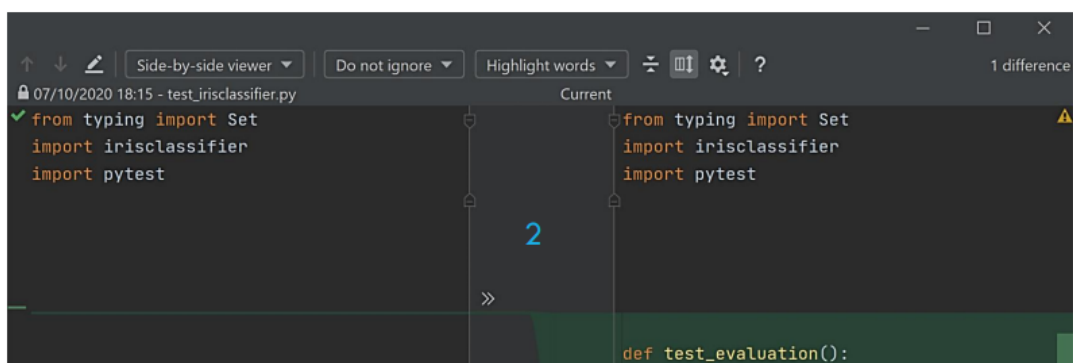
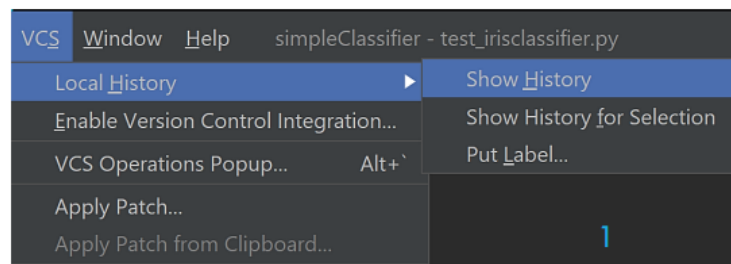
PYCHARM VCS



BASIC VERSION CONTROL

In Pycharm the Version Control can be accessed locally:

1. via **VCS > Local History**
2. Stores only local and recent change in the project highlighting them

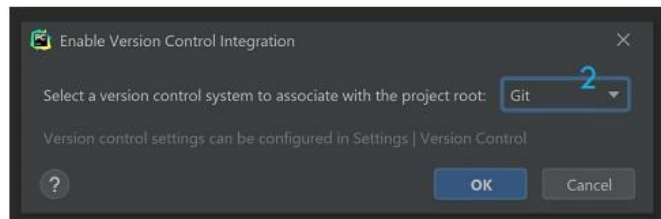
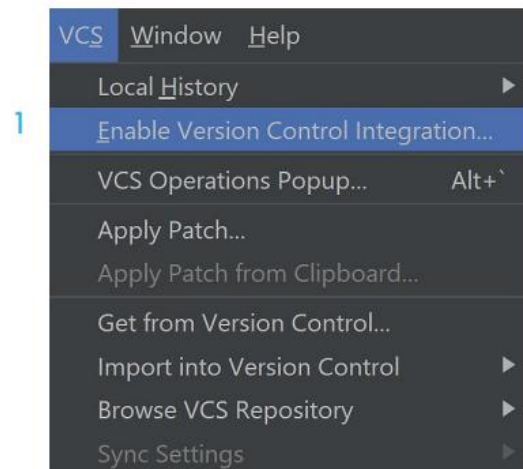


In verde vedrai le differenze, le modifiche però sono tutte locali.

MANAGE PROJECTS WITH GITHUB

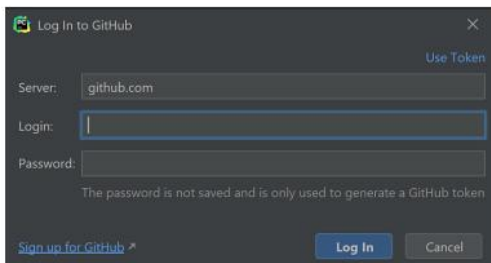
GitHub is way more convenient especially when a project is developed in group. Now you need to:

1. Install Git using only the default settings in the installation process
2. After VC integration is enabled (1 and 2), PyCharm will ask you whether you want to share project settings files via VCS.
3. In the dialog you can choose "**Always Add**" to synchronize project settings with other repository users who work with PyCharm.

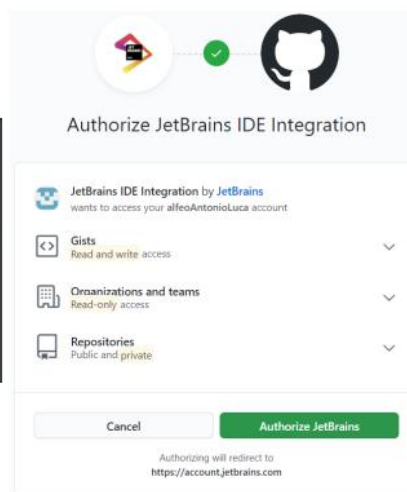


ENABLE VCI TO USE GITHUB

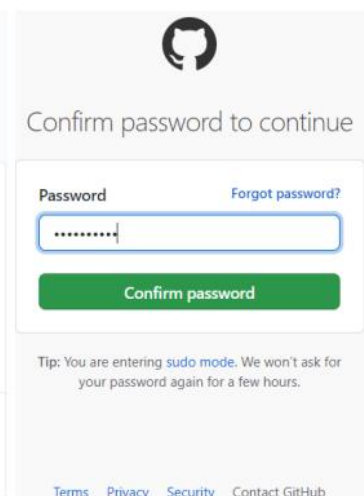
Login into GitHub, or request a new access token with your login and password (1). Then, authorize JetBrains IDE integration (2 and 3).



1



2

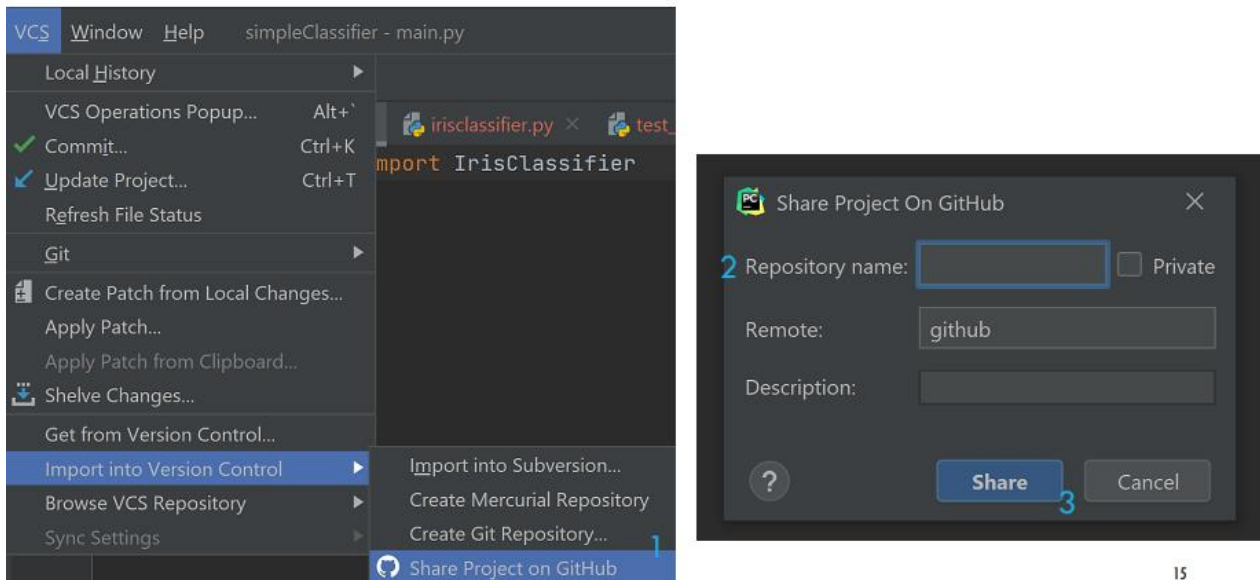


3

14

SHARE A PROJECT 1/2

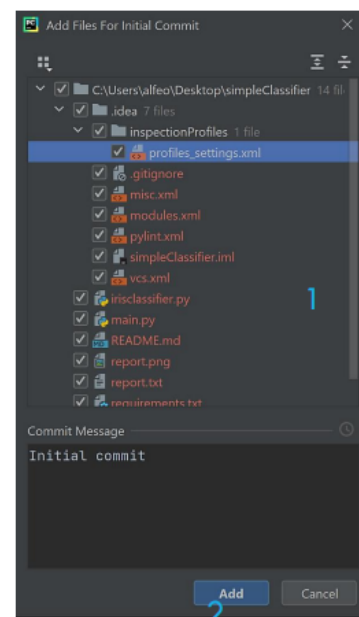
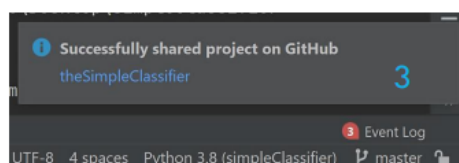
1. When connection to GitHub has been established, the Share Project on GitHub dialog appears in VCS > Import Version Control > Share ...(1).
2. Name the repository (2) and click share (3)



15

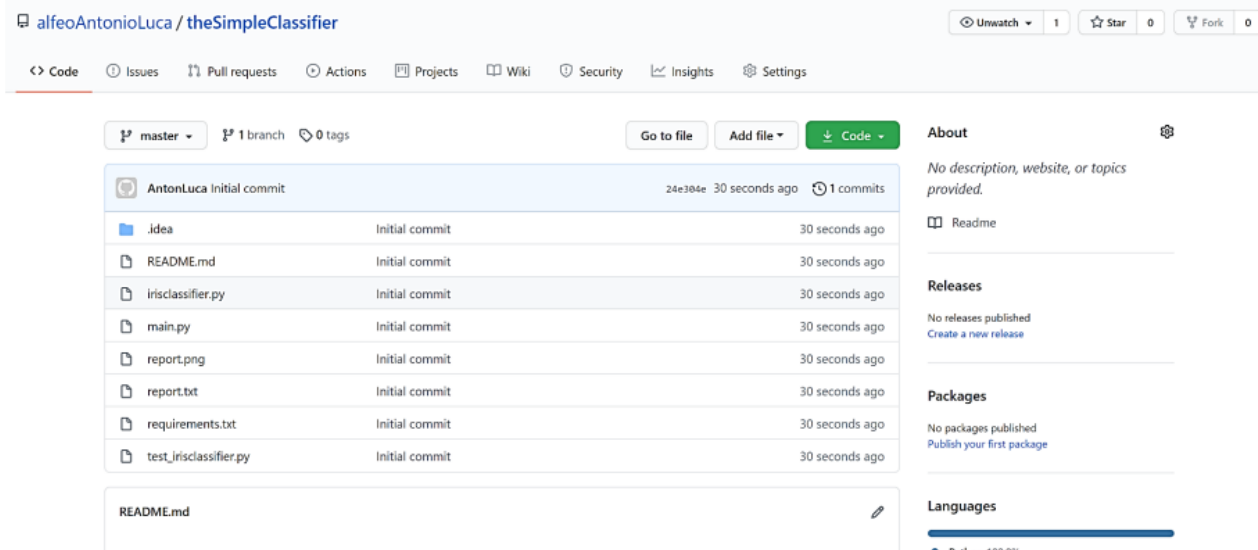
SHARE A PROJECT 2/2

1. Being the initial commit, of the project every file is highlighted in red (they are not in the VCS).
2. Click Add
3. PyCharm will notify once the process is finalized



Tutti i file in rosso significa che non sono stati caricati su github, per metterli devi fare commit e poi push.

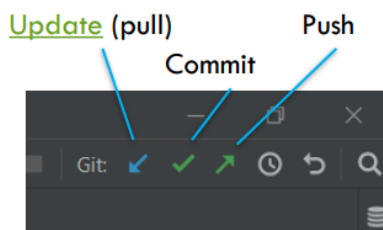
... ON GITHUB



Let's say you have a **central repository** for your project, the contribution of each coder will be highlighted by the commits she/he made. In the next slide you will see how to “**sign**” your commit °

17

COMMIT AND PUSH

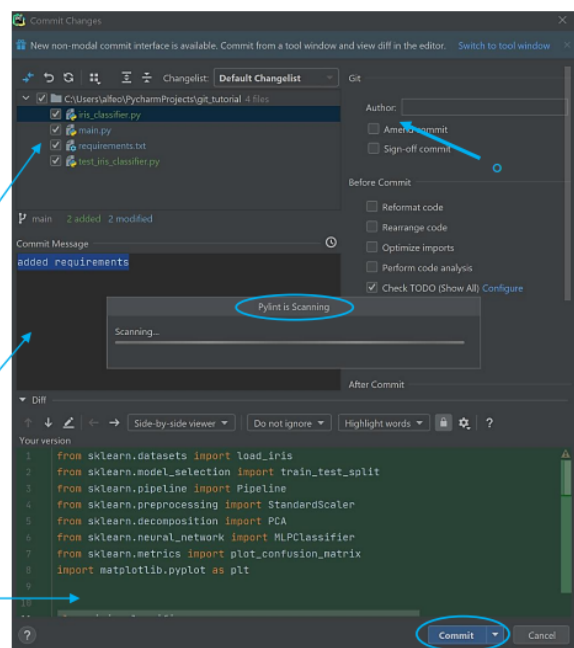


Color code:

- Blue: modified
- Green: commit done
- Red: not in the VCS
- White: push done

Explain clearly the change provided

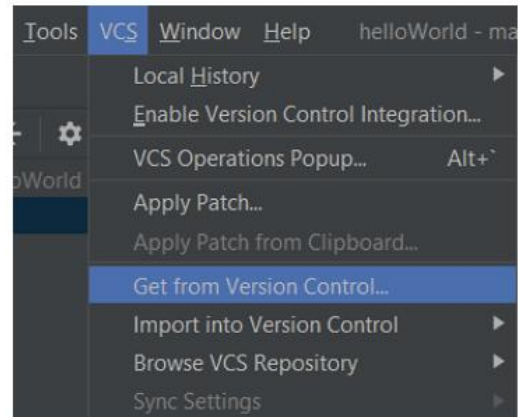
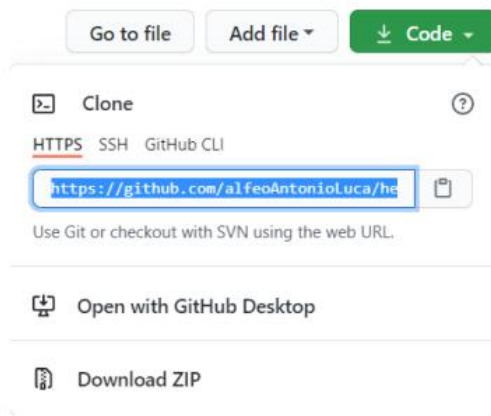
After the first commit here there will be the “before and after” comparison



- In rosso significa che è solo in locale.
- Blu: modificato ma non caricato

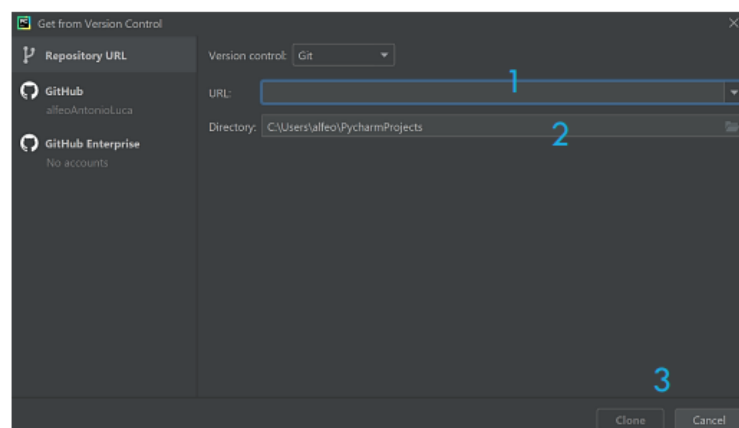
CLONE FROM GIT 1/2

1. Go to GitHub and annotate the **URL** of the repository you are interested in (i.e. the project repo we will provide you with).
2. From the main menu, choose **VCS > Get from Version Control**, and choose **GitHub** on the left.



CLONE FROM GIT 2/2

1. Specify the URL of the repository that you want to clone (the one you annotated from GitHub).
2. In the Directory field, enter the path to the folder where your local Git repository will be created.
3. Click **Clone**. If you want to create a project based on these sources, click Yes in the confirmation dialog. PyCharm will automatically set Git root mapping to the project root directory.



MORE ON PYCHARM VCS

PyCharm provides [version control integration](#), a unified user interface for many VCS such as [GitHub](#).

It allows you to [contribute to a GitHub project](#) via commit, push, pull, change lists, cloning... and even branch, merge, and conflict via [Git](#).

Here you have a couple of video-tutorials [[1](#), [2](#), [3](#)]

12. Dalle performance fino alla productivity evaluation

Le metriche si misurano con esperimenti o a tavolino (con dei calcoli). Ora vediamo quelle sperimentali:

bisogna misurare se l'implementazione è scalabile e robusta prima della fase di produzione

“NON” → 0 è perfetto, tutta automatizzata, meno lo è più è manuale e più costa

A paradigm shift from optimizing performances to optimizing productivity.

Metrics:

1) **non-automation** (cognitive effort)

Consider each human step of the BPMN process, described in a use case, and give a 1-to-6 score to the step, according to the cognitive taxonomy (appendix A): *remember* (1), *understand* (2), *apply* (3), *analyze* (4), *evaluate* (5), and *create* (6). Multiply the score for the # of steps of the use case, for the percentage of occurrence (in case of alternatives), and for the salary of the actor.

2) **non-responsiveness** (time)

Every begin-to-end path in the BPMN process (a scenario) is measured from time perspective, by calculating the initial and final instants; the average time is calculated over the different scenarios and multiplied by their percentage of occurrence.

3) **non-elasticity** (data load)

To feed the system with increasing no. of inputs; consider the frequency of each path; a plot of the processing time is generated, for each value of considered

1 → l'obiettivo del software è sempre quello di automatizzare, vogliamo misurare quanta automazione c'è nella fabbrica. La fabbrica potrebbe essere autoconfigurabile, quando c'è da fare il deployment, quando il classificatore è pronto e si passa da development a production, ora è manuale, ma potrebbe esserci un messaggio di configurazione che viene mandato ad un sistema di configurazione vero e proprio che riceve e manda dei messaggi ai vari sistemi per farli autoconfigurare (esempio di automazione). Per misurare l'automazione si prende ogni caso d'uso sviluppato, ogni step del caso d'uso ha un costo, sommando poi i costi si ha il costo cognitivo totale (quanto sforzo umano serve per il caso d'uso, quindi se automatizziamo diminuiamo questo sforzo cognitivo se non è più alto è peggio). Il costo si ottiene come moltiplicazione del salario, dello sforzo cognitivo e delle occorrenze. Se ho un ciclo for che viene fatto un certo numero di volte, devo moltiplicare per il numero di volte del ciclo, se un if ho l'occorrenza (0.8 probabilità if, 0.2 else).

Salario: ruoli che mettiamo nel caso d'uso: data analyst, ... I ruoli vengono descritti nei link, si vedono i salari e le competenze (il machine learning engineer vede la curva di training ad esempio). Si divide per il salario minimo. Se ci metto un tecnico invece che un laureato costa di meno. Con la moltiplicazione spieghiamo lo sforzo cognitivo.

Lo sforzo cognitivo si basa su 6 livelli:

- Livello 1 → il passo è lo stesso ogni volta, quando clicco su link, faccio copia incolla, quindi l'azione mentale è sempre la stessa.
- Livello 2 → non faccio ogni volta la stessa cosa ma devo esaminare una scelta tra un numero predefinito di categoria, tipo il record del quality check, il software mi fa vedere quale soglia (verde o rossa) è stata sforata, quindi per scegliere basta guardare il colore, il resto mi serve per capire il perché. Si misura il costo cognitivo nominale, nel caso standard, per decidere, vedo se una delle due soglie è rossa quindi non va bene il classificatore.
- Livello 3 → applico delle regole, guardo quante generazioni ho fatto con la curva piatta o non piatta. Ogni regola vale 3. Svolgo regole magari matematiche, osservo numeri, li confronto, vedo se è più alto o più basso.
- Livello 4 → livello più alto della nostra fabbrica, perché non fa innovazione. Gli operai al massimo arrivano a 4. Si chiama "analizza", si usa l'esperienza umana, non si possono inferire delle regole, quando analizzo il plot della distribuzione delle feature, guardo sulla base dell'esperienza se la distribuzione è corretta o no, la decisione è multi-criteriale, non bloccata da regole.
- Livello 5 → innovazione

Ora va indicato quale Human operator è, il ruolo professionale

Si possono avere attori primari e secondari in un singolo caso d'uso.

Quando si passa a production, viene scritto un messaggio "passa al production", gli altri fanno copia incolla, solo uno sostiene il costo di scrittura del valore, per quella persona il costo c'è per gli altri il costo è 1, non si pensa alla cosa.

Il costo 4 deve essere una decisione con più criteri, per esempio nel nostro progetto: la distribution dei valori che non rispetta la distribuzione attesa, è quindi di livello 4, ma va moltiplicato per il numero di criteri a cui l'essere umano pensa. Environment, activity che hanno una certa distribuzione. Ci sono 5 environment, 5 possibilità, non si evince bene dal nostro diagramma. Sono stati raccolti dati su 1 o 2 pendenze, l'osservatore deve dire: "le altre pendenze appariranno o non appariranno mai?". Se possono venire dobbiamo raccogliere nuovi dati, quindi per decidere se vengono o meno devo fare un questionario: "che attività fai nella tua routing mensile?", "su che ambienti ti sposti". Devo capire se i dati che ho raccolto corrispondono alla distribuzione per vedere se è corretta. Magari se un utente ha l'alzheimer le bande dell'helmet sono influenzate. Ho quindi una serie di criteri (patologie, ambiente, attività..) sulle quali mi baso per capire se le distribuzioni sono corrette o meno. Quindi devo moltiplicare 4 per 3 (quelle trovate fino ad ora), ma limitiamoci a 3. $4 \times 3 \times (\text{occorrenze}) \times (\text{stipendio})$

Occorrenze per ogni feature (sia quelle del for each che del for).

Dove non abbiamo regole (secondo esperienza). Moltiplichiamo il costo cognitivo 4 per 3 (deciso ora), quest'ultimo è il numero di criteri.

2 → bisogna vedere se la fabbrica è responsive, cioè se è in tempo reale in base al contesto, tipo la sedia a rotelle deve rispondere in millisecondi. Ognuno ha il suo dominio applicativo, siamo una catena con diversi sistemi applicativi. C'è uno script python che simula il lato client, accede ad un file csv, legge i record e li spara all'ingestion, che li elabora e li manda al prossimo. Quando il dato è letto, mette in un log il tempo, da quando manda il primo record si salva il tempo. Nel csv con il record 41 c'è una marca temporale di invio e si calcola in tempo con il ritorno. Dobbiamo solo fare una misura, non dobbiamo velocizzare. Faccio 20/30 misure, bastano anche 10 valori per fare il valore medio. Nel csv si mette marca temporale inizio e fine, si mette in excel e si fa la media. Il valor medio si mette nella relazione. Se è 3 secondi si vede il perché, qual è il sistema applicativo collo di bottiglia.

3 → richieste incremental, elasticità è nel carico non nel tempo. Si incrementa il carico alla CPU per vedere come reagisce il pc. Si leggono a raffica gli stessi dati e li spariamo come fossero tanti client che li mandano. Il tempo cresce non linearmente. C'è una linea dopo cui non è più responsive, per il brain è 1 secondo sennò non riesco a comandarla. Non bisogna arrivare per forza al ginocchio, poi ci aggiusteremo.

Misuriamo la catena: marca dal primo classificatore, poi si manda la notifica del classificatore completato. Noi misuriamo la fabbrica e facciamo una interfaccia per misurarla.

inputs. Evaluate the trend (linear, quadratic, cubic, ...) and motivate. A horizontal line at the value of responsiveness highlights the threshold of responsiveness.

4) **non-resiliency** (data error)

The system is tested with various types of wrong input (missing samples, late samples, unbalanced classes, ..); a 1-to-5 score is provided according to the level of management: 5- the error is not detected, 4- the error is detected and filtered, 3- the error is detected and an alert is sent, 2- the error is managed and classified, 1- the error is solved automatically. Calculate the summation of the scores.

5) **non-interoperability** (non-smooth communication)

To produce the different I/O between applications and to check whether all of them are considered in the entire toolchain. Increase by one this numeric score if the application does not take into account some feature considered by the previous/next application. Compute the summation of scores.

4 → se mancano campioni vengono interpolati, ottima resilienza (0). Dobbiamo misurare le cose, dare motivazioni per problemi. “Gestisci e classifica” dà indicazione per fare la manutenzione in maniera effettiva. Proprietà di un software di reagire quando ci sono errori, dobbiamo solo capire il livello e il perché, non massimizzarla. Filtrato indica che butto via l’input, non indica che l’errore è risolto, se arrivano troppi campioni mancanti mi portano a perdere quel dato.

Resilienza: il sistema riesce a gestire gli errori, l’obiettivo è generare una serie di errori nell’input, ci possono essere 4/5 errori, alcuni vengono gestiti, altri danno eccezioni. Si danno score da 1 a 5, 1 è molto resiliente, l’errore è gestito automaticamente. Se mancano tot sample il sistema li interpola, quindi score 1. Se l’errore viene gestito e classificato (faccio delle categorie, da noi però non c’è questo), magari se i tipi di errori sono frequenti, l’errore c’è ma non viene risolto, viene rilevato, un messaggio viene mandato all’utente. Il punto 3: errore rilevato e inviata una allerta (non abbiamo). Nel 4 l’errore viene rilevato e filtrato, per esempio quando ho troppi sample mancanti, butto la sessione, quindi filtro l’errore per evitare che crei casino, nel 5 invece si genera una eccezione, causa problemi magari porta ai dati inconsistenti ecc.

Etichette mancanti, sample mancanti, record duplicati.

Casi e conseguenze, a tavolino

3/4/5

5 → software A non ha messo certe funzioni perché non ha tempo, chi mette le cose chi no, si fa la tabella di interoperabilità, si aggiungono i punti e poi si fa la somma. Se uno è pronto e uno no vedi e assegni l’interoperabilità.

Avendo alcuni applicativi più complessi e altri meno complessi, alcuni hanno fatto delle semplificazioni, non abbiano implementato interamente tutte le feature.

Applicativi A che sono pronti per alcune feature, applicativi B che non le sfruttano interamente.

Bilanciamento dei dati (feature) : Quando abbiamo il data balancing si chiedono nuovi dati per bilanciare, questi dati vengono richiesti quindi arrivano nuove sessioni ma non è detto che il segregation sia in grado di collegare dati nuovi e dati vecchi, fa confusione magari (magari è predisposto per averli uno dopo l'altro, fa un segregation set di un classificatore, se poi arriva un altro set di dati di quello prima non riesce a collegarli, li mette in quello nuovo (di classificatore)). Non vanno sistemati i problemi, gli errori, bisogna ben misurare la metrica e motivarla, mettiamo in chiaro tutti i limiti del nostro software. Quindi in questo caso non sono interoperabili perché l'ingestion a riguardo del bilanciamento è interoperabile, quando arrivano nuove sessioni, le prende e le manda, il preparation pure ma il segregation no, il development è interoperabile anch'esso, pronto per ricevere un set più grande e sviluppare una nuova rete.

Ogni sistema che non è interoperabile porta ad un punto da sommare. Si possono pensare a feature simili a quella della resilienza, tipo alcuni mandano dati duplicati, alcuni riescono a gestirli altri no.

Quali sistemi non sono interoperabili, producendo problemi nella gestione di una feature.

La feature può essere una funzionalità prevista nel flusso, oppure un qualcosa che è un errore, poterlo gestire, qual è il sistema che è in grado di interoperare con gli altri per gestire quell'errore.

Multiple clients send different records to the system at the same time: l'applicativo è pensato per avere dati di un classificatore alla volta, poi di un altro ecc (non averli mescolati). Se invece arrivassero mescolati, quali sono preparati a questo scopo e quali no.

Retrieve new sessions if data coverage is not satisfying: se il data coverage non è soddisfatto e si chiedono nuovi dati, simile a quello di prima, arrivano nuovi dati e li ricollegiamo (identico).

Cercare quali sistemi non sono interoperabili con gli altri per risolvere le feature.

3/4/5

Automazione porta a richiedere meno capacità umane.

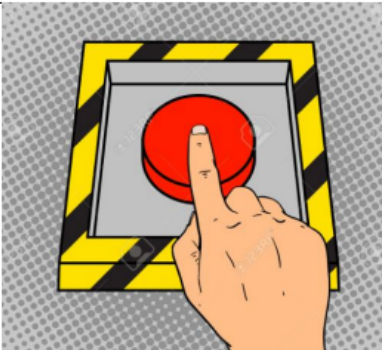
Non automation → valori incrementali abbiamo più umani coinvolti. Nella nostra factory ci sono umani, non dobbiamo rimuovere gli umani, non dobbiamo automatizzare troppo. Gli umani sono necessari in particolare nelle decisioni, gli umani vanno tenuti. Dobbiamo capire come misurare lo sforzo umano.

Abbiamo task umani, gli umani guardano i plot, pensano e decidono. Dobbiamo misurare lo sforzo degli umani, utilizziamo un metodo standard basato in uno score da 1 a 6. Non stiamo innovando,

abbiamo machine learning classifier e stiamo standardizzando le componenti, arriviamo max a livello 4, perché poi devi innovare a livello 5 e 6, noi standardizziamo le decisioni.

Quando premi un bottone per aprire alcune finestre, il task è sempre lo stesso “premere quel bottone”, oppure copiare ed incollare un indirizzo ip. Quando l’attività è la stessa lo sforzo è 1.

Appendix A: cognitive taxonomy (Anderson)

1 – remember - <i>definition: the step can be carried out by remembering another occurrence of the same step;</i> - <i>example: to press a button, to select a panel, and so on, need to remember the same step for any occurrence, or to read the manual;</i>																												
2 – understand - <i>the step can be carried out by finding a value in a set of predefined categories;</i> - <i>example: the result of a quality check report automatically computed via two thresholds: “satisfied” in green color (1st category) or “not satisfied” in red color (2nd category, red). The human understands two possible outputs, green or red.</i>	<table><tr><th colspan="3">QUALITY CHECK ERROR REPORT</th></tr><tr><th>Expert label class</th><th>Anomaly detector label class</th><th>Result</th></tr><tr><td>1</td><td>2</td><td>✗</td></tr><tr><td>2</td><td>2</td><td>✓</td></tr><tr><td>3</td><td>3</td><td>✓</td></tr><tr><td>3</td><td>1</td><td>✗</td></tr><tr><td>4</td><td>5</td><td>✗</td></tr><tr><td>3</td><td>4</td><td>✗</td></tr><tr><td>2</td><td>2</td><td>✓</td></tr></table> Max number of errors tolerated (th1): 5 Max number of <u>consecutive</u> errors tolerated (th2): 2 Total errors: 4 Max consecutive errors: 3 th1 satisfied (4 < 5) th2 exceeded (3 > 2)	QUALITY CHECK ERROR REPORT			Expert label class	Anomaly detector label class	Result	1	2	✗	2	2	✓	3	3	✓	3	1	✗	4	5	✗	3	4	✗	2	2	✓
QUALITY CHECK ERROR REPORT																												
Expert label class	Anomaly detector label class	Result																										
1	2	✗																										
2	2	✓																										
3	3	✓																										
3	1	✗																										
4	5	✗																										
3	4	✗																										
2	2	✓																										

Premere il bottone → 1, semplice

Alcune categorie sono predefinite, ci sono delle computazioni, errori, ecc. il sistema calcola i vari errori e altre cose. Il risultato è verde o rosso. Gli umani investigano perché è verde o rosso, capiscono perché il sistema suggerisce quel risultato, non è parte dello use case.

Guardare il diagramma → livello cognitivo 2

3 – apply

- the step can be carried out by executing a predefined procedure, encoded by the company.

- for example, to determine whether the maximum number of iterations is good or not requires a procedure, and each step is at level 3:

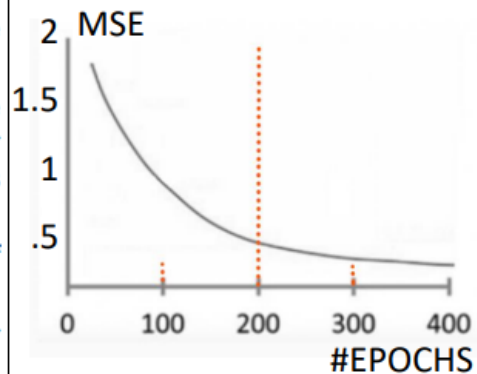
(a) if the loss is flat for at least the half of the iterations,

(a.1) then reduce by one third the number of iterations to manage overfitting;

(b) if the loss is not flat at the end of the iterations,

(b.1) then enlarge by one third the number of iterations;

(c) otherwise, the no. of iterations is good.



Se puoi stabilire un certo numero di regole, 6/7 il livello di effort è 3.

4 – analyze

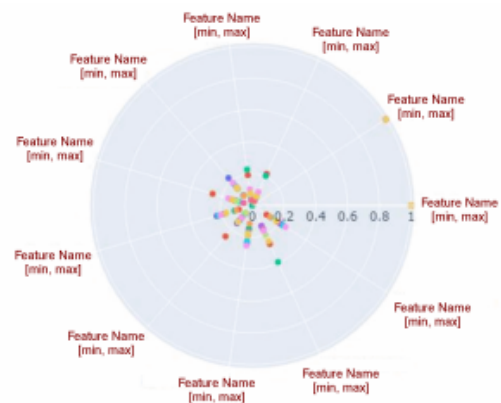
- the step can be carried out by finding unknown categories;

- for example, to analyze the radar diagram for assessing the input coverage;

(a) for each radius (feature)

(a.1) if the distribution is not uniform as expected

(a.1.1) then input coverage is not satisfied



5 – evaluate (n.a. in this factory)

- to detect (in-)consistencies between a product and external criteria;

- to detect the appropriateness of a procedure for a given problem

6 – create (n.a. in this factory)

- to come up with alternative hypotheses based on criteria;

- to devise a procedure for accomplishing some task, to invent a product.

La decisione è fatta da esperienza, non possono essere definite delle regole. Guardi il diagramma e decidi. Per esempio, pensare che sia “uniforme come ci si aspetta” devi soddisfare alcuni criteri, magari devi pensare il livello di malattia e la giovinezza della persona in sedia a rotelle.

Per ogni step assegniamo un certo costo, poi si moltiplica il costo per il numero di criteri. In base ai segnali del cervello, la distribuzione cambia anche per via delle caratteristiche del paziente.